

ĐẠI HỌC BÁCH KHOA HÀ NỘI

*

BÁO CÁO HỌC PHẦN

PROJECT 1



SOICT

TÊN ĐỀ TÀI

HỆ THỐNG QUẢN LÝ CÔNG VIỆC

Giảng viên hướng dẫn : Th.S Phạm Quang Hiếu
Sinh viên thực hiện : Nguyễn Trọng Nhân – 202490073
Lớp : CNTT 1.1 - K69

Hà Nội 05 - 2026

Mục lục

Lời cảm ơn	6
1 CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI	7
1.1 Giới thiệu đề tài	7
1.2 Tính cấp thiết của đề tài	7
1.3 Mục tiêu đề tài	8
1.4 Phạm vi đề tài	8
1.5 Đối tượng sử dụng	9
2 CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	11
2.1 Cách tiếp cận và hướng giải quyết	11
2.1.1 Phương pháp phân tích bài toán	11
2.1.2 Hướng thiết kế giải pháp	11
2.1.3 Các quyết định kỹ thuật chính	12
2.2 Các nền tảng và công nghệ sử dụng	12
2.2.1 Các nền tảng liên quan đến ứng dụng	12
2.2.2 Các nền tảng liên quan đến dịch vụ đám mây	15
2.2.3 Các nền tảng khác	18
3 CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	21
3.1 Phân tích hệ thống	21
3.1.1 Khảo sát và xác định yêu cầu	21
3.1.2 Xác định tác nhân và chức năng hệ thống	24
3.1.3 Đặc tả Use Case	29
3.1.4 Phân tích luồng nghiệp vụ	56
3.1.5 Phân tích dữ liệu	92
3.2 Thiết kế hệ thống	94
3.2.1 Thiết kế kiến trúc tổng thể	94
3.2.2 Thiết kế tương tác giữa các thành phần	96
3.2.3 Thiết kế cơ sở dữ liệu	111
3.2.4 Thiết kế giao diện	125
4 CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ	126
4.1 Các chức năng đã hoàn thành	126
4.2 Kết quả đạt được	127
4.2.1 Thống kê tổng quát	127
4.2.2 Kết quả kiểm thử theo module	127
4.2.3 Các màn hình giao diện đã hoàn thành	129
4.3 Hình ảnh minh họa	133

4.4	Đánh giá kết quả	134
4.4.1	Ưu điểm	134
4.4.2	Hạn chế	134
5	CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	136
5.1	Kết luận	136
5.1.1	Kết quả đạt được	136
5.2	Các vấn đề còn tồn tại	137
5.2.1	Chức năng chưa hoàn thiện	137
5.2.2	Hạn chế kỹ thuật	137
5.3	Phương hướng phát triển	138
5.3.1	Ưu tiên cao — Hoàn thiện hệ thống hiện tại	138
5.3.2	Ưu tiên trung bình — Mở rộng tính năng	139
5.3.3	Ưu tiên thấp — Phát triển dài hạn	139
A	PHỤ LỤC A: DANH SÁCH API ENDPOINT ĐẦY ĐỦ	141
B	PHỤ LỤC B: CHI TIẾT BẢNG CƠ SỞ DỮ LIỆU	143
C	PHỤ LỤC C: DANH SÁCH KỊCH BẢN KIỂM THỬ	149

Danh sách hình vẽ

1	Logo ReactJS	13
2	Logo Java Spring Boot	14
3	Logo PostgreSQL	15
4	Logo Docker	19
5	Logo Nginx	19
6	Biểu đồ Use Case tổng quát hệ thống Nello	26
7	Biểu đồ Use Case UC-01: Quản lý xác thực	29
8	Biểu đồ Use Case UC-02: Quản lý người dùng	33
9	Biểu đồ Use Case UC-03: Quản lý Workspace	35
10	Biểu đồ Use Case UC-04: Quản lý Board	39
11	Biểu đồ Use Case UC-05: Quản lý List	43
12	Biểu đồ Use Case UC-06: Quản lý Card	46
13	Biểu đồ Use Case UC-07: Quản lý Checklist	50
14	Biểu đồ Use Case UC-08: Quản lý thành viên Card	54
15	Biểu đồ hoạt động UC-01.1: Đăng ký tài khoản	57
16	Biểu đồ hoạt động UC-01.2: Xác nhận tài khoản bằng OTP	58
17	Biểu đồ hoạt động UC-01.3: Gửi lại mã OTP	59
18	Biểu đồ hoạt động UC-01.4: Đăng nhập	60
19	Biểu đồ hoạt động UC-01.5: Làm mới access token	61
20	Biểu đồ hoạt động UC-02.1: Xem thông tin cá nhân	62
21	Biểu đồ hoạt động UC-02.2: Chỉnh sửa thông tin cá nhân	63
22	Biểu đồ hoạt động UC-03.1: Tạo Workspace	64
23	Biểu đồ hoạt động UC-03.2: Xem danh sách Workspace	65
24	Biểu đồ hoạt động UC-03.3: Xem chi tiết Workspace	66
25	Biểu đồ hoạt động UC-03.4: Mời thành viên vào Workspace	67
26	Biểu đồ hoạt động UC-03.5: Xem danh sách thành viên Workspace	68
27	Biểu đồ hoạt động UC-04.1: Tạo Board	69
28	Biểu đồ hoạt động UC-04.2: Xem chi tiết Board	70
29	Biểu đồ hoạt động UC-04.3: Cập nhật tên Board	71
30	Biểu đồ hoạt động UC-04.4: Xóa Board	72
31	Biểu đồ hoạt động UC-04.5: Thêm thành viên vào Board	73
32	Biểu đồ hoạt động UC-05.1: Tạo List	74
33	Biểu đồ hoạt động UC-05.2: Đổi tên List	75
34	Biểu đồ hoạt động UC-05.3: Xóa List	76
35	Biểu đồ hoạt động UC-05.4: Sắp xếp thứ tự List	77
36	Biểu đồ hoạt động UC-06.1: Tạo Card	78
37	Biểu đồ hoạt động UC-06.2: Xem chi tiết Card	79
38	Biểu đồ hoạt động UC-06.3: Cập nhật Card	80

39	Biểu đồ hoạt động UC-06.4: Đánh dấu Card hoàn thành . . .	81
40	Biểu đồ hoạt động UC-06.5: Di chuyển Card giữa các List . .	82
41	Biểu đồ hoạt động UC-06.6: Xóa Card	83
42	Biểu đồ hoạt động UC-07.1: Tạo Checklist	84
43	Biểu đồ hoạt động UC-07.2: Cập nhật Checklist	85
44	Biểu đồ hoạt động UC-07.3: Xóa Checklist	86
45	Biểu đồ hoạt động UC-07.4: Tạo Checklist Item	87
46	Biểu đồ hoạt động UC-07.5: Cập nhật Checklist Item	88
47	Biểu đồ hoạt động UC-07.6: Đánh dấu Checklist Item hoàn thành	89
48	Biểu đồ hoạt động UC-07.7: Xóa Checklist Item	90
49	Biểu đồ hoạt động UC-08.1: Gán thành viên vào Card	91
50	Biểu đồ hoạt động UC-08.2: Hủy gán thành viên khỏi Card . .	92
51	Biểu đồ thành phần hệ thống Nello	95
52	Kiến trúc triển khai hệ thống Nello trên AWS	96
53	Biểu đồ tuần tự SD-01: Luồng đăng ký tài khoản	97
54	Biểu đồ tuần tự SD-02: Luồng đăng nhập	98
55	Biểu đồ tuần tự SD-03: Luồng tạo Workspace	99
56	Biểu đồ tuần tự SD-04: Luồng tạo Board	100
57	Biểu đồ tuần tự SD-05: Luồng tạo List	101
58	Biểu đồ tuần tự SD-06: Luồng tạo Card	102
59	Biểu đồ tuần tự SD-07: Luồng tải chi tiết Board	104
60	Biểu đồ tuần tự SD-08: Luồng thêm thành viên vào Board . .	106
61	Biểu đồ tuần tự SD-09: Luồng di chuyển Card (Drag-and-Drop)	108
62	Biểu đồ tuần tự SD-10: Luồng di chuyển List (Drag-and-Drop)	110
63	Sơ đồ quan hệ cơ sở dữ liệu hệ thống Nello	112
64	Màn hình Đăng ký tài khoản	130
65	Màn hình Xác nhận OTP	130
66	Màn hình Đăng nhập	131
67	Màn hình Dashboard	131
68	Màn hình Chi tiết Workspace	132
69	Màn hình Board Kanban	132
70	Tính năng kéo thả Card giữa các List	133
71	Modal Chi tiết Card với Checklist, Due Date và thành viên . .	133

Danh sách bảng

1	Yêu cầu chức năng nhóm 1	21
2	Yêu cầu chức năng nhóm 2	22
3	Yêu cầu chức năng nhóm 3	22
4	Yêu cầu chức năng nhóm 4	22
5	Yêu cầu chức năng nhóm 5	23
6	Yêu cầu chức năng nhóm 6	23
7	Yêu cầu chức năng nhóm 7	23
8	Danh sách Use Case	28
9	Bảng <code>roles</code>	115
10	Bảng <code>images</code>	115
11	Bảng <code>workspace_categories</code>	115
12	Bảng <code>users</code>	116
13	Bảng <code>workspaces</code>	117
14	Bảng <code>workspace_members</code>	118
15	Bảng <code>boards</code>	119
16	Bảng <code>board_members</code>	120
17	Bảng <code>lists</code>	121
18	Bảng <code>cards</code>	122
19	Bảng <code>card_members</code>	123
20	Bảng <code>checklists</code>	124
21	Bảng <code>checklist_items</code>	125
22	Danh sách chức năng đã hoàn thành (27/31 — 87,1%)	127
23	Tổng hợp kết quả thực hiện	127
24	Kết quả kiểm thử module Authentication	128
25	Kết quả kiểm thử module Workspace & Board	128
26	Kết quả kiểm thử module Card & Checklist	128
27	Kết quả kiểm thử tích hợp Frontend (W6)	129
28	Kết quả kiểm thử giao diện Board (W7)	129
29	Tổng hợp kết quả học phần Project I	136
30	Các tính năng chưa được triển khai	137
31	Danh sách API endpoint đầy đủ của hệ thống	142

Lời cảm ơn

Em xin chân thành cảm ơn giảng viên hướng dẫn, Th.S Phạm Quang Hiếu, đã tận tình hướng dẫn và hỗ trợ em trong suốt quá trình thực hiện học phần Project 1.

Em cũng xin cảm ơn Trường Công nghệ Thông tin và Truyền thông, Đại học Bách Khoa Hà Nội đã tạo điều kiện học tập và nghiên cứu thuận lợi.

Trong quá trình thực hiện báo cáo, do kiến thức và kinh nghiệm còn hạn chế, không tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý từ thầy cô để có thể hoàn thiện hơn trong thời gian tới.

Sinh viên thực hiện

Nguyễn Trọng Nhân

1 CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu đề tài

Trong quá trình quản lý dự án, việc tổ chức và theo dõi công việc luôn là một bài toán phức tạp đối với các cá nhân hay tổ chức. Khi số lượng công việc gia tăng, các phương pháp quản lý truyền thống hoặc rời rạc thường bộc lộ nhiều hạn chế, dẫn đến những khó khăn trực tiếp trong việc kiểm soát tiến độ, phân chia nguồn lực và duy trì sự phối hợp giữa các thành viên. Tình trạng thất lạc thông tin, thiếu minh bạch về trạng thái thực hiện các hạng mục công việc làm suy giảm đáng kể hiệu suất chung. Xuất phát từ thực trạng này, bài toán đặt ra là cần xây dựng một giải pháp phần mềm hỗ trợ quản lý dự án tập trung, trực quan, giúp người dùng dễ dàng tổ chức các đầu việc, theo dõi tiến độ và nâng cao hiệu quả cộng tác.

1.2 Tính cấp thiết của đề tài

Hiện nay, mặc dù đã có nhiều giải pháp phần mềm hỗ trợ quản lý, nhiều cá nhân và tổ chức tại Việt Nam vẫn sử dụng các phương pháp thủ công như ghi chép sổ tay, bảng trắng, hoặc các công cụ kỹ thuật số rời rạc và thiếu tính trực quan như email, bảng tính Excel, hay các ứng dụng nhắn tin nhanh. Các phương thức tiếp cận truyền thống này bộc lộ nhiều hạn chế nghiêm trọng trong môi trường làm việc hiện đại:

- **Thất lạc và phân mảnh thông tin:** Các yêu cầu công việc, phản hồi và tài liệu đính kèm bị phân tán trên nhiều kênh giao tiếp khác nhau, dẫn đến khó khăn trong việc tìm kiếm và dễ bỏ sót nhiệm vụ.
- **Khó theo dõi tiến độ tổng thể:** Người quản lý và các thành viên gặp nhiều khó khăn để có được cái nhìn tổng quan về trạng thái hiện tại của dự án, dẫn đến chậm trễ tiến độ mà không có biện pháp cảnh báo hoặc xử lý kịp thời.
- **Thiếu tính tương tác trực quan:** Các công cụ truyền thống không hỗ trợ cập nhật động trạng thái công việc và luồng di chuyển của các nhiệm vụ theo thời gian thực.
- **Giảm hiệu suất cộng tác nhóm:** Việc phân chia trách nhiệm và giao việc không được định nghĩa rõ ràng, gây chồng chéo công việc và suy giảm hiệu quả làm việc chung.

Từ những hạn chế trên, việc nghiên cứu và xây dựng “Hệ thống quản lý công việc” là vô cùng cấp thiết. Hệ thống được phát triển nhằm khắc phục các

nhược điểm của phương pháp truyền thống bằng việc cung cấp một giao diện quản lý trực quan theo mô hình Kanban. Giải pháp này giúp chuẩn hóa quy trình giao việc, nâng cao tính minh bạch trong tập thể, tối ưu hóa quy trình làm việc và giảm thiểu tối đa các sai sót thông tin trong suốt quá trình vận hành dự án.

1.3 Mục tiêu đề tài

Mục tiêu chính của đề tài là nghiên cứu, thiết kế và phát triển một nền tảng quản lý công việc và thẻ công việc (task/card) hiệu quả, trực quan, dễ tiếp cận và có khả năng tương tác cao. Các mục tiêu chi tiết bao gồm:

- **Xây dựng nền tảng quản lý khoa học:** Cho phép người dùng dễ dàng tổ chức công việc theo các không gian làm việc (workspace) riêng biệt và các bảng công việc (board) theo từng dự án cụ thể.
- **Quản lý thẻ công việc chi tiết:** Cung cấp đầy đủ các thuộc tính quản lý cho một nhiệm vụ như tiêu đề, mô tả chi tiết, gán thành viên thực hiện, thiết lập thời hạn hoàn thành (due date) và xây dựng các danh sách kiểm tra công việc con (checklist) để theo dõi chi tiết.
- **Tích hợp cơ chế tương tác trực quan:** Triển khai thành công tính năng kéo thả (drag and drop) các thẻ công việc giữa các danh sách trạng thái khác nhau, mang lại trải nghiệm mượt mà và trực quan cho người dùng.
- **Đồng bộ dữ liệu và cộng tác hiệu quả:** Đảm bảo các cập nhật về trạng thái công việc, phân công nhân sự và thời hạn được đồng bộ hóa nhanh chóng đến toàn bộ thành viên trong bảng công việc.

1.4 Phạm vi đề tài

Đề tài tập trung nghiên cứu và triển khai các chức năng cốt lõi phục vụ nhu cầu quản lý công việc hàng ngày của cá nhân và đội ngũ, cụ thể bao gồm:

- **Hệ thống xác thực và quản lý tài khoản:** Triển khai các tính năng đăng ký, đăng nhập, đăng xuất và cập nhật thông tin cá nhân của người dùng, tích hợp giải pháp xác thực bảo mật thông qua dịch vụ điện toán đám mây Amazon Cognito.
- **Quản lý Workspace:** Tạo lập, cập nhật thông tin, xóa và tổ chức các không gian làm việc để quản lý tập trung nhiều dự án liên quan.

- **Quản lý Board:** Tạo mới, cập nhật thông tin, xóa các bảng công việc theo dự án; triển khai tính năng tìm kiếm và thêm thành viên vào bảng công việc.
- **Quản lý danh sách công việc (List):** Tạo mới, đổi tên, thay đổi thứ tự sắp xếp và xóa các danh sách trong mỗi bảng nhằm phân loại trạng thái công việc (ví dụ: Cần làm, Đang làm, Đã hoàn thành).
- **Quản lý thẻ công việc (Card):** Thực hiện các thao tác CRUD (Tạo, Đọc, Cập nhật, Xóa) đối với thẻ công việc; hỗ trợ thêm mô tả chi tiết, gán thành viên chịu trách nhiệm, thiết lập thời hạn hoàn thành và hiển thị trạng thái cảnh báo trực quan khi công việc sắp đến hạn hoặc quá hạn.
- **Quản lý danh sách kiểm tra (Checklist):** Cho phép tạo các checklist trong mỗi card, thêm các hạng mục công việc con (checklist item), đánh dấu hoàn thành/chưa hoàn thành và hiển thị thanh tiến độ (progress bar) trực quan theo tỷ lệ phần trăm hoàn thành.
- **Tương tác kéo thả (Drag and Drop):** Phát triển thuật toán và giao diện hỗ trợ kéo thả thẻ công việc giữa các danh sách hoặc thay đổi vị trí của các thẻ trong cùng một danh sách, đảm bảo tính nhất quán về vị trí hiển thị giữa Frontend và Backend.

Các nội dung và tính năng nằm ngoài phạm vi nghiên cứu thực hiện của học phần bao gồm:

- Các chức năng phân tích số liệu nâng cao, dự báo tiến độ và tự động xuất các báo cáo thống kê định kỳ dưới dạng biểu đồ phức tạp.
- Việc tích hợp sâu hệ thống quản lý công việc với các nền tảng và dịch vụ của bên thứ ba như Slack, Google Calendar, hay các công cụ lưu trữ đám mây bên ngoài.
- Tối ưu hóa hạ tầng và cơ chế cân bằng tải để phục vụ cho hệ thống quy mô lớn có hàng triệu người dùng hoạt động đồng thời.

1.5 Đối tượng sử dụng

Hệ thống quản lý công việc hướng đến việc giải quyết nhu cầu của các nhóm đối tượng người dùng chính sau:

- **Cá nhân:** Người dùng có nhu cầu sắp xếp, lên kế hoạch và quản lý các công việc học tập, nghiên cứu hoặc làm việc cá nhân hàng ngày một cách khoa học, giúp tăng tính kỷ luật và hiệu suất cá nhân.

- **Nhóm làm việc và phòng ban:** Các đội ngũ phát triển dự án, nhóm nghiên cứu hoặc các phòng ban chức năng trong tổ chức cần một không gian cộng tác số thống nhất để phân công công việc, phối hợp thực hiện và cùng theo dõi tiến độ công việc một cách minh bạch, rõ ràng.
- **Quản lý dự án:** Những người chịu trách nhiệm điều phối và giám sát toàn bộ hoạt động của dự án, cần theo dõi tổng quan tiến độ của từng hạng mục công việc, kiểm soát thời hạn hoàn thành (deadlines) và tối ưu hóa việc phân bổ nguồn lực nhân sự.

2 CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Cách tiếp cận và hướng giải quyết

2.1.1 Phương pháp phân tích bài toán

Bài toán được tiếp cận theo hướng phân tích từ trên xuống (top-down), bắt đầu từ việc nghiên cứu các hệ thống quản lý công việc phổ biến hiện có trên thị trường như Trello và Jira, từ đó xác định mô hình nghiệp vụ, các thực thể dữ liệu cốt lõi và luồng hoạt động của hệ thống. Qua nghiên cứu, mô hình Kanban được lựa chọn làm nền tảng tổ chức công việc do tính trực quan và khả năng phản ánh trạng thái công việc một cách rõ ràng theo các cột trạng thái (To Do, In Progress, Done).

Mô hình dữ liệu nghiệp vụ được xác định xoay quanh sáu thực thể chính theo cấu trúc phân cấp:

- **Workspace:** không gian làm việc tổng thể của người dùng, quản lý tập trung nhiều dự án liên quan.
- **Board:** đại diện cho một dự án hoặc bảng công việc cụ thể bên trong Workspace.
- **List:** các cột trạng thái bên trong Board, thể hiện giai đoạn thực hiện của công việc.
- **Card:** nhiệm vụ cụ thể cần thực hiện, được đặt trong các List tương ứng.
- **Checklist:** danh sách các công việc con nằm bên trong thẻ công việc (Card), hỗ trợ theo dõi chi tiết tiến trình thực hiện.
- **ChecklistItem:** các hạng mục cụ thể bên trong Checklist, có thể được đánh dấu trạng thái hoàn thành.

2.1.2 Hướng thiết kế giải pháp

Giải pháp được thiết kế theo kiến trúc client-server với sự tách biệt rõ ràng giữa Frontend và Backend thông qua giao tiếp RESTful API. Hướng tiếp cận này giúp hai thành phần có thể phát triển song song, dễ dàng bảo trì và mở rộng độc lập. Toàn bộ logic nghiệp vụ được tập trung xử lý tại Backend, trong khi Frontend chịu trách nhiệm trình bày giao diện và tương tác với người dùng.

2.1.3 Các quyết định kỹ thuật chính

Các quyết định kỹ thuật quan trọng được đưa ra trong quá trình thiết kế hệ thống bao gồm:

- **Phân tách Frontend và Backend:** Frontend xây dựng bằng ReactJS, Backend xây dựng bằng Java Spring Boot, giao tiếp qua REST API, tạo ra sự linh hoạt cao trong phát triển và triển khai.
- **Ủy quyền xác thực cho dịch vụ bên thứ ba:** Sử dụng Amazon Cognito để quản lý toàn bộ vòng đời người dùng (đăng ký, xác thực email, đăng nhập), giúp Backend tập trung vào logic nghiệp vụ thay vì tự xử lý xác thực.
- **Lưu trữ token bằng HttpOnly Cookie:** Access token và refresh token được lưu trong HttpOnly cookie thay vì localStorage, ngăn chặn các cuộc tấn công XSS (Cross-Site Scripting).
- **Cơ sở dữ liệu quan hệ PostgreSQL:** Phù hợp với mô hình dữ liệu có cấu trúc quan hệ rõ ràng và hỗ trợ tốt cho JPA/Hibernate trong Spring Boot.
- **Triển khai trên nền tảng Cloud:** Ứng dụng được triển khai hoàn toàn lên môi trường điện toán đám mây thông qua hệ sinh thái các dịch vụ của AWS (bao gồm EC2, RDS, S3, CloudFront và ALB), giúp hệ thống tận dụng tối đa khả năng mở rộng linh hoạt, tối ưu chi phí hạ tầng và đảm bảo tính sẵn sàng cao.

2.2 Các nền tảng và công nghệ sử dụng

2.2.1 Các nền tảng liên quan đến ứng dụng

2.2.1.1 ReactJS

- **Giới thiệu:** ReactJS là thư viện JavaScript mã nguồn mở được phát triển bởi Meta (Facebook), chuyên dùng để xây dựng giao diện người dùng (UI) tương tác và có hiệu suất cao.



Hình 1: Logo ReactJS

- **Chức năng:** ReactJS hoạt động theo cơ chế component-based, cho phép chia nhỏ giao diện thành các thành phần độc lập, có thể tái sử dụng. Virtual DOM giúp cập nhật giao diện một cách hiệu quả bằng cách chỉ render lại những phần thực sự thay đổi, thay vì cập nhật toàn bộ cây DOM.
- **Vai trò trong hệ thống:** ReactJS được sử dụng để xây dựng toàn bộ tầng giao diện (View layer) của hệ thống quản lý công việc, bao gồm các màn hình xác thực (đăng ký, đăng nhập), màn hình Dashboard hiển thị danh sách Workspace và Board, cũng như màn hình Board chi tiết với các chức năng quản lý List, Card và Drag and Drop.
- **Lý do lựa chọn:** Đối với các dạng ứng dụng web trang đơn (Single Page Application - SPA), ReactJS là một thư viện phổ biến, được sử dụng rất nhiều bởi các lập trình viên nhờ tính linh hoạt và khả năng triển khai dễ dàng, nhanh chóng.

2.2.1.2 Java Spring Boot

- **Giới thiệu:** Spring Boot là framework phát triển ứng dụng Java được xây dựng trên nền tảng Spring Framework, cung cấp cơ chế tự động cấu hình (auto-configuration) giúp giảm thiểu thời gian thiết lập dự án và tập trung vào triển khai logic nghiệp vụ.



Hình 2: Logo Java Spring Boot

- **Chức năng:** Spring Boot hỗ trợ xây dựng các ứng dụng RESTful API thông qua module Spring Web, quản lý kết nối cơ sở dữ liệu thông qua Spring Data JPA và Hibernate, đồng thời cung cấp cơ chế dependency injection giúp tổ chức code theo hướng lập trình hướng đối tượng rõ ràng.
- **Vai trò trong hệ thống:** Spring Boot đóng vai trò là tầng Backend của hệ thống, xử lý toàn bộ logic nghiệp vụ, xây dựng và cung cấp các RESTful API cho Frontend tiêu thụ. Cấu trúc project được tổ chức theo mô hình phân lớp bao gồm: `controller`, `service`, `repository` và `config`.
- **Lý do lựa chọn:** Spring Boot là một framework hỗ trợ công việc viết code cho các lập trình viên một cách mạnh mẽ và tối ưu. Kết hợp cùng với Java - một ngôn ngữ lập trình chặt chẽ, có tính ổn định cao, việc sử dụng Spring Boot giúp giảm thiểu tối đa các lỗi hệ thống có thể xảy ra trong quá trình vận hành.

2.2.1.3 PostgreSQL

- **Giới thiệu:** PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ, hỗ trợ đầy đủ các tính năng của SQL chuẩn cùng nhiều tính năng nâng cao như kiểu dữ liệu JSON, UUID, các ràng buộc phức tạp.



Hình 3: Logo PostgreSQL

- **Chức năng:** PostgreSQL đảm bảo tính toàn vẹn dữ liệu thông qua các ràng buộc FK (foreign key), hỗ trợ các truy vấn phức tạp với nhiều bảng quan hệ, đồng thời cung cấp hiệu suất đọc/ghi ổn định phù hợp với các ứng dụng web thông thường.
- **Vai trò trong hệ thống:** PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu chính của hệ thống, lưu trữ toàn bộ dữ liệu nghiệp vụ bao gồm thông tin người dùng, Workspace, Board, List, Card, Checklist và các bảng quan hệ trung gian. Kết nối với Backend thông qua Spring Data JPA và trình điều khiển PostgreSQL JDBC.

2.2.1.4 Mô hình MVC (Model – View – Controller)

MVC là mẫu kiến trúc phần mềm (architectural pattern) phân tách ứng dụng thành ba thành phần độc lập: **Model** chịu trách nhiệm quản lý dữ liệu và logic nghiệp vụ; **View** đảm nhận việc trình bày dữ liệu ra giao diện người dùng; **Controller** đóng vai trò trung gian, tiếp nhận yêu cầu từ người dùng, điều phối Model xử lý và trả kết quả về View.

2.2.2 Các nền tảng liên quan đến dịch vụ đám mây

2.2.2.1 Amazon Cognito

- **Giới thiệu:** Amazon Cognito là dịch vụ quản lý danh tính và xác thực người dùng do Amazon Web Services (AWS) cung cấp, hỗ trợ các luồng xác thực theo chuẩn OAuth 2.0 và OpenID Connect.
- **Chức năng:** Amazon Cognito cung cấp hai thành phần chính: User Pool để quản lý người dùng và xác thực danh tính, và Identity Pool để cấp quyền truy cập tài nguyên AWS. Dịch vụ này tự động xử lý việc gửi email xác nhận, quản lý mật khẩu, làm mới token và bảo vệ tài khoản theo các chính sách bảo mật chuẩn.

- **Vai trò trong hệ thống:** Amazon Cognito được tích hợp trực tiếp với tầng Backend thông qua AWS SDK for Java để xử lý toàn bộ luồng xác thực người dùng. Theo kiến trúc này, Client không gọi trực tiếp đến Cognito mà tương tác qua Backend; Backend sẽ giao tiếp với Cognito để đăng nhập, đăng ký và nhận JWT (access token, refresh token), sau đó trả về cho Client.

2.2.2.2 Amazon RDS

- **Giới thiệu:** Amazon RDS (Relational Database Service) là dịch vụ cơ sở dữ liệu quan hệ được quản lý hoàn toàn bởi AWS, hỗ trợ nhiều hệ quản trị cơ sở dữ liệu phổ biến trong đó có PostgreSQL.
- **Chức năng:** Amazon RDS tự động hóa các tác vụ quản trị cơ sở dữ liệu như sao lưu định kỳ, vá lỗi phần mềm, phát hiện và khôi phục lỗi tự động (automatic failure detection and recovery) và mở rộng tài nguyên. Dịch vụ cũng cung cấp tính năng High Availability thông qua triển khai Multi-AZ, đảm bảo cơ sở dữ liệu luôn sẵn sàng phục vụ.
- **Vai trò trong hệ thống:** Amazon RDS được dự kiến sử dụng để hosting cơ sở dữ liệu PostgreSQL trên môi trường đám mây, thay thế cho cơ sở dữ liệu cục bộ trong quá trình phát triển, nhằm chuẩn bị cho việc triển khai hệ thống lên môi trường Staging trong giai đoạn tiếp theo.

2.2.2.3 Amazon EC2 (Elastic Compute Cloud)

- **Giới thiệu:** Amazon EC2 là một dịch vụ web cốt lõi của AWS, cung cấp khả năng điện toán (compute capacity) an toàn và có thể thay đổi kích thước linh hoạt trên đám mây.
- **Chức năng:** EC2 cho phép khởi tạo các máy chủ ảo (instances) để chạy các ứng dụng. Người dùng có toàn quyền kiểm soát cấu hình phần cứng ảo (CPU, RAM), hệ điều hành và cài đặt các phần mềm cần thiết trên đó.
- **Vai trò trong hệ thống:** EC2 được sử dụng làm máy chủ (đặt trong mạng nội bộ - Private Subnet) để triển khai cả ứng dụng Backend (chạy Spring Boot) và Frontend (chạy ReactJS qua Nginx và Docker). Việc đặt EC2 trong mạng private giúp cách ly hoàn toàn các máy chủ khỏi Internet, tăng cường tối đa tính bảo mật.

2.2.2.4 Amazon ALB (Application Load Balancer)

- **Giới thiệu:** Amazon ALB là một dịch vụ cân bằng tải ở tầng ứng dụng (Layer 7) của AWS, chuyên phân phối lưu lượng truy cập HTTP và HTTPS một cách thông minh.
- **Chức năng:** ALB có khả năng phân phối lưu lượng truy cập mạng từ bên ngoài đến nhiều máy chủ đích (như các EC2 instances). Nó cung cấp các tính năng như định tuyến linh hoạt, hỗ trợ bảo mật SSL/TLS và tự động kiểm tra sức khỏe (health check) của các máy chủ đích để loại bỏ các máy chủ đang gặp lỗi.
- **Vai trò trong hệ thống:** ALB đóng vai trò là điểm truy cập (entry point) duy nhất cho toàn bộ hệ thống, tiếp nhận các request từ người dùng và phân phối tải đều đến các máy chủ EC2. Điều này giúp nâng cao tính sẵn sàng, cải thiện hiệu suất và hỗ trợ mở rộng hệ thống một cách dễ dàng khi lưu lượng truy cập tăng cao.

2.2.2.5 Amazon S3 (Simple Storage Service)

- **Giới thiệu:** Amazon S3 là dịch vụ lưu trữ đối tượng (object storage) do AWS cung cấp, mang lại khả năng lưu trữ và truy xuất lượng dữ liệu bất kỳ từ mọi nơi trên Internet với độ bền và tính bảo mật cao.
- **Chức năng:** S3 cho phép lưu trữ không giới hạn các loại tệp tin tĩnh (static files) và dữ liệu phi cấu trúc. Nó cũng hỗ trợ tính năng lưu trữ website tĩnh (static website hosting) để phục vụ trực tiếp các tệp HTML, CSS, JavaScript cho các ứng dụng web.
- **Vai trò trong hệ thống:** Trong dự án này, do Frontend đã được đưa lên máy chủ EC2, Amazon S3 được sử dụng chuyên biệt làm nơi lưu trữ an toàn cho các tài nguyên đa phương tiện do người dùng tải lên, ví dụ như hình ảnh đại diện (avatar) hoặc các tệp đính kèm trong thẻ công việc (Card).

2.2.2.6 Amazon CloudFront

- **Giới thiệu:** Amazon CloudFront là dịch vụ mạng phân phối nội dung (Content Delivery Network - CDN) tốc độ cao của AWS, giúp truyền tải dữ liệu và ứng dụng tới người dùng trên toàn cầu với độ trễ cực thấp.

- **Chức năng:** CloudFront lưu trữ bộ nhớ đệm (caching) các nội dung tĩnh và động tại các vị trí biên (edge locations) trên khắp thế giới, giúp rút ngắn khoảng cách vật lý giữa người dùng và dữ liệu, từ đó cải thiện đáng kể tốc độ tải trang web.
- **Vai trò trong hệ thống:** CloudFront được cấu hình để chỉ tương tác trực tiếp với Client và Amazon S3. Nó đóng vai trò trung gian phân phối các tệp tin đa phương tiện (từ S3) đến tay người dùng một cách tối ưu nhất thông qua bộ nhớ đệm (caching), đồng thời cung cấp giao thức HTTPS bảo mật và hạn chế truy cập công khai vào S3.

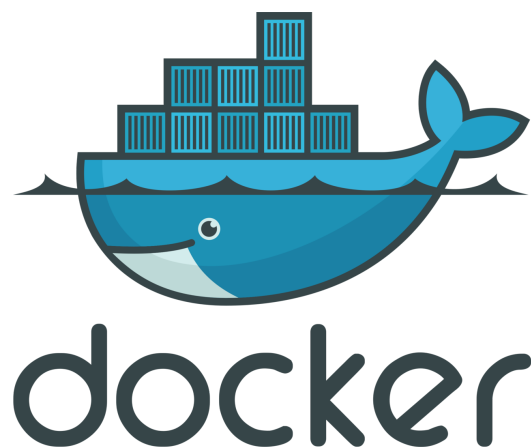
2.2.2.7 JWT (JSON Web Token)

- **Giới thiệu:** JWT là tiêu chuẩn mở (RFC 7519) định nghĩa cách truyền tải thông tin an toàn giữa các bên dưới dạng một đối tượng JSON được ký số (digitally signed).
- **Chức năng:** Một JWT bao gồm ba phần được mã hóa Base64 và ngăn cách bởi dấu chấm: Header (loại thuật toán ký), Payload (thông tin người dùng và thời hạn token) và Signature (chữ ký xác thực). Token có thể được xác minh mà không cần truy vấn cơ sở dữ liệu, giúp giảm tải cho hệ thống.
- **Vai trò trong hệ thống:** JWT được Amazon Cognito phát hành và trả về cho Backend sau khi người dùng đăng nhập thành công. Backend sau đó trả JWT về cho Client để lưu trữ trong HttpOnly cookie. Backend sử dụng token này để xác thực chữ ký trên mỗi request gọi API tiếp theo nhằm đảm bảo tính an toàn.

2.2.3 Các nền tảng khác

2.2.3.1 Docker

- **Giới thiệu:** Docker là nền tảng container hóa mã nguồn mở cho phép đóng gói ứng dụng cùng toàn bộ các phụ thuộc (thư viện, cấu hình, runtime) vào một đơn vị độc lập gọi là container. Container Docker có thể chạy nhất quán trên mọi môi trường từ máy phát triển đến máy chủ sản xuất.



Hình 4: Logo Docker

- **Chức năng:** Docker cung cấp cơ chế cô lập tiến trình thông qua namespace và cgroup của nhân Linux, giúp nhiều container chạy song song trên cùng một máy chủ mà không xung đột tài nguyên. Dockerfile cho phép định nghĩa môi trường build theo dạng khai báo, đảm bảo tính tái tạo (reproducibility) của quá trình build.
- **Vai trò trong hệ thống:** Ứng dụng Frontend (ReactJS) được đóng gói thành Docker image và triển khai trên máy chủ EC2. Docker đảm bảo môi trường chạy ứng dụng nhất quán giữa môi trường phát triển cục bộ và môi trường triển khai trên AWS, đồng thời đơn giản hóa quy trình cập nhật phiên bản ứng dụng.

2.2.3.2 Nginx

- **Giới thiệu:** Nginx là web server hiệu năng cao và reverse proxy mã nguồn mở, nổi tiếng với khả năng xử lý đồng thời lượng lớn kết nối với mức tiêu thụ tài nguyên thấp nhờ kiến trúc hướng sự kiện bất đồng bộ (event-driven, asynchronous).



Hình 5: Logo Nginx

- **Chức năng:** Nginx đảm nhiệm các vai trò phục vụ tệp tĩnh (static file serving), reverse proxy chuyển tiếp request đến ứng dụng phía sau, cấu hình HTTPS và gzip compression để giảm kích thước phản hồi. Ngoài ra, Nginx hỗ trợ cấu hình định tuyến linh hoạt thông qua file cấu hình theo cú pháp khai báo.
- **Vai trò trong hệ thống:** Nginx được cài đặt bên trong Docker container trên máy chủ EC2 Frontend, đóng vai trò là web server phục vụ các tệp tĩnh của ứng dụng ReactJS (HTML, CSS, JavaScript). Ngoài ra, Nginx xử lý định tuyến phía client (client-side routing) bằng cách chuyển hướng mọi request về file `index.html`, đảm bảo ứng dụng Single Page Application hoạt động đúng khi người dùng truy cập trực tiếp vào các đường dẫn con.

3 CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Phân tích hệ thống

3.1.1 Khảo sát và xác định yêu cầu

3.1.1.1 Yêu cầu chức năng

Các yêu cầu chức năng được xác định dựa trên phân tích nghiệp vụ quản lý công việc theo mô hình Kanban, bao gồm các nhóm chức năng chính sau:

Nhóm 1: Xác thực và quản lý tài khoản

Mã	Tên chức năng	Mô tả
FR-01	Đăng ký tài khoản	Người dùng nhập họ tên đầy đủ, địa chỉ email và mật khẩu để tạo tài khoản mới. Hệ thống gửi mã OTP xác nhận về email sau khi đăng ký thành công.
FR-02	Xác nhận email	Người dùng nhập mã OTP để kích hoạt tài khoản. Hỗ trợ chức năng gửi lại mã nếu mã đã hết hạn.
FR-03	Đăng nhập	Người dùng xác thực bằng email và mật khẩu. Sau khi xác thực thành công, access token và refresh token được lưu trong HttpOnly cookie.
FR-04	Đăng xuất	Phiên làm việc được kết thúc, cookie xác thực bị xóa khỏi trình duyệt.
FR-05	Làm mới phiên làm việc	Tự động làm mới access token bằng refresh token khi token hết hạn, đảm bảo phiên làm việc liên tục.
FR-32	Đổi mật khẩu	Người dùng thay đổi mật khẩu hiện tại thông qua xác thực với Amazon Cognito.
FR-33	Cập nhật hồ sơ cá nhân	Người dùng chỉnh sửa thông tin cá nhân như họ tên và ảnh đại diện.

Bảng 1: Yêu cầu chức năng nhóm 1

Nhóm 2: Quản lý Workspace

Mã	Tên chức năng	Mô tả
FR-06	Tạo Workspace	Người dùng tạo không gian làm việc mới với tên, loại và mô tả.
FR-07	Xem danh sách Workspace	Hệ thống hiển thị toàn bộ Workspace mà người dùng sở hữu hoặc là thành viên.
FR-08	Cập nhật Workspace	Người dùng chỉnh sửa tên, mô tả của Workspace.
FR-09	Quản lý thành viên Workspace	Thêm thành viên vào Workspace bằng địa chỉ email.
FR-30	Xóa Workspace	Người dùng (chủ sở hữu) xóa vĩnh viễn Workspace cùng toàn bộ Board, List và Card bên trong.

Bảng 2: Yêu cầu chức năng nhóm 2

Nhóm 3: Quản lý Board

Mã	Tên chức năng	Mô tả
FR-10	Tạo Board	Tạo bảng công việc mới trong Workspace với tiêu đề và hình nền tùy chọn.
FR-11	Xem Board	Tải và hiển thị toàn bộ danh sách List và Card trong Board theo mô hình Kanban.
FR-12	Cập nhật Board	Chỉnh sửa tiêu đề và hình nền của Board.
FR-13	Quản lý thành viên Board	Thêm thành viên từ Workspace vào Board với vai trò nhất định (ADMIN hoặc MEMBER).
FR-31	Xóa Board	Người dùng có quyền ADMIN xóa vĩnh viễn Board cùng toàn bộ List và Card bên trong.

Bảng 3: Yêu cầu chức năng nhóm 3

Nhóm 4: Quản lý List

Mã	Tên chức năng	Mô tả
FR-14	Tạo List	Tạo cột danh sách công việc mới trong Board.
FR-15	Cập nhật List	Đổi tên hoặc thay đổi vị trí của List trong Board.
FR-16	Xóa List	Xóa List và toàn bộ Card bên trong.

Bảng 4: Yêu cầu chức năng nhóm 4

Nhóm 5: Quản lý Card

Mã	Tên chức năng	Mô tả
FR-17	Tạo Card	Tạo thẻ công việc mới trong một List cụ thể với tiêu đề ban đầu.
FR-18	Xem chi tiết Card	Mở modal hiển thị đầy đủ thông tin Card bao gồm tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn, thành viên và danh sách kiểm tra.
FR-19	Cập nhật Card	Chỉnh sửa tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn và trạng thái hoàn thành của Card.
FR-20	Xóa Card	Xóa thẻ công việc khỏi hệ thống.
FR-22	Gán thành viên vào Card	Gán hoặc hủy gán thành viên chịu trách nhiệm thực hiện Card. Avatar thành viên được hiển thị trực tiếp trên Card.

Bảng 5: Yêu cầu chức năng nhóm 5

Nhóm 6: Kéo thả (Drag and Drop)

Mã	Tên chức năng	Mô tả
FR-23	Kéo thả Card	Di chuyển Card sang List khác hoặc sắp xếp lại thứ tự Card trong cùng một List; cập nhật <code>position</code> (và <code>listId</code> nếu đổi List) đồng bộ với Backend.
FR-24	Kéo thả List trong Board	Thay đổi thứ tự hiển thị của các List trong Board.

Bảng 6: Yêu cầu chức năng nhóm 6

Nhóm 7: Quản lý Checklist

Mã	Tên chức năng	Mô tả
FR-26	Tạo Checklist	Tạo danh sách kiểm tra mới trong Card.
FR-27	Cập nhật Checklist	Đổi tên Checklist.
FR-28	Xóa Checklist	Xóa Checklist và toàn bộ ChecklistItem liên quan.
FR-29	Quản lý ChecklistItem	Thêm, cập nhật trạng thái hoàn thành và xóa các hạng mục công việc con. Hiển thị thanh tiến độ (progress bar) theo tỷ lệ phần trăm hoàn thành.

Bảng 7: Yêu cầu chức năng nhóm 7

3.1.1.2 Yêu cầu phi chức năng

- **Bảo mật:** Toàn bộ giao tiếp giữa Client và Backend được thực hiện qua HTTP/HTTPS. Access token và refresh token được lưu trong HttpOnly cookie, ngăn chặn truy cập từ JavaScript phía Client (chống tấn công XSS). Xác thực người dùng được ủy thác cho Amazon Cognito. Các tài nguyên Backend và cơ sở dữ liệu được cách ly trong mạng nội bộ VPC.
- **Tính sẵn sàng:** Hệ thống được triển khai trên hạ tầng AWS với cân bằng tải qua ALB, đảm bảo tính sẵn sàng cao và khả năng phục hồi khi có sự cố.
- **Khả năng mở rộng:** Kiến trúc tách biệt Frontend và Backend cho phép mở rộng độc lập từng thành phần. Sử dụng Amazon RDS và EC2 giúp dễ dàng nâng cấp tài nguyên khi lưu lượng tăng.
- **Tính nhất quán dữ liệu:** Mọi thao tác kéo thả cập nhật trường position trong cơ sở dữ liệu, đảm bảo thứ tự hiển thị nhất quán giữa Frontend và Backend sau mỗi phiên làm việc.
- **Khả năng sử dụng:** Giao diện được thiết kế theo mô hình Kanban quen thuộc, hỗ trợ đầy đủ trên trình duyệt web hiện đại.

3.1.2 Xác định tác nhân và chức năng hệ thống

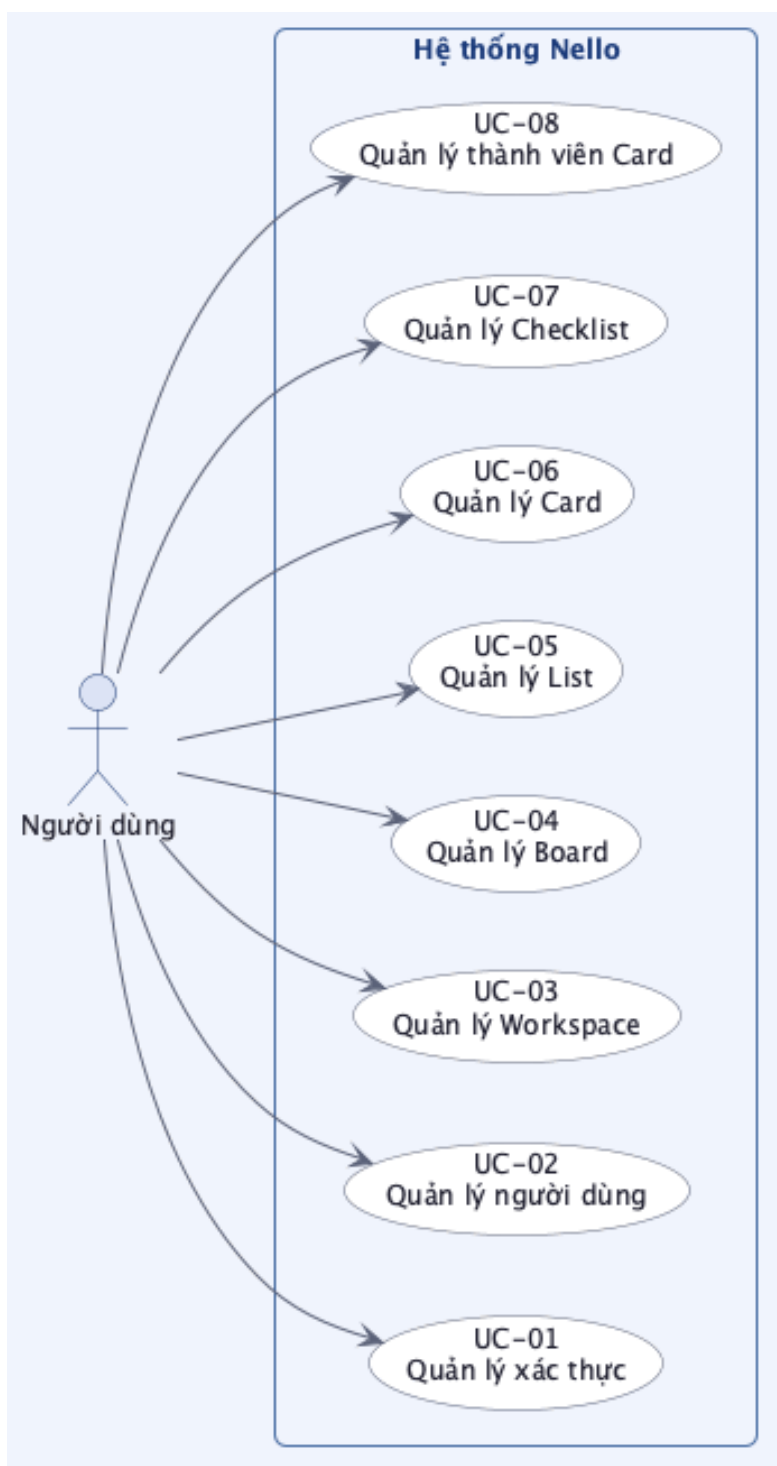
3.1.2.1 Biểu đồ Use Case tổng quát

Biểu đồ Use Case tổng quát mô tả toàn bộ các nhóm chức năng chính mà người dùng có thể thực hiện trong hệ thống. Hệ thống có duy nhất một tác nhân là **Người dùng**, bao gồm mọi đối tượng tương tác với hệ thống từ giai đoạn chưa xác thực đến đã đăng nhập. Quyền thực thi từng chức năng phụ thuộc vào trạng thái xác thực và vai trò (ADMIN, MEMBER, VIEWER) của người dùng trong Workspace hoặc Board tương ứng.

Các nhóm chức năng chính bao gồm:

- Quản lý xác thực
- Quản lý người dùng
- Quản lý Workspace
- Quản lý Board

- Quản lý List
- Quản lý Card
- Quản lý Checklist
- Quản lý thành viên Card



Hình 6: Biểu đồ Use Case tổng quát hệ thống Nello

3.1.2.2 Phân rã Use Case

Mỗi nhóm chức năng trong biểu đồ tổng quát được phân rã thành các use case thành phần cụ thể như sau:

UC-01: Quản lý xác thực – Bao gồm đăng ký tài khoản, xác nhận OTP, gửi lại OTP, đăng nhập và làm mới access token.

UC-02: Quản lý người dùng – Bao gồm xem thông tin cá nhân và chỉnh sửa thông tin cá nhân.

UC-03: Quản lý Workspace – Bao gồm tạo, xem danh sách, xem chi tiết Workspace, mời thành viên và xem danh sách thành viên.

UC-04: Quản lý Board – Bao gồm tạo, xem, cập nhật, xóa Board và thêm thành viên vào Board.

UC-05: Quản lý List – Bao gồm tạo, đổi tên, xóa List và sắp xếp thứ tự List.

UC-06: Quản lý Card – Bao gồm tạo, xem, cập nhật, đánh dấu hoàn thành, di chuyển và xóa Card.

UC-07: Quản lý Checklist – Bao gồm tạo, cập nhật, xóa Checklist; tạo, cập nhật, đánh dấu hoàn thành và xóa Checklist Item.

UC-08: Quản lý thành viên Card – Bao gồm gán và hủy gán thành viên vào Card.

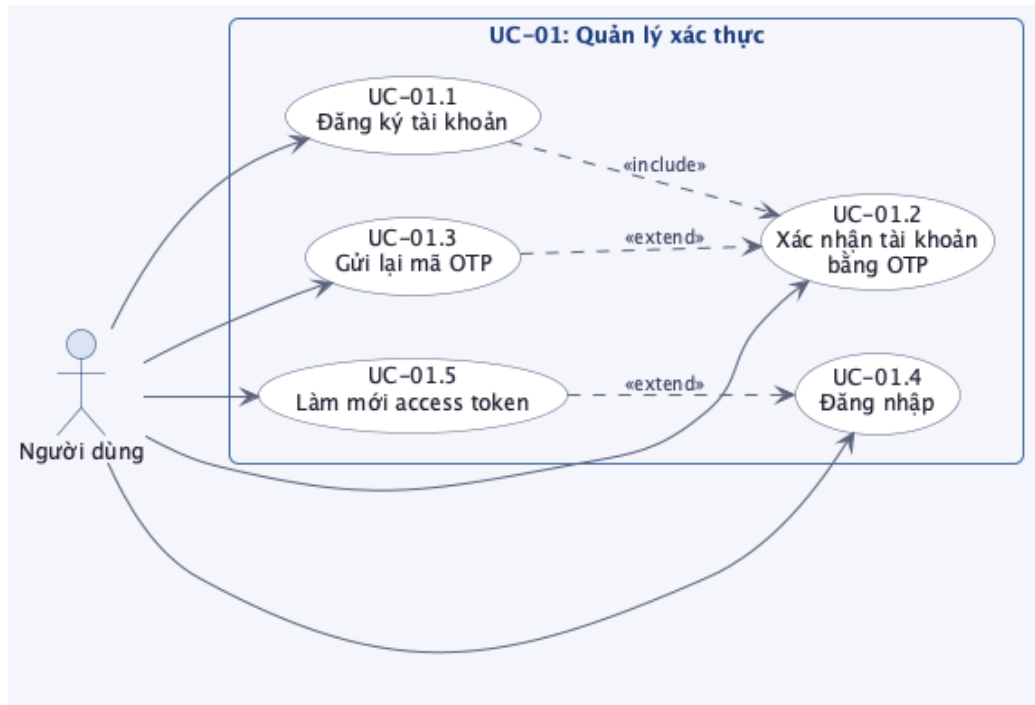
3.1.2.3 Danh sách Use Case

Mã Use Case	Tên Use Case
UC-01.1	Đăng ký tài khoản
UC-01.2	Xác nhận tài khoản bằng OTP
UC-01.3	Gửi lại mã OTP
UC-01.4	Đăng nhập
UC-01.5	Làm mới access token
UC-02.1	Xem thông tin cá nhân
UC-02.2	Chỉnh sửa thông tin cá nhân
UC-03.1	Tạo Workspace
UC-03.2	Xem danh sách Workspace
UC-03.3	Xem chi tiết Workspace
UC-03.4	Mời thành viên vào Workspace
UC-03.5	Xem danh sách thành viên Workspace
UC-04.1	Tạo Board
UC-04.2	Xem chi tiết Board
UC-04.3	Cập nhật tên Board
UC-04.4	Xóa Board
UC-04.5	Thêm thành viên vào Board
UC-05.1	Tạo List
UC-05.2	Đổi tên List
UC-05.3	Xóa List
UC-05.4	Sắp xếp thứ tự List
UC-06.1	Tạo Card
UC-06.2	Xem chi tiết Card
UC-06.3	Cập nhật Card
UC-06.4	Đánh dấu Card hoàn thành
UC-06.5	Di chuyển Card giữa các List
UC-06.6	Xóa Card
UC-07.1	Tạo Checklist
UC-07.2	Cập nhật Checklist
UC-07.3	Xóa Checklist
UC-07.4	Tạo Checklist Item
UC-07.5	Cập nhật Checklist Item
UC-07.6	Đánh dấu Checklist Item hoàn thành
UC-07.7	Xóa Checklist Item
UC-08.1	Gán thành viên vào Card
UC-08.2	Hủy gán thành viên khỏi Card

Bảng 8: Danh sách Use Case

3.1.3 Đặc tả Use Case

UC-01: Quản lý xác thực



Hình 7: Biểu đồ Use Case UC-01: Quản lý xác thực

Thành phần	Nội dung
Mã Use Case	UC-01.1
Tên Use Case	Đăng ký tài khoản
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng mới tạo tài khoản trong hệ thống thông qua Amazon Cognito.
Điều kiện	Người dùng chưa có tài khoản trong hệ thống.
Luồng chính	1. Người dùng truy cập trang đăng ký. 2. Hệ thống hiển thị form nhập họ tên, email và mật khẩu. 3. Người dùng điền thông tin và nhấn Đăng ký. 4. Hệ thống gọi Amazon Cognito tạo tài khoản. 5. Cognito gửi mã OTP xác nhận về địa chỉ email. 6. Nếu email đã tồn tại, hệ thống trả về lỗi HTTP 409.
Luồng thay thế	Email đã tồn tại: Hệ thống trả về lỗi HTTP 409. Mật khẩu không đáp ứng yêu cầu bảo mật: Hệ thống hiển thị thông báo lỗi tại trường mật khẩu. Lỗi kết nối Amazon Cognito: Hệ thống trả về lỗi HTTP 500.

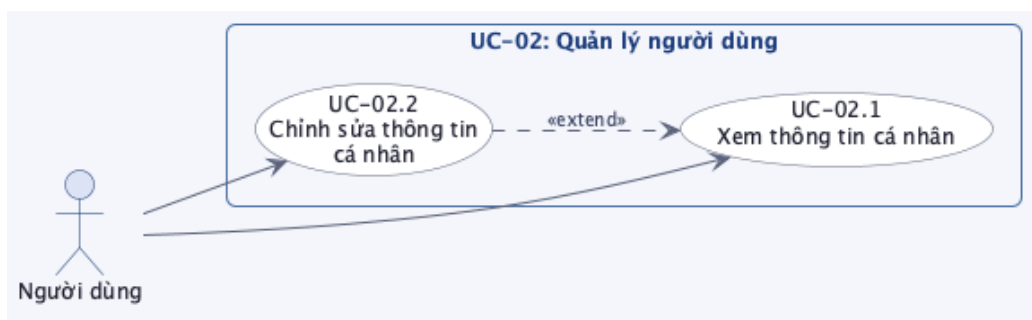
Thành phần	Nội dung
Mã Use Case	UC-01.2
Tên Use Case	Xác nhận tài khoản bằng OTP
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng nhập mã OTP nhận được qua email để kích hoạt tài khoản.
Điều kiện	Tài khoản đã được tạo; mã OTP đã được gửi về email.
Luồng chính	1. Người dùng nhập mã OTP nhận được qua email. 2. Người dùng nhấn Xác nhận. 3. Hệ thống gửi yêu cầu xác nhận đến Amazon Cognito. 4. Cognito kích hoạt tài khoản. 5. Hệ thống chuyển hướng người dùng về trang đăng nhập. 6. Nếu mã OTP sai hoặc hết hạn, hệ thống hiển thị thông báo lỗi.
Luồng thay thế	Mã OTP sai hoặc hết hạn: Hệ thống hiển thị thông báo lỗi, cho phép người dùng yêu cầu gửi lại mã. Lỗi kết nối Amazon Cognito: Hệ thống trả về lỗi HTTP 500.

Thành phần	Nội dung
Mã Use Case	UC-01.3
Tên Use Case	Gửi lại mã OTP
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng yêu cầu hệ thống gửi lại mã OTP khi mã cũ đã hết hạn hoặc chưa nhận được.
Điều kiện	Tài khoản đã đăng ký nhưng chưa xác nhận; mã OTP đã hết hạn hoặc chưa nhận được.
Luồng chính	1. Người dùng nhấn Gửi lại mã OTP trên trang xác nhận. 2. Hệ thống gọi Amazon Cognito để gửi lại mã OTP. 3. Cognito gửi mã OTP mới về email. 4. Hệ thống thông báo gửi lại thành công. 5. Nếu quá số lần yêu cầu cho phép, hệ thống hiển thị thông báo lỗi.
Luồng thay thế	Vượt quá số lần yêu cầu gửi lại cho phép: Hệ thống hiển thị thông báo giới hạn và yêu cầu đợi. Lỗi kết nối Amazon Cognito: Hệ thống trả về lỗi HTTP 500.

Thành phần	Nội dung
Mã Use Case	UC-01.4
Tên Use Case	Đăng nhập
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xác thực danh tính bằng email và mật khẩu để truy cập hệ thống.
Điều kiện	Người dùng đã có tài khoản và đã xác nhận email.
Luồng chính	1. Người dùng truy cập trang đăng nhập. 2. Hệ thống hiển thị form nhập email và mật khẩu. 3. Người dùng nhập thông tin và nhấn Đăng nhập. 4. Backend gọi Amazon Cognito để xác thực thông tin. 5. Cognito trả về JWT (access token và refresh token). 6. Backend lưu token vào HttpOnly cookie và chuyển hướng người dùng đến Dashboard. 7. Nếu email hoặc mật khẩu sai, hệ thống hiển thị thông báo lỗi.
Luồng thay thế	Email hoặc mật khẩu không đúng: Hệ thống hiển thị thông báo lỗi xác thực. Tài khoản chưa xác nhận email: Hệ thống yêu cầu xác nhận email trước khi đăng nhập. Lỗi kết nối Amazon Cognito: Hệ thống trả về lỗi HTTP 500.

Thành phần	Nội dung
Mã Use Case	UC-01.5
Tên Use Case	Làm mới access token
Tác nhân chính	Người dùng
Mô tả tóm tắt	Hệ thống tự động làm mới access token bằng refresh token khi token hết hạn.
Điều kiện	Người dùng đã đăng nhập; access token đã hết hạn; refresh token còn hiệu lực.
Luồng chính	1. Hệ thống phát hiện access token hết hạn khi xử lý request. 2. Hệ thống sử dụng refresh token trong HttpOnly cookie để gọi endpoint làm mới token. 3. Backend gọi Amazon Cognito để cấp access token mới. 4. Backend cập nhật HttpOnly cookie với access token mới. 5. Request gốc của người dùng được thực hiện lại. 6. Nếu refresh token hết hạn, hệ thống yêu cầu người dùng đăng nhập lại.
Luồng thay thế	Refresh token đã hết hạn: Hệ thống xóa cookie và yêu cầu người dùng đăng nhập lại. Lỗi kết nối Amazon Cognito: Hệ thống trả về lỗi HTTP 500.

UC-02: Quản lý người dùng

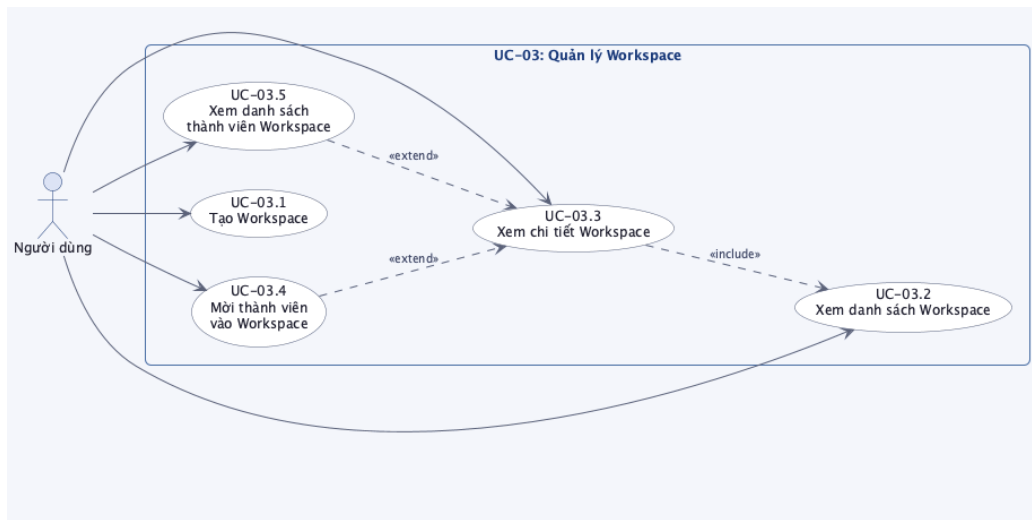


Hình 8: Biểu đồ Use Case UC-02: Quản lý người dùng

Thành phần	Nội dung
Mã Use Case	UC-02.1
Tên Use Case	Xem thông tin cá nhân
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xem thông tin hồ sơ cá nhân của mình trong hệ thống.
Điều kiện	Người dùng đã đăng nhập thành công.
Luồng chính	1. Người dùng truy cập trang thông tin cá nhân. 2. Hệ thống gọi API GET /api/v1/users/me với access token. 3. Backend trích xuất thông tin người dùng từ JWT và truy vấn cơ sở dữ liệu. 4. Hệ thống hiển thị họ tên, email, ảnh đại diện và các thông tin cá nhân khác.
Luồng thay thế	Access token không hợp lệ hoặc hết hạn: Hệ thống chuyển hướng về trang đăng nhập. Lỗi truy vấn cơ sở dữ liệu: Hệ thống trả về lỗi HTTP 500.

Thành phần	Nội dung
Mã Use Case	UC-02.2
Tên Use Case	Chỉnh sửa thông tin cá nhân
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng cập nhật các thông tin cá nhân như họ tên, ảnh đại diện, số điện thoại và địa chỉ.
Điều kiện	Người dùng đã đăng nhập và đang ở trang thông tin cá nhân.
Luồng chính	1. Người dùng truy cập trang thông tin cá nhân. 2. Người dùng chọn chỉnh sửa và thay đổi các trường thông tin. 3. Người dùng xác nhận lưu. 4. Hệ thống gọi API cập nhật và lưu thông tin mới vào cơ sở dữ liệu. 5. Hệ thống hiển thị thông báo cập nhật thành công.
Luồng thay thế	Thông tin nhập không hợp lệ (họ tên bị bỏ trống): Hệ thống hiển thị thông báo lỗi và yêu cầu nhập lại.

UC-03: Quản lý Workspace



Hình 9: Biểu đồ Use Case UC-03: Quản lý Workspace

Thành phần	Nội dung
Mã Use Case	UC-03.1
Tên Use Case	Tạo Workspace
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tạo không gian làm việc mới để quản lý các Board liên quan.
Điều kiện	Người dùng đã đăng nhập thành công.
Luồng chính	1. Người dùng chọn chức năng tạo Workspace. 2. Hệ thống hiển thị form nhập tên, loại và mô tả Workspace. 3. Người dùng điền thông tin và nhấn Tạo. 4. Hệ thống gọi API POST /api/v1/workspaces. 5. Backend tạo Workspace và tự động thêm người tạo làm thành viên với vai trò ADMIN. 6. Hệ thống hiển thị Workspace mới trong danh sách. 7. Nếu tên Workspace bị bỏ trống, hệ thống trả về lỗi HTTP 400.
Luồng thay thế	Tên Workspace bị bỏ trống: Hệ thống trả về lỗi HTTP 400. Lỗi không xác định: Hệ thống trả về lỗi HTTP 500.

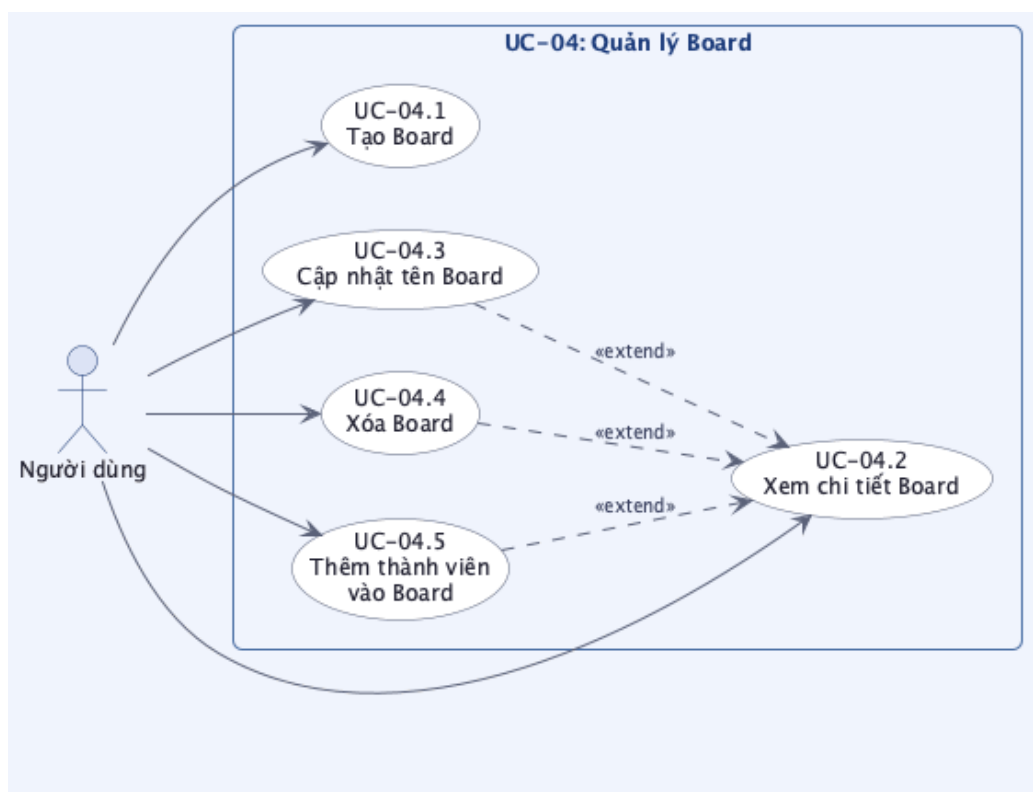
Thành phần	Nội dung
Mã Use Case	UC-03.2
Tên Use Case	Xem danh sách Workspace
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xem toàn bộ danh sách Workspace mà mình sở hữu hoặc là thành viên.
Điều kiện	Người dùng đã đăng nhập thành công.
Luồng chính	1. Người dùng truy cập trang Dashboard. 2. Hệ thống gọi API GET /api/v1/workspaces. 3. Backend truy vấn danh sách Workspace mà người dùng sở hữu hoặc là thành viên. 4. Hệ thống hiển thị danh sách Workspace kèm tên và mô tả.
Luồng thay thế	Người dùng chưa có Workspace nào: Hệ thống hiển thị giao diện trống kèm gợi ý tạo Workspace mới.

Thành phần	Nội dung
Mã Use Case	UC-03.3
Tên Use Case	Xem chi tiết Workspace
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xem thông tin chi tiết và danh sách Board trong một Workspace.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Workspace.
Luồng chính	1. Người dùng chọn một Workspace từ danh sách. 2. Hệ thống gọi API GET /api/v1/workspaces/:id. 3. Backend kiểm tra quyền truy cập của người dùng. 4. Hệ thống hiển thị tên, mô tả, loại Workspace và danh sách Board bên trong. 5. Nếu Workspace không tồn tại hoặc người dùng không có quyền, hệ thống trả về lỗi HTTP 404 hoặc HTTP 403.
Luồng thay thế	Workspace không tồn tại: Hệ thống trả về lỗi HTTP 404. Người dùng không có quyền truy cập: Hệ thống trả về lỗi HTTP 403.

Thành phần	Nội dung
Mã Use Case	UC-03.4
Tên Use Case	Mời thành viên vào Workspace
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng có vai trò ADMIN mời người dùng khác tham gia vào Workspace.
Điều kiện	Người dùng đã đăng nhập và có vai trò ADMIN trong Workspace.
Luồng chính	1. Người dùng truy cập trang quản lý thành viên Workspace. 2. Người dùng tìm kiếm thành viên cần mời theo email. 3. Người dùng chọn thành viên từ kết quả tìm kiếm và nhấn Mời. 4. Hệ thống gọi API POST <code>/api/v1/workspaces-members/:workspaceId</code>. 5. Backend kiểm tra người dùng tồn tại và chưa là thành viên. 6. Backend thêm người dùng vào Workspace với vai trò MEMBER. 7. Hệ thống cập nhật danh sách thành viên. 8. Nếu người dùng đã là thành viên, hệ thống hiển thị thông báo lỗi.
Luồng thay thế	Người dùng được mời không tồn tại trong hệ thống: Hệ thống hiển thị thông báo lỗi. Người dùng đã là thành viên của Workspace: Hệ thống hiển thị thông báo trùng lặp. Người dùng không có quyền ADMIN: Hệ thống trả về lỗi HTTP 403.

Thành phần	Nội dung
Mã Use Case	UC-03.5
Tên Use Case	Xem danh sách thành viên Workspace
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xem danh sách các thành viên hiện có trong Workspace cùng vai trò của từng người.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Workspace.
Luồng chính	1. Người dùng truy cập trang thành viên của Workspace. 2. Hệ thống gọi API GET /api/v1/workspaces/:id/members. 3. Backend truy vấn danh sách thành viên cùng vai trò trong Workspace. 4. Hệ thống hiển thị danh sách thành viên kèm tên, email và vai trò.
Luồng thay thế	Workspace không tồn tại: Hệ thống trả về lỗi HTTP 404. Người dùng không có quyền truy cập: Hệ thống trả về lỗi HTTP 403.

UC-04: Quản lý Board



Hình 10: Biểu đồ Use Case UC-04: Quản lý Board

Thành phần	Nội dung
Mã Use Case	UC-04.1
Tên Use Case	Tạo Board
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tạo bảng công việc mới trong Workspace để quản lý các List và Card theo mô hình Kanban.
Điều kiện	Người dùng đã đăng nhập và có vai trò ADMIN trong Workspace.
Luồng chính	1. Người dùng chọn chức năng tạo Board trong Workspace. 2. Hệ thống hiển thị form nhập tên và chọn hình nền Board. 3. Người dùng điền tên, chọn hình nền và nhấn Tạo. 4. Hệ thống gọi API POST /api/v1/boards với workspaceId, title và backgroundColor. 5. Backend kiểm tra quyền ADMIN của người dùng trong Workspace. 6. Backend tạo Board và trả về thông tin Board mới. 7. Hệ thống hiển thị Board mới trong danh sách Workspace. 8. Nếu người dùng không có quyền ADMIN, hệ thống trả về lỗi HTTP 403.
Luồng thay thế	Người dùng không có quyền ADMIN trong Workspace: Hệ thống trả về lỗi HTTP 403. Tên Board bị bỏ trống: Hệ thống trả về lỗi HTTP 400.

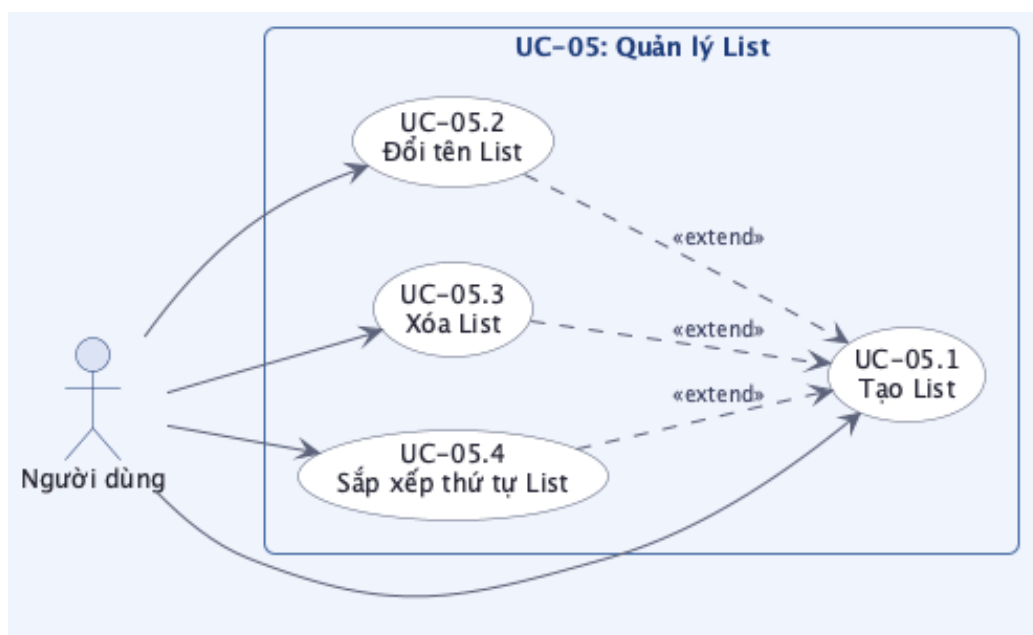
Thành phần	Nội dung
Mã Use Case	UC-04.2
Tên Use Case	Xem chi tiết Board
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tải và xem toàn bộ nội dung Board bao gồm danh sách List và Card theo mô hình Kanban.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng chọn Board từ danh sách. 2. Hệ thống gọi API GET /api/v1/boards/:id. 3. Backend kiểm tra quyền của người dùng trong Board. 4. Hệ thống tải và hiển thị toàn bộ List và Card theo mô hình Kanban. 5. Nếu Board không tồn tại hoặc người dùng không có quyền, hệ thống trả về lỗi HTTP 404 hoặc HTTP 403.
Luồng thay thế	Board không tồn tại: Hệ thống trả về lỗi HTTP 404. Người dùng không có quyền truy cập Board: Hệ thống trả về lỗi HTTP 403.

Thành phần	Nội dung
Mã Use Case	UC-04.3
Tên Use Case	Cập nhật tên Board
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng thay đổi tên của Board.
Điều kiện	Người dùng đã đăng nhập và có vai trò ADMIN trong Board.
Luồng chính	1. Người dùng chọn chỉnh sửa tên Board. 2. Hệ thống hiển thị ô nhập tên mới. 3. Người dùng nhập tên mới và xác nhận. 4. Hệ thống gọi API PUT /api/v1/boards/:id với tên mới. 5. Backend cập nhật tên Board trong cơ sở dữ liệu. 6. Hệ thống hiển thị tên Board đã cập nhật.
Luồng thay thế	Tên mới bị bỏ trống: Hệ thống không cho phép cập nhật. Người dùng không có quyền ADMIN: Hệ thống trả về lỗi HTTP 403.

Thành phần	Nội dung
Mã Use Case	UC-04.4
Tên Use Case	Xóa Board
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xóa vĩnh viễn Board cùng toàn bộ List và Card bên trong.
Điều kiện	Người dùng đã đăng nhập và có vai trò ADMIN trong Board.
Luồng chính	1. Người dùng chọn xóa Board. 2. Hệ thống hiển thị hộp thoại xác nhận xóa. 3. Người dùng xác nhận thao tác. 4. Hệ thống gọi API DELETE /api/v1/boards/:id. 5. Backend xóa Board và toàn bộ List, Card bên trong theo cơ chế cascade delete. 6. Hệ thống chuyển hướng người dùng về trang Workspace. 7. Nếu người dùng không có quyền ADMIN, hệ thống trả về lỗi HTTP 403.
Luồng thay thế	Người dùng không có quyền ADMIN trong Board: Hệ thống trả về lỗi HTTP 403. Board không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-04.5
Tên Use Case	Thêm thành viên vào Board
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng có vai trò ADMIN thêm thành viên từ Workspace vào Board với vai trò nhất định.
Điều kiện	Người dùng đã đăng nhập và có vai trò ADMIN trong Board; thành viên cần thêm phải là thành viên của Workspace.
Luồng chính	1. Người dùng truy cập trang quản lý thành viên Board. 2. Người dùng tìm kiếm thành viên trong Workspace và chọn người cần thêm. 3. Người dùng chọn vai trò (ADMIN, MEMBER hoặc VIEWER) và nhấn Thêm. 4. Hệ thống gọi API POST /api/v1/boards-members với userIds, boardId và roleId. 5. Backend kiểm tra thành viên thuộc Workspace và chưa có trong Board. 6. Backend thêm thành viên vào Board với vai trò được chỉ định. 7. Hệ thống cập nhật danh sách thành viên Board.
Luồng thay thế	Thành viên không thuộc Workspace: Hệ thống không cho phép thêm vào Board. Thành viên đã có trong Board: Hệ thống hiển thị thông báo trùng lặp. Người dùng không có quyền ADMIN: Hệ thống trả về lỗi HTTP 403.

UC-05: Quản lý List



Hình 11: Biểu đồ Use Case UC-05: Quản lý List

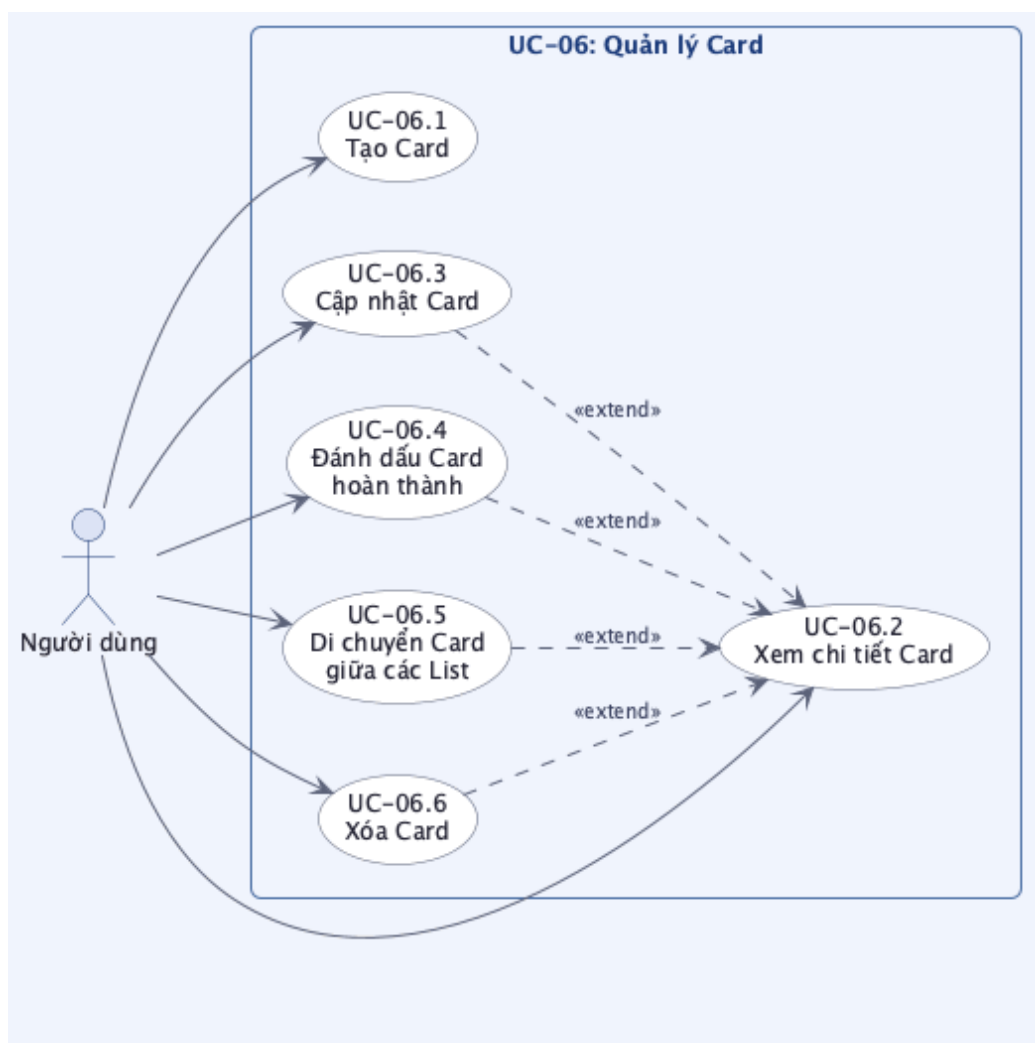
Thành phần	Nội dung
Mã Use Case	UC-05.1
Tên Use Case	Tạo List
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tạo cột danh sách công việc mới trong Board.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng chọn thêm List mới trong Board. 2. Hệ thống hiển thị ô nhập tên List. 3. Người dùng nhập tên và nhấn Thêm. 4. Hệ thống gọi API POST /api/v1/lists với boardId và title. 5. Backend tạo List mới và trả về listId, title, boardId. 6. Hệ thống hiển thị List mới ở cuối Board. 7. Nếu tên bị bỏ trống, hệ thống không cho phép tạo.
Luồng thay thế	Tên List bị bỏ trống: Hệ thống không cho phép tạo. Board không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-05.2
Tên Use Case	Đổi tên List
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng thay đổi tên của một List trong Board.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng nhấp vào tên List hiện tại. 2. Hệ thống chuyển sang chế độ chỉnh sửa tên. 3. Người dùng nhập tên mới và xác nhận. 4. Hệ thống gọi API PATCH /api/v1/lists/:id với tên mới. 5. Backend cập nhật tên List trong cơ sở dữ liệu. 6. Hệ thống hiển thị tên List đã cập nhật.
Luồng thay thế	Tên mới bị bỏ trống: Hệ thống không cho phép cập nhật. List không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-05.3
Tên Use Case	Xóa List
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xóa List và toàn bộ Card bên trong.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng chọn xóa List. 2. Hệ thống hiển thị hộp thoại xác nhận. 3. Người dùng xác nhận thao tác. 4. Hệ thống gọi API DELETE /api/v1/lists/:id. 5. Backend xóa List và toàn bộ Card bên trong theo cơ chế cascade delete. 6. Hệ thống cập nhật giao diện Board.
Luồng thay thế	List không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-05.4
Tên Use Case	Sắp xếp thứ tự List
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng kéo thả để thay đổi thứ tự hiển thị của các List trong Board.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng nhấn giữ và kéo List sang vị trí mong muốn. 2. Frontend tính toán giá trị position mới dựa trên thuật toán fractional indexing. 3. Giao diện cập nhật thứ tự List ngay lập tức (optimistic update). 4. Hệ thống gọi API PATCH /api/v1/lists/:id với position mới. 5. Backend cập nhật trường position trong cơ sở dữ liệu.
Luồng thay thế	Lỗi xảy ra khi cập nhật position: Hệ thống hoàn tác về thứ tự cũ và hiển thị thông báo lỗi.

UC-06: Quản lý Card



Hình 12: Biểu đồ Use Case UC-06: Quản lý Card

Thành phần	Nội dung
Mã Use Case	UC-06.1
Tên Use Case	Tạo Card
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tạo thẻ công việc mới trong một List.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng chọn thêm Card trong một List. 2. Hệ thống hiển thị ô nhập tiêu đề Card. 3. Người dùng nhập tiêu đề và nhấn Thêm. 4. Hệ thống gọi API POST /api/v1/cards với listId, title và position. 5. Backend kiểm tra quyền người dùng trong Board. 6. Backend tạo Card và trả về cardId, title, listId, position. 7. Hệ thống hiển thị Card mới ở cuối List.
Luồng thay thế	Tiêu đề Card bị bỏ trống: Hệ thống không cho phép tạo. List không tồn tại: Hệ thống trả về lỗi HTTP 404. Người dùng không có quyền trong Board: Hệ thống trả về lỗi HTTP 403.

Thành phần	Nội dung
Mã Use Case	UC-06.2
Tên Use Case	Xem chi tiết Card
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xem đầy đủ thông tin chi tiết của một Card bao gồm mô tả, ngày tháng, thành viên và checklist.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng nhấp vào Card. 2. Hệ thống gọi API GET /api/v1/cards/:id. 3. Backend trả về đầy đủ thông tin Card bao gồm tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn, trạng thái hoàn thành, thành viên phụ trách và danh sách Checklist. 4. Hệ thống hiển thị modal chi tiết Card.
Luồng thay thế	Card không tồn tại: Hệ thống trả về lỗi HTTP 404. Người dùng không có quyền trong Board: Hệ thống trả về lỗi HTTP 403.

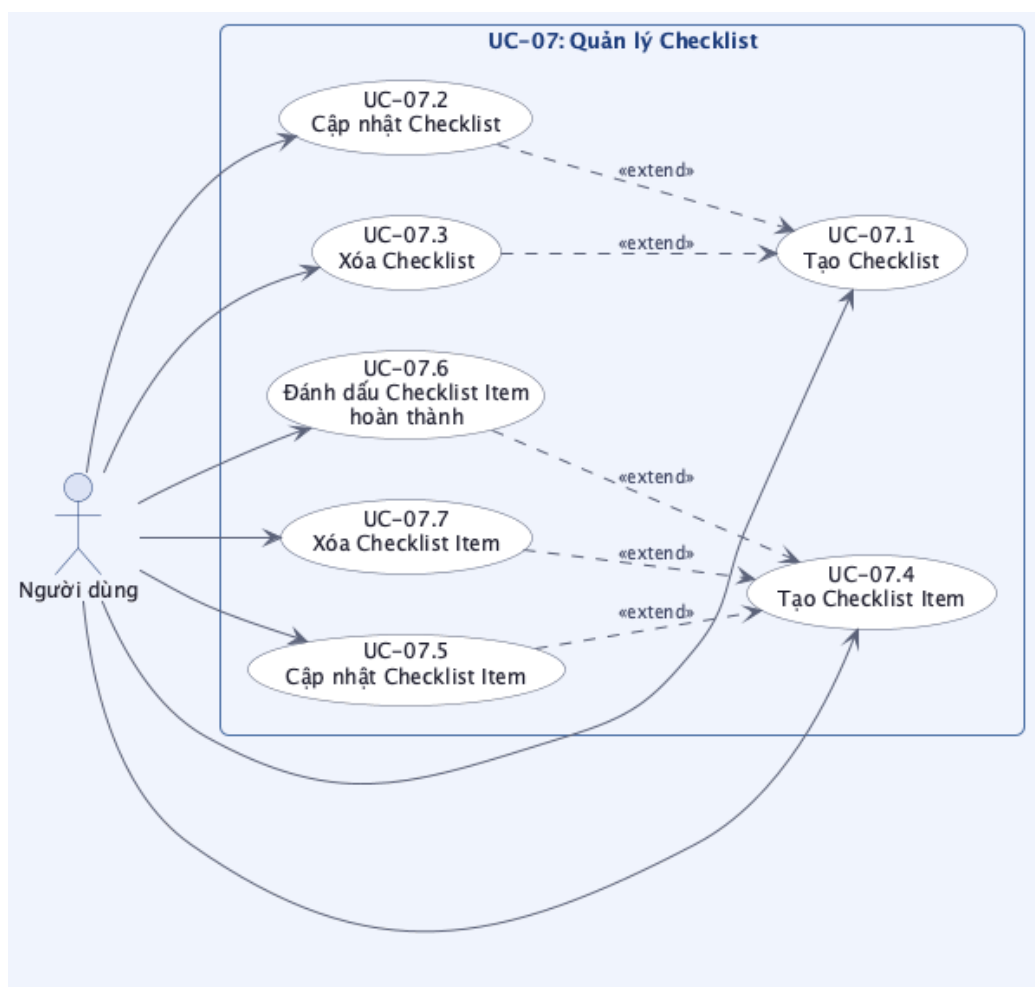
Thành phần	Nội dung
Mã Use Case	UC-06.3
Tên Use Case	Cập nhật Card
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng chỉnh sửa thông tin của Card bao gồm tiêu đề, mô tả, ngày bắt đầu và ngày đến hạn.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng mở modal chi tiết Card. 2. Người dùng chỉnh sửa các trường thông tin cần thiết (tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn). 3. Hệ thống gọi API PATCH /api/v1/cards/:id với các trường đã thay đổi. 4. Backend cập nhật Card trong cơ sở dữ liệu. 5. Hệ thống hiển thị thông tin Card đã cập nhật.
Luồng thay thế	Tiêu đề bị bỏ trống: Hệ thống không cho phép lưu. Lỗi không xác định: Hệ thống trả về lỗi HTTP 500.

Thành phần	Nội dung
Mã Use Case	UC-06.4
Tên Use Case	Đánh dấu Card hoàn thành
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng cập nhật trạng thái hoàn thành của Card.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng mở modal chi tiết Card. 2. Người dùng tích vào ô đánh dấu hoàn thành. 3. Hệ thống gọi API PATCH /api/v1/cards/:id với isCompleted = true. 4. Backend cập nhật trạng thái hoàn thành trong cơ sở dữ liệu. 5. Hệ thống hiển thị trạng thái hoàn thành trên Card và trong modal.
Luồng thay thế	Lỗi xảy ra khi cập nhật: Hệ thống hoàn tác về trạng thái cũ và hiển thị thông báo lỗi.

Thành phần	Nội dung
Mã Use Case	UC-06.5
Tên Use Case	Di chuyển Card giữa các List
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng kéo thả Card sang List khác hoặc thay đổi vị trí trong cùng một List.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng nhấn giữ và kéo Card sang List mới hoặc vị trí khác trong cùng List. 2. Frontend tính toán listId đích và giá trị position mới. 3. Giao diện cập nhật vị trí Card ngay lập tức (optimistic update). 4. Hệ thống gọi API PATCH /api/v1/cards/:id với listId mới và position mới. 5. Backend cập nhật listId và position của Card trong cơ sở dữ liệu.
Luồng thay thế	Lỗi xảy ra khi cập nhật position: Hệ thống hoàn tác Card về vị trí cũ và hiển thị thông báo lỗi.

Thành phần	Nội dung
Mã Use Case	UC-06.6
Tên Use Case	Xóa Card
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xóa Card và toàn bộ Checklist, ChecklistItem, CardMember liên quan.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board.
Luồng chính	1. Người dùng mở modal chi tiết Card và chọn xóa. 2. Hệ thống hiển thị hộp thoại xác nhận. 3. Người dùng xác nhận thao tác. 4. Hệ thống gọi API DELETE /api/v1/cards/:id. 5. Backend xóa Card và toàn bộ dữ liệu liên quan theo cơ chế cascade delete. 6. Hệ thống đóng modal và cập nhật giao diện Board.
Luồng thay thế	Card không tồn tại: Hệ thống trả về lỗi HTTP 404. Lỗi không xác định: Hệ thống trả về lỗi HTTP 500.

UC-07: Quản lý Checklist



Hình 13: Biểu đồ Use Case UC-07: Quản lý Checklist

Thành phần	Nội dung
Mã Use Case	UC-07.1
Tên Use Case	Tạo Checklist
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tạo danh sách kiểm tra mới bên trong một Card.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; Card đang mở.
Luồng chính	1. Người dùng chọn thêm Checklist trong modal Card. 2. Hệ thống hiển thị ô nhập tên Checklist. 3. Người dùng nhập tên và nhấn Thêm. 4. Hệ thống gọi API POST /api/v1/checklists với cardId, name và position. 5. Backend tạo Checklist và trả về thông tin. 6. Hệ thống hiển thị Checklist mới trong modal Card.
Luồng thay thế	Tên Checklist bị bỏ trống: Hệ thống không cho phép tạo. Card không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-07.2
Tên Use Case	Cập nhật Checklist
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng thay đổi tên của Checklist.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; Checklist tồn tại.
Luồng chính	1. Người dùng nhấp vào tên Checklist để chỉnh sửa. 2. Người dùng nhập tên mới và xác nhận. 3. Hệ thống gọi API PATCH /api/v1/checklists/:id với tên mới. 4. Backend cập nhật tên Checklist trong cơ sở dữ liệu. 5. Hệ thống hiển thị tên Checklist đã cập nhật.
Luồng thay thế	Tên mới bị bỏ trống: Hệ thống không cho phép cập nhật. Checklist không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-07.3
Tên Use Case	Xóa Checklist
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xóa Checklist và toàn bộ ChecklistItem bên trong.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; Checklist tồn tại.
Luồng chính	1. Người dùng chọn xóa Checklist. 2. Hệ thống hiển thị hộp thoại xác nhận. 3. Người dùng xác nhận thao tác. 4. Hệ thống gọi API DELETE /api/v1/checklists/:id. 5. Backend xóa Checklist và toàn bộ ChecklistItem bên trong theo cơ chế cascade delete. 6. Hệ thống cập nhật modal Card.
Luồng thay thế	Checklist không tồn tại: Hệ thống trả về lỗi HTTP 404.

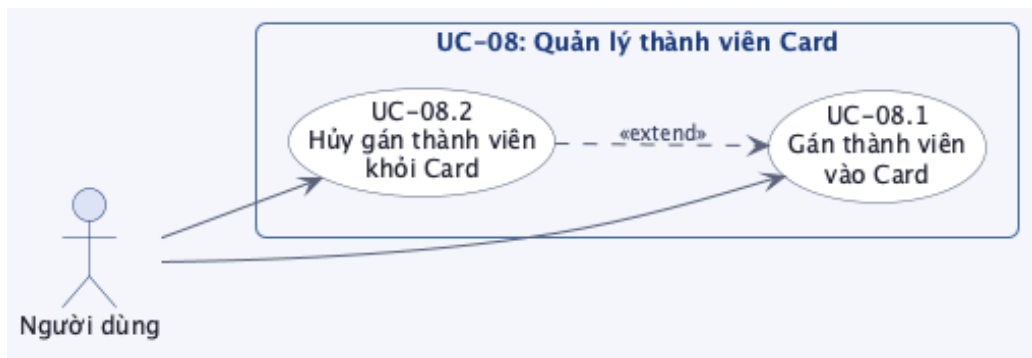
Thành phần	Nội dung
Mã Use Case	UC-07.4
Tên Use Case	Tạo Checklist Item
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng thêm hạng mục công việc con mới vào Checklist.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; Checklist tồn tại.
Luồng chính	1. Người dùng chọn thêm hạng mục trong Checklist. 2. Hệ thống hiển thị ô nhập tên và ngày đến hạn. 3. Người dùng nhập thông tin và nhấn Thêm. 4. Hệ thống gọi API POST /api/v1/checklist-items với checklistId, name, position và dueDate. 5. Backend tạo ChecklistItem với isCompleted = false. 6. Hệ thống hiển thị hạng mục mới và cập nhật thanh tiến độ.
Luồng thay thế	Tên hạng mục bị bỏ trống: Hệ thống không cho phép thêm. Checklist không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-07.5
Tên Use Case	Cập nhật Checklist Item
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng chỉnh sửa tên hoặc ngày đến hạn của một hạng mục trong Checklist.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; ChecklistItem tồn tại.
Luồng chính	1. Người dùng nhấp vào tên hạng mục để chỉnh sửa. 2. Người dùng thay đổi tên hoặc ngày đến hạn. 3. Người dùng xác nhận thay đổi. 4. Hệ thống gọi API PATCH /api/v1/checklist-items/:id với thông tin mới. 5. Backend cập nhật ChecklistItem trong cơ sở dữ liệu. 6. Hệ thống hiển thị thông tin đã cập nhật.
Luồng thay thế	Tên mới bị bỏ trống: Hệ thống không cho phép cập nhật. ChecklistItem không tồn tại: Hệ thống trả về lỗi HTTP 404.

Thành phần	Nội dung
Mã Use Case	UC-07.6
Tên Use Case	Đánh dấu Checklist Item hoàn thành
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng tích hoặc bỏ tích trạng thái hoàn thành của một hạng mục, thanh tiến độ được cập nhật tự động.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; ChecklistItem tồn tại.
Luồng chính	1. Người dùng tích vào ô bên cạnh tên hạng mục. 2. Hệ thống gọi API PATCH /api/v1/checklist-items/:id với isCompleted = true (hoặc false nếu bỏ tích). 3. Backend cập nhật trạng thái hoàn thành trong cơ sở dữ liệu. 4. Frontend tính lại tỷ lệ hoàn thành và cập nhật thanh tiến độ.
Luồng thay thế	Lỗi xảy ra khi cập nhật: Hệ thống hoàn tác về trạng thái cũ, thanh tiến độ không thay đổi.

Thành phần	Nội dung
Mã Use Case	UC-07.7
Tên Use Case	Xóa Checklist Item
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng xóa một hạng mục khỏi Checklist.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; ChecklistItem tồn tại.
Luồng chính	1. Người dùng chọn xóa hạng mục. 2. Hệ thống gọi API DELETE /api/v1/checklist-items/:id. 3. Backend xóa ChecklistItem khỏi cơ sở dữ liệu. 4. Frontend tính lại tỷ lệ hoàn thành và cập nhật thanh tiến độ.
Luồng thay thế	ChecklistItem không tồn tại: Hệ thống trả về lỗi HTTP 404.

UC-08: Quản lý thành viên Card



Hình 14: Biểu đồ Use Case UC-08: Quản lý thành viên Card

Thành phần	Nội dung
Mã Use Case	UC-08.1
Tên Use Case	Gán thành viên vào Card
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng gán một thành viên Board làm người phụ trách cho Card.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; Card đang mở.
Luồng chính	1. Người dùng chọn chức năng gán thành viên trong modal Card. 2. Hệ thống hiển thị danh sách thành viên Board có thể gán. 3. Người dùng chọn thành viên cần gán. 4. Hệ thống gọi API POST <code>/api/v1/card-members</code> với <code>userId</code> và <code>cardId</code>. 5. Backend kiểm tra thành viên chưa được gán vào Card này. 6. Backend tạo bản ghi CardMember. 7. Hệ thống hiển thị avatar thành viên trực tiếp trên Card.
Luồng thay thế	Thành viên đã được gán vào Card: Hệ thống hiển thị thông báo trùng lặp. Thành viên không thuộc Board: Hệ thống không cho phép gán. Lỗi không xác định: Hệ thống trả về lỗi HTTP 500.

Thành phần	Nội dung
Mã Use Case	UC-08.2
Tên Use Case	Hủy gán thành viên khỏi Card
Tác nhân chính	Người dùng
Mô tả tóm tắt	Người dùng hủy gán thành viên khỏi Card, loại bỏ trách nhiệm phụ trách.
Điều kiện	Người dùng đã đăng nhập và là thành viên của Board; thành viên đã được gán vào Card.
Luồng chính	1. Người dùng mở modal Card và truy cập danh sách thành viên đã gán. 2. Người dùng chọn thành viên cần hủy gán. 3. Hệ thống gọi API DELETE /api/v1/card-members/:id. 4. Backend xóa bản ghi CardMember. 5. Hệ thống cập nhật danh sách thành viên và xóa avatar khỏi Card.
Luồng thay thế	Bản ghi CardMember không tồn tại: Hệ thống trả về lỗi HTTP 404.

3.1.4 Phân tích luồng nghiệp vụ

Biểu đồ hoạt động (Activity Diagram) được sử dụng để mô tả trình tự và luồng xử lý của từng chức năng trong hệ thống. Khác với biểu đồ Use Case chỉ trả lời “Hệ thống làm gì?”, biểu đồ hoạt động trả lời “Chức năng đó diễn ra như thế nào?”, thể hiện chi tiết các bước xử lý, nhánh điều kiện và luồng thay thế khi xảy ra lỗi.

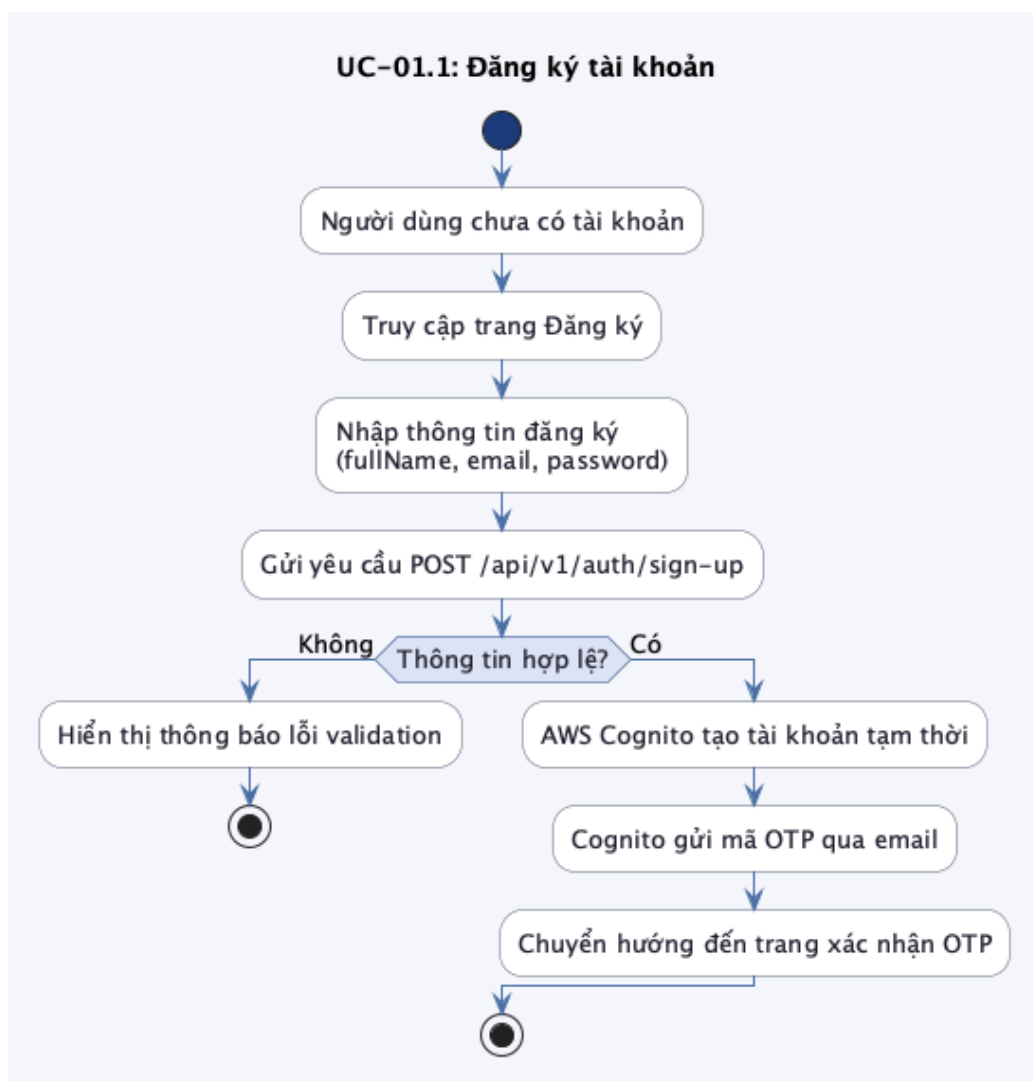
Phần này trình bày 36 biểu đồ hoạt động, mỗi biểu đồ tương ứng với một use case thành phần (UC-01.1 đến UC-08.2), mô tả chi tiết luồng nghiệp vụ của từng chức năng trong hệ thống Nello.

3.1.4.1 UC-01: Quản lý xác thực

Nhóm chức năng xác thực bao gồm năm luồng nghiệp vụ: đăng ký tài khoản, xác nhận OTP, gửi lại OTP, đăng nhập và làm mới token. Tất cả các luồng này đều ủy quyền xử lý xác thực cho dịch vụ Amazon Cognito.

UC-01.1: Đăng ký tài khoản

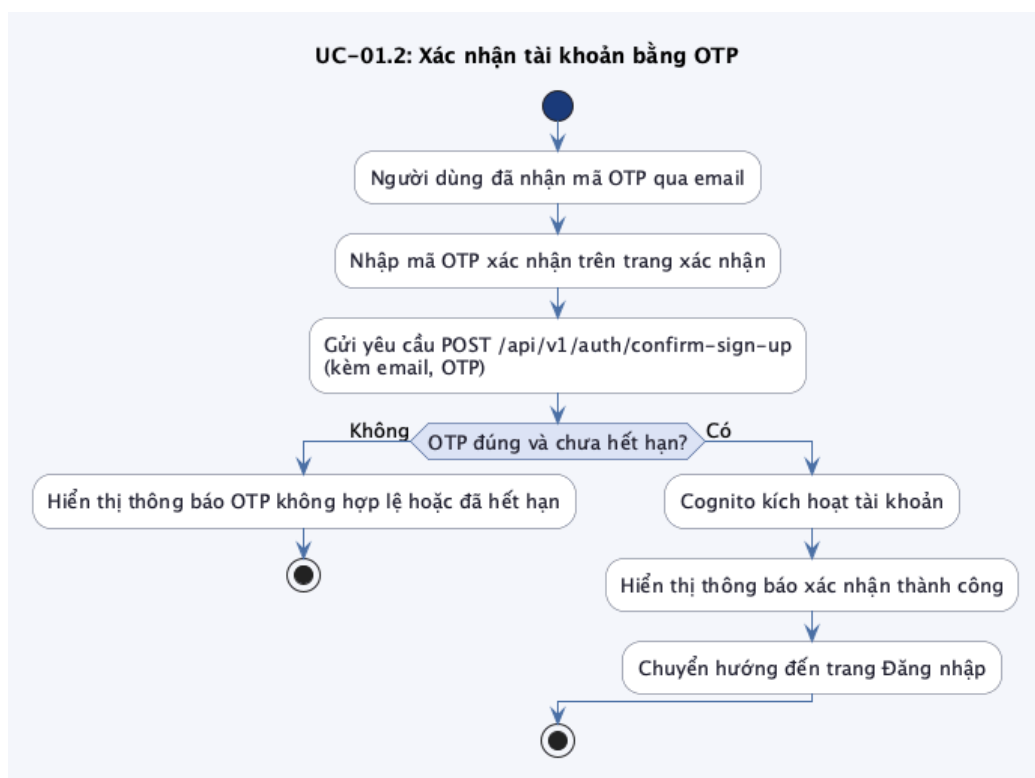
Người dùng nhập thông tin đăng ký; hệ thống xác thực dữ liệu đầu vào, sau đó ủy quyền cho Amazon Cognito tạo tài khoản tạm thời và gửi mã OTP qua email để tiến hành xác nhận.



Hình 15: Biểu đồ hoạt động UC-01.1: Đăng ký tài khoản

UC-01.2: Xác nhận tài khoản bằng OTP

Người dùng nhập mã OTP nhận được qua email; hệ thống gửi yêu cầu xác nhận đến Cognito, kích hoạt tài khoản nếu mã hợp lệ và chuyển hướng đến trang đăng nhập.



Hình 16: Biểu đồ hoạt động UC-01.2: Xác nhận tài khoản bằng OTP

UC-01.3: Gửi lại mã OTP

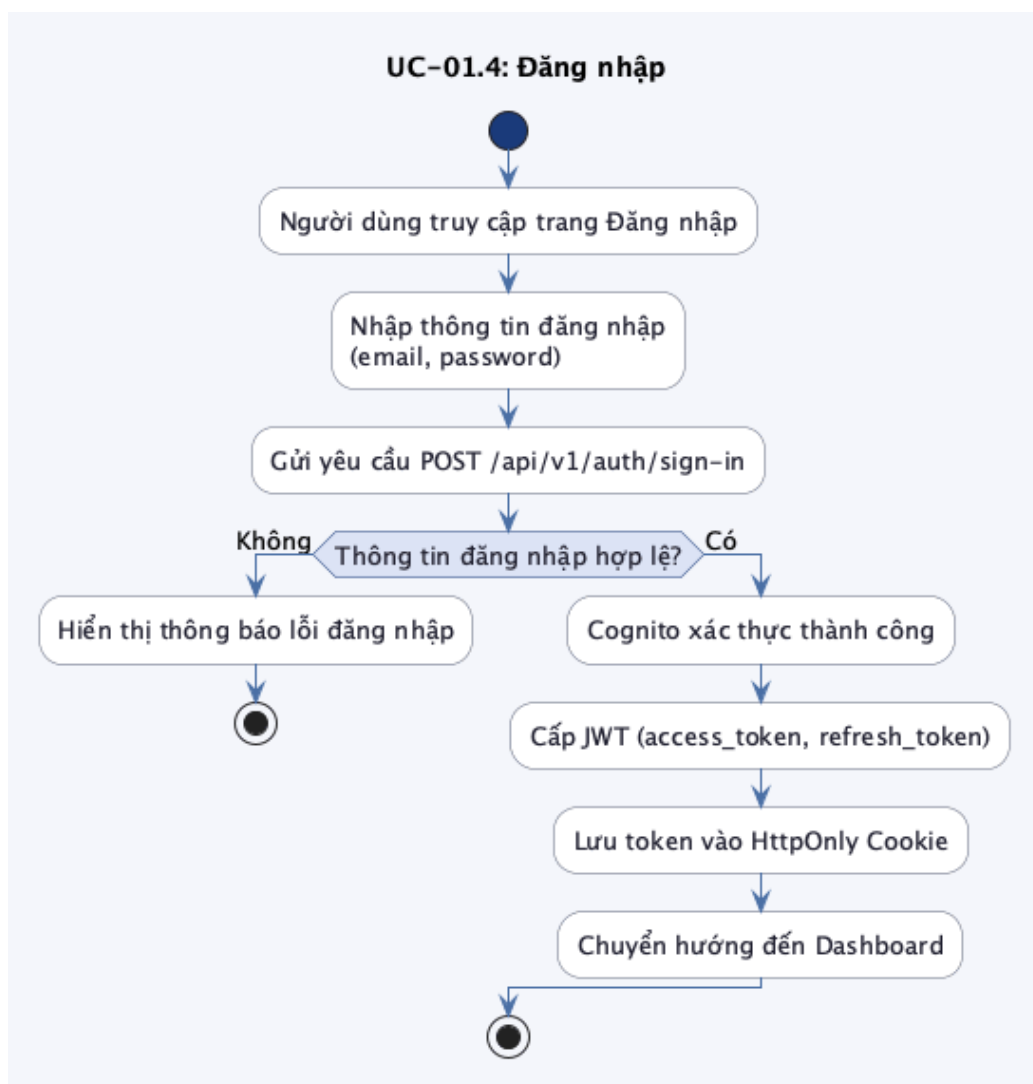
Khi mã OTP hết hạn hoặc không nhận được, người dùng yêu cầu gửi lại; hệ thống gọi Cognito để phát hành mã OTP mới và gửi về email đã đăng ký.



Hình 17: Biểu đồ hoạt động UC-01.3: Gửi lại mã OTP

UC-01.4: Đăng nhập

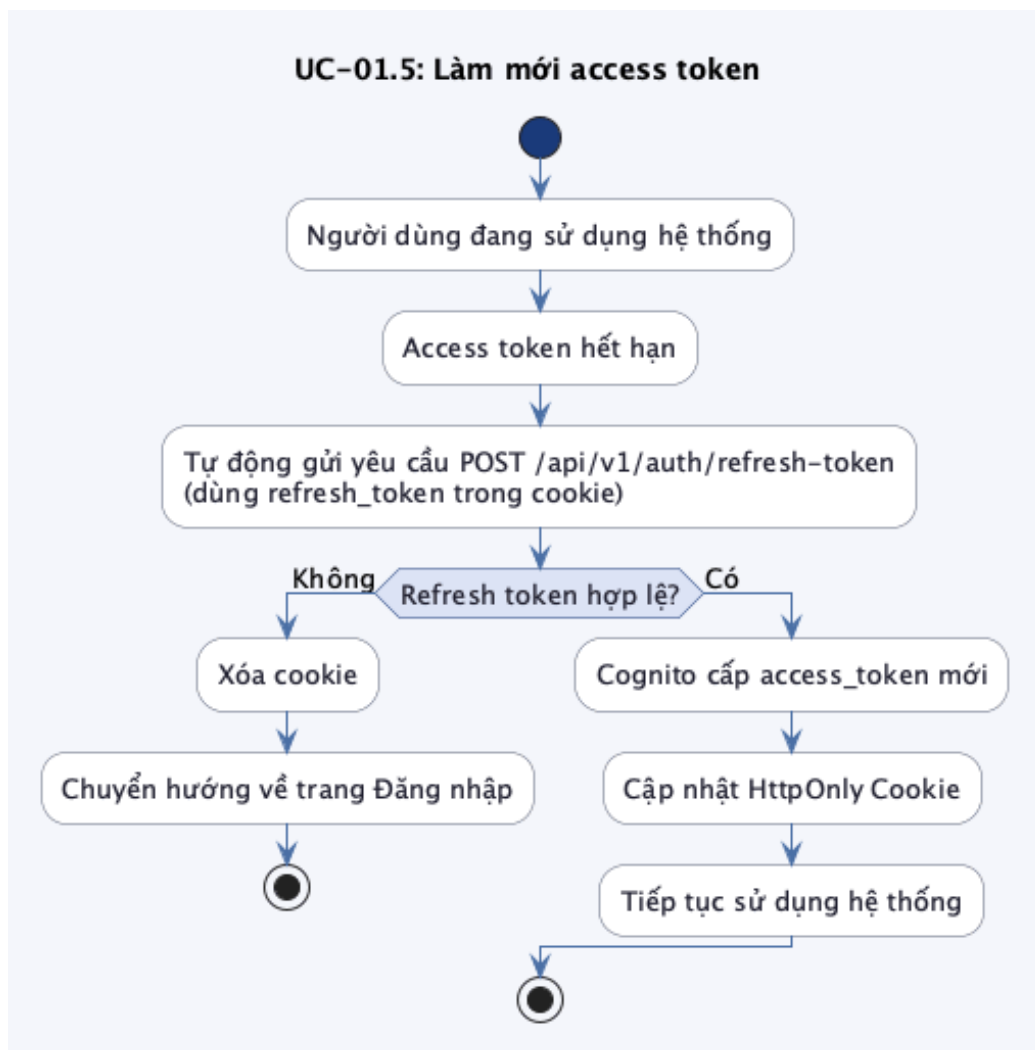
Người dùng nhập thông tin đăng nhập; Cognito xác thực và cấp cặp JWT (access token, refresh token) được lưu vào HttpOnly Cookie, sau đó chuyển hướng đến Dashboard.



Hình 18: Biểu đồ hoạt động UC-01.4: Đăng nhập

UC-01.5: Làm mới access token

Khi access token hết hạn, hệ thống tự động gửi refresh token đến Cognito để lấy access token mới; nếu refresh token không còn hợp lệ, phiên làm việc bị hủy và người dùng được chuyển về trang đăng nhập.



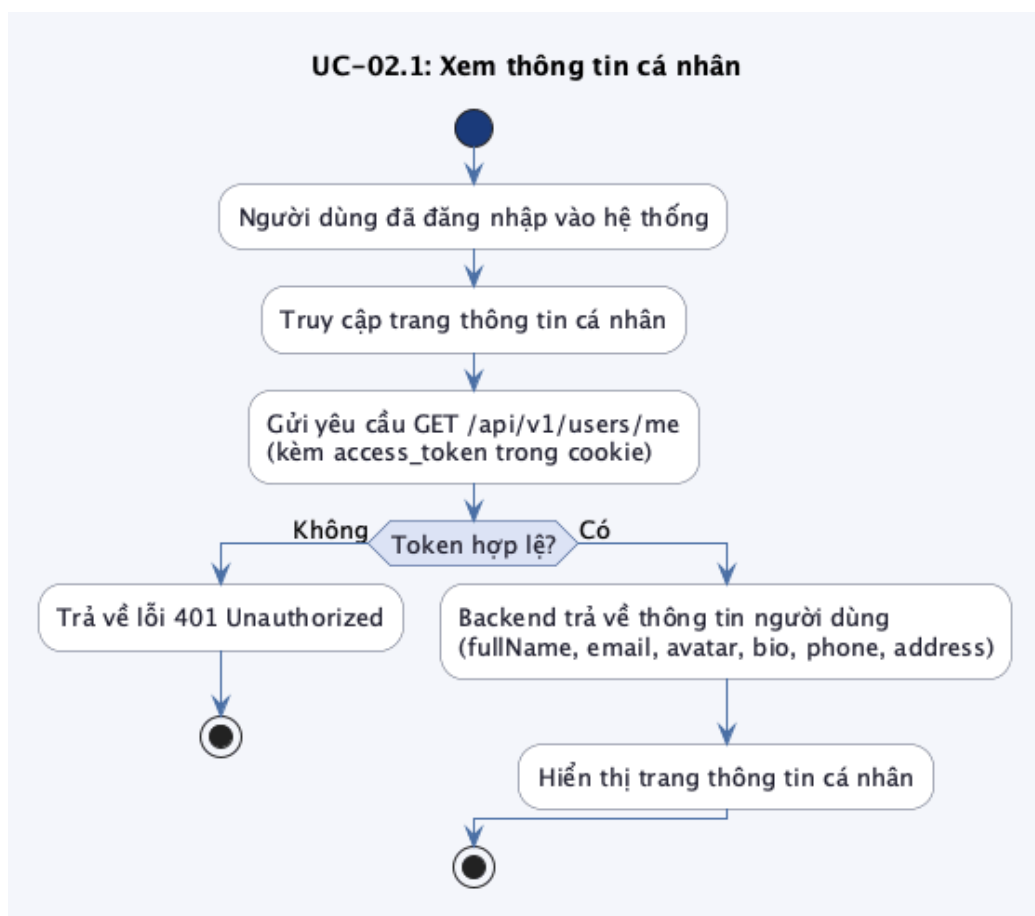
Hình 19: Biểu đồ hoạt động UC-01.5: Làm mới access token

3.1.4.2 UC-02: Quản lý người dùng

Nhóm chức năng quản lý người dùng gồm hai luồng: xem và chỉnh sửa thông tin cá nhân. Cả hai luồng đều yêu cầu token hợp lệ để xác thực danh tính người dùng.

UC-02.1: Xem thông tin cá nhân

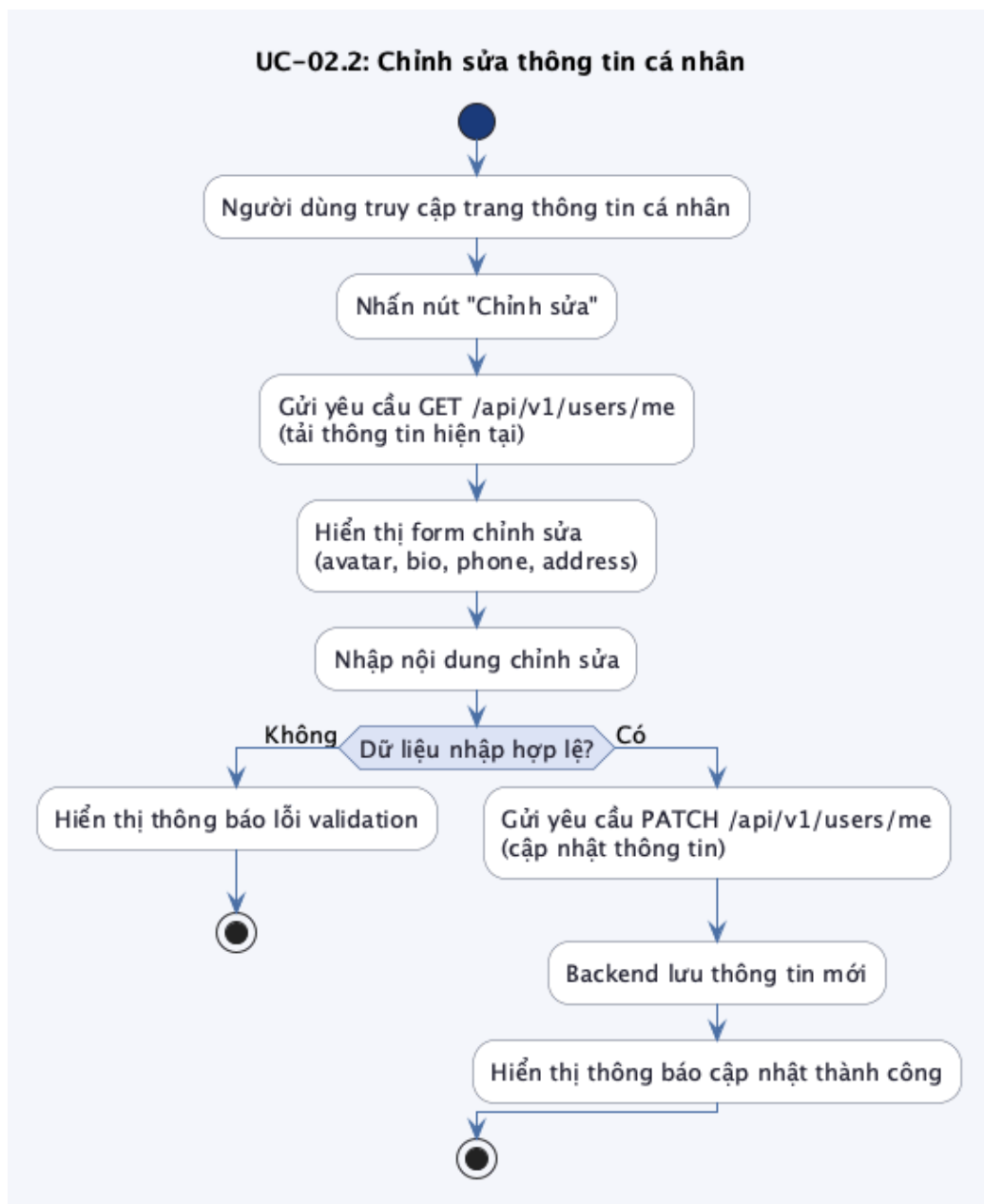
Hệ thống xác thực token, gọi API lấy thông tin hồ sơ và hiển thị đầy đủ thông tin cá nhân của người dùng đang đăng nhập.



Hình 20: Biểu đồ hoạt động UC-02.1: Xem thông tin cá nhân

UC-02.2: Chỉnh sửa thông tin cá nhân

Người dùng tải thông tin hiện tại, chỉnh sửa trên form và gửi yêu cầu cập nhật; hệ thống kiểm tra tính hợp lệ, lưu thay đổi và thông báo kết quả.



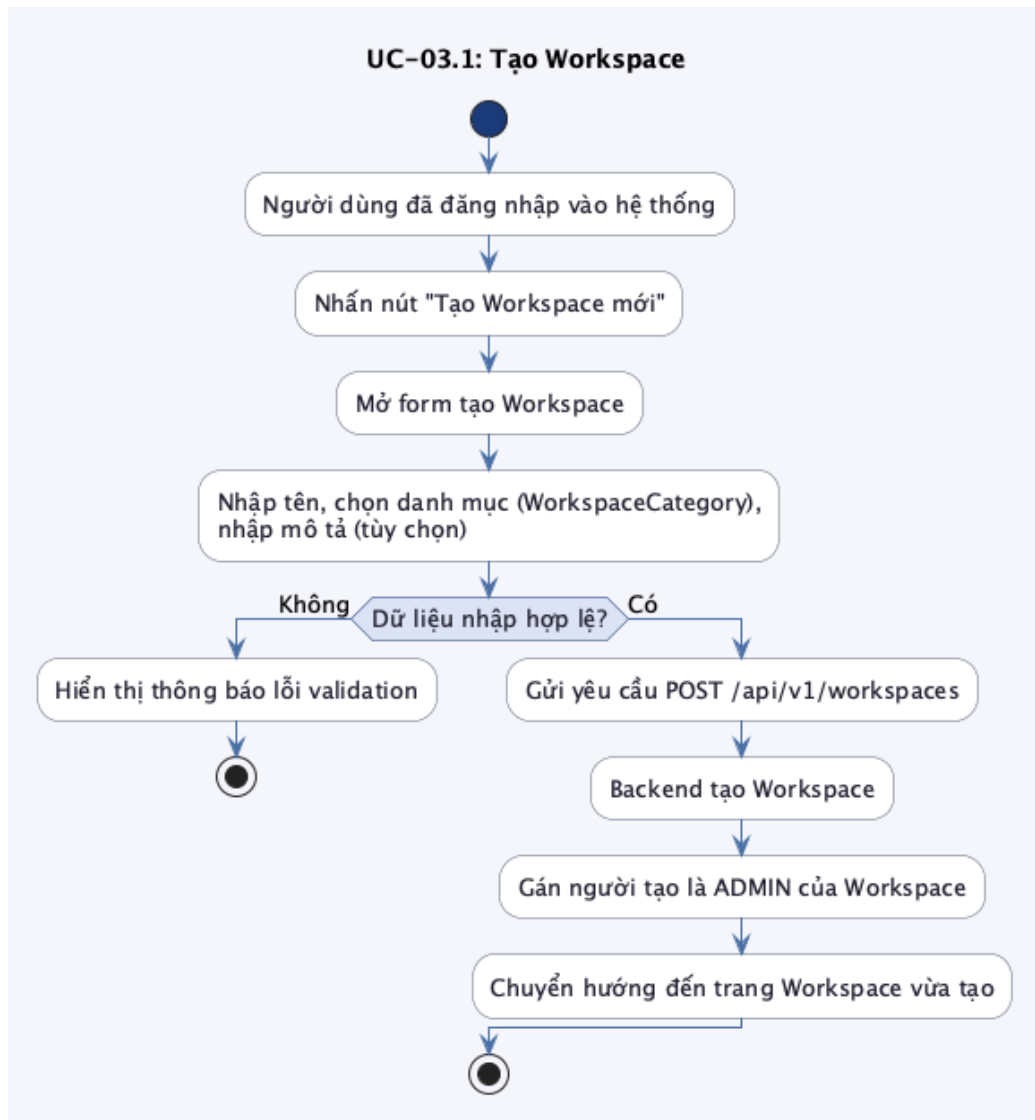
Hình 21: Biểu đồ hoạt động UC-02.2: Chỉnh sửa thông tin cá nhân

3.1.4.3 UC-03: Quản lý Workspace

Nhóm chức năng quản lý Workspace gồm năm luồng: tạo, xem danh sách, xem chi tiết, mời thành viên và xem danh sách thành viên. Người tạo Workspace được tự động gán vai trò quản trị viên.

UC-03.1: Tạo Workspace

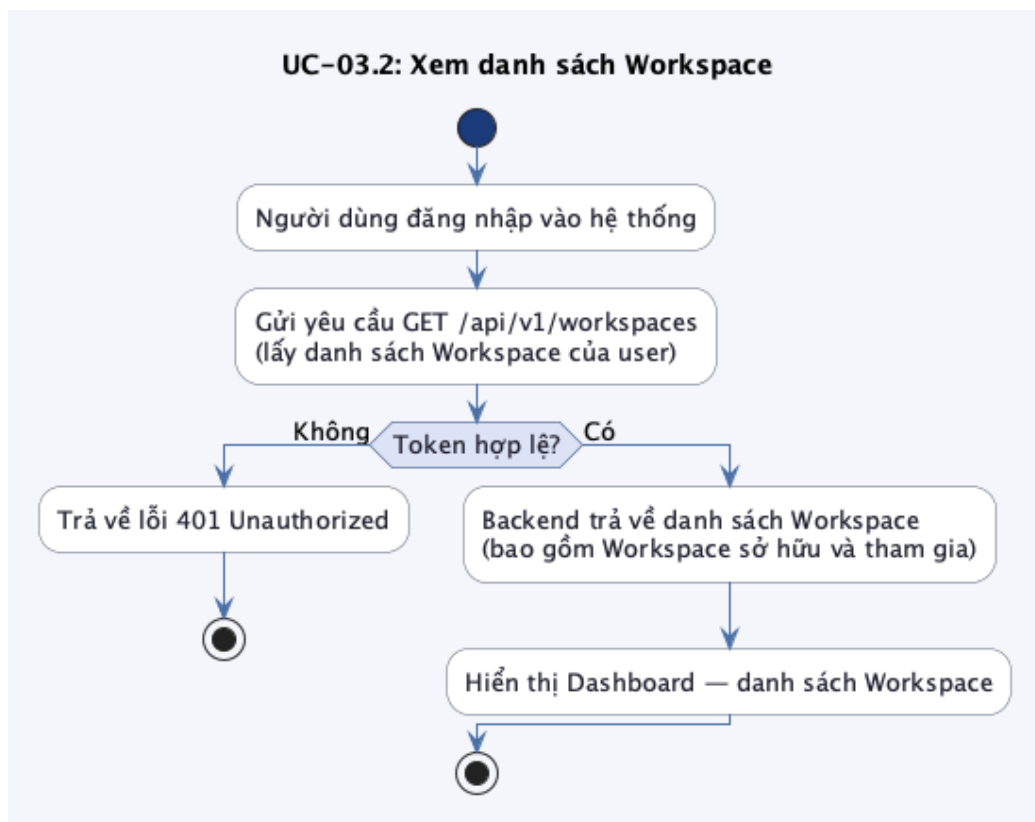
Người dùng nhập tên, chọn danh mục và mô tả; hệ thống tạo Workspace và gán người tạo vai trò ADMIN, sau đó chuyển hướng đến trang chi tiết vừa tạo.



Hình 22: Biểu đồ hoạt động UC-03.1: Tạo Workspace

UC-03.2: Xem danh sách Workspace

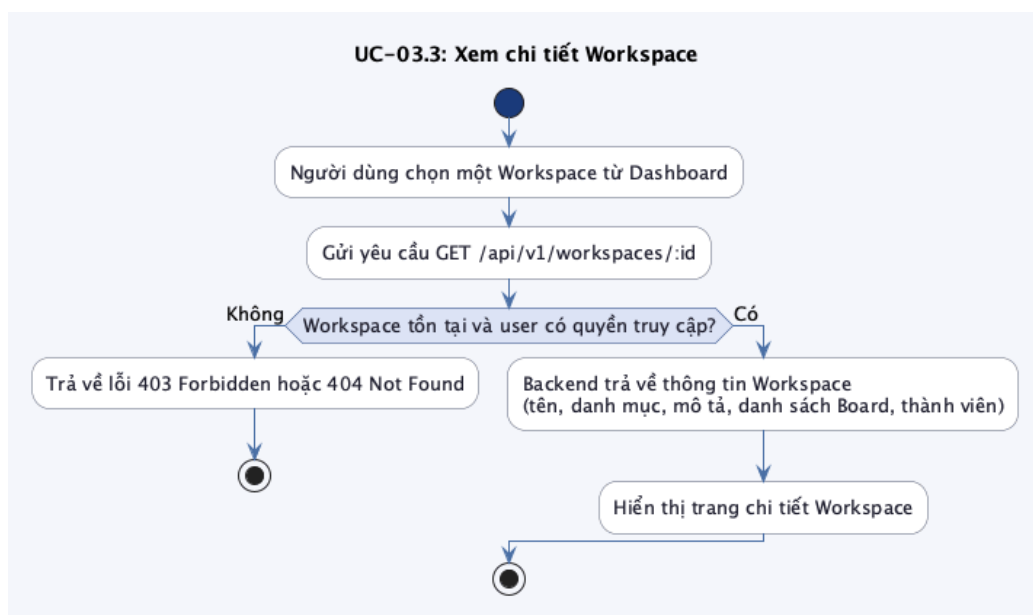
Hệ thống tải toàn bộ Workspace mà người dùng sở hữu hoặc là thành viên và hiển thị trên trang Dashboard.



Hình 23: Biểu đồ hoạt động UC-03.2: Xem danh sách Workspace

UC-03.3: Xem chi tiết Workspace

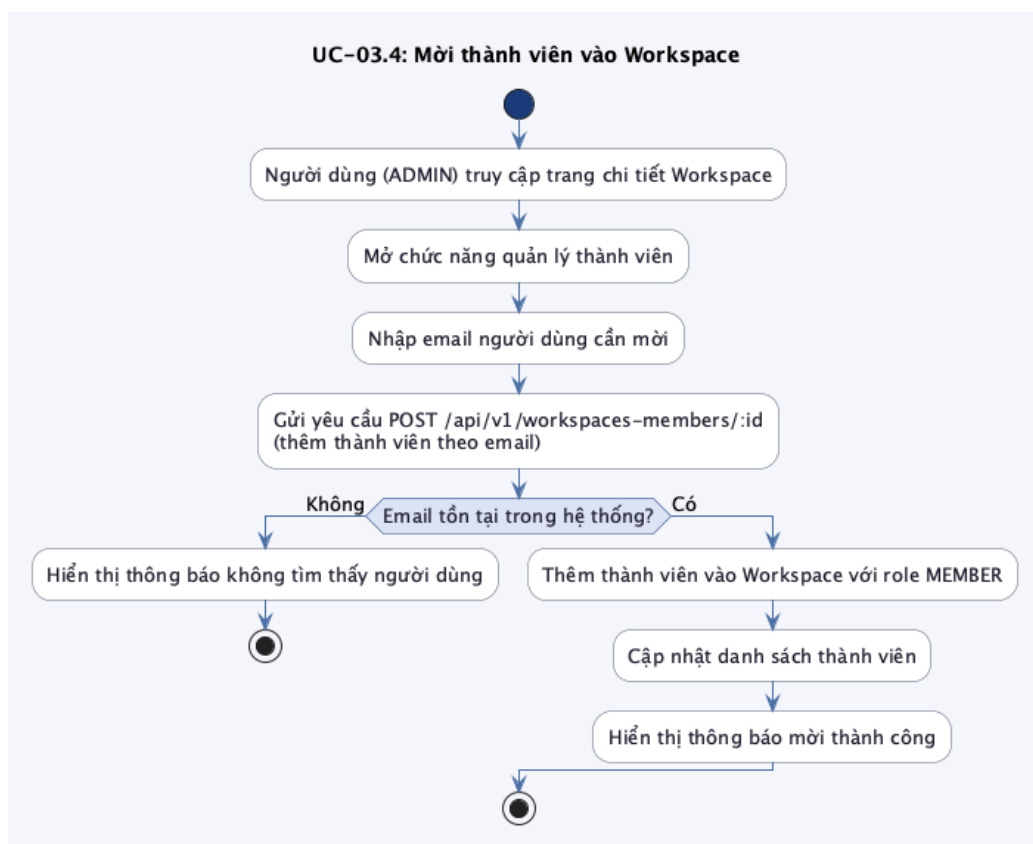
Hệ thống kiểm tra quyền truy cập và trả về thông tin Workspace gồm tên, danh mục, mô tả, danh sách Board và thành viên.



Hình 24: Biểu đồ hoạt động UC-03.3: Xem chi tiết Workspace

UC-03.4: Mời thành viên vào Workspace

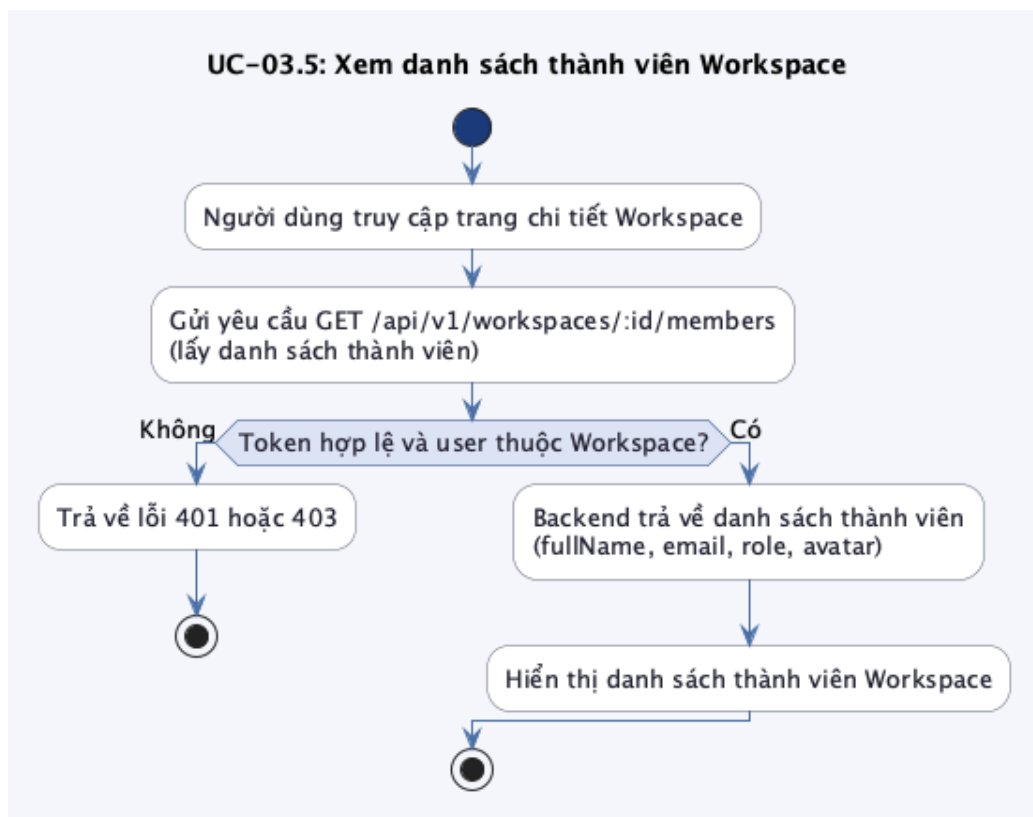
Quản trị viên nhập email người dùng cần mời; hệ thống xác minh email tồn tại trong hệ thống, thêm thành viên với vai trò MEMBER và thông báo kết quả.



Hình 25: Biểu đồ hoạt động UC-03.4: Mời thành viên vào Workspace

UC-03.5: Xem danh sách thành viên Workspace

Hệ thống xác thực quyền truy cập và trả về danh sách thành viên Workspace kèm vai trò và thông tin cá nhân tương ứng.



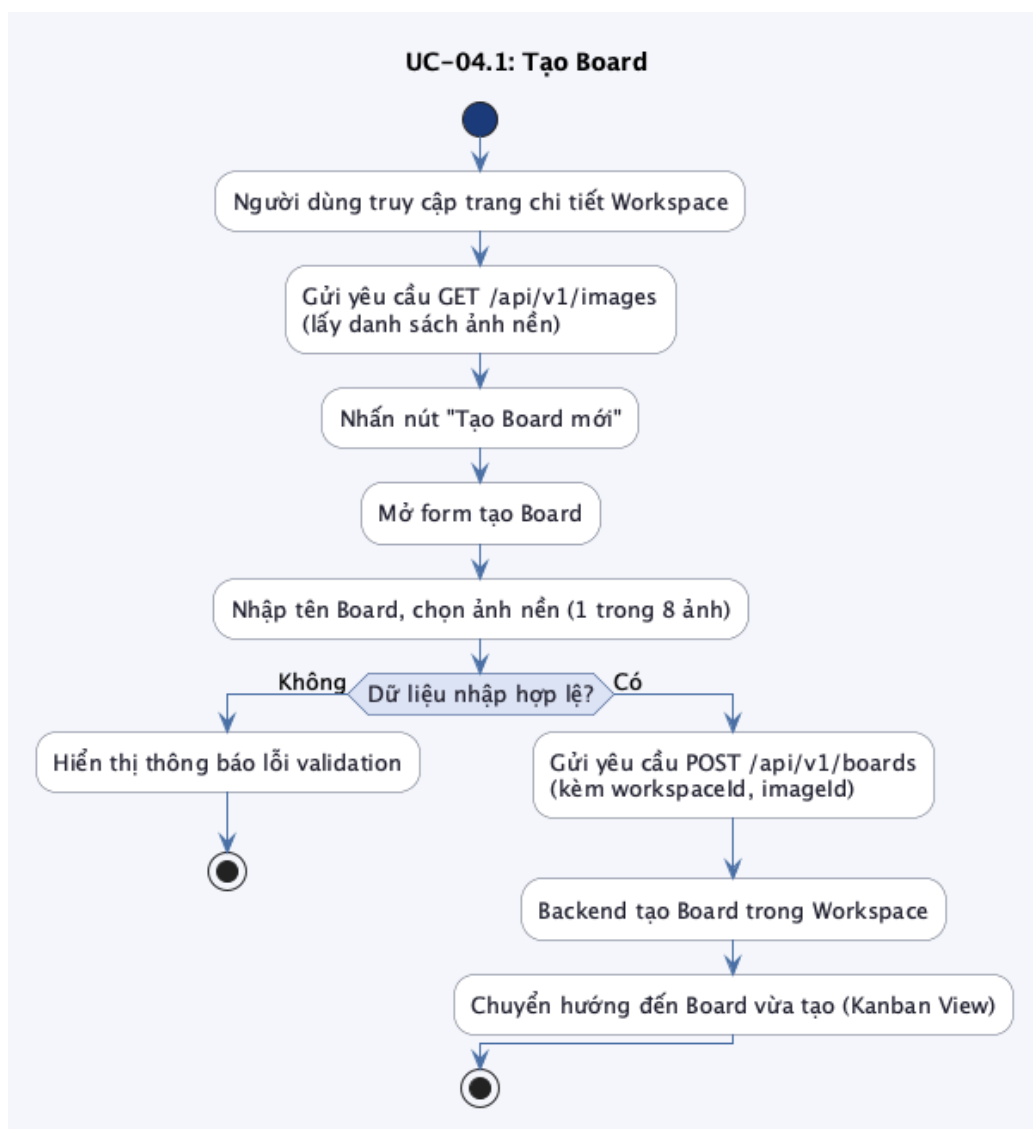
Hình 26: Biểu đồ hoạt động UC-03.5: Xem danh sách thành viên Workspace

3.1.4.4 UC-04: Quản lý Board

Nhóm chức năng quản lý Board gồm năm luồng: tạo, xem chi tiết, cập nhật tên, xóa và thêm thành viên. Các thao tác thay đổi cấu trúc Board yêu cầu người dùng có vai trò quản trị viên.

UC-04.1: Tạo Board

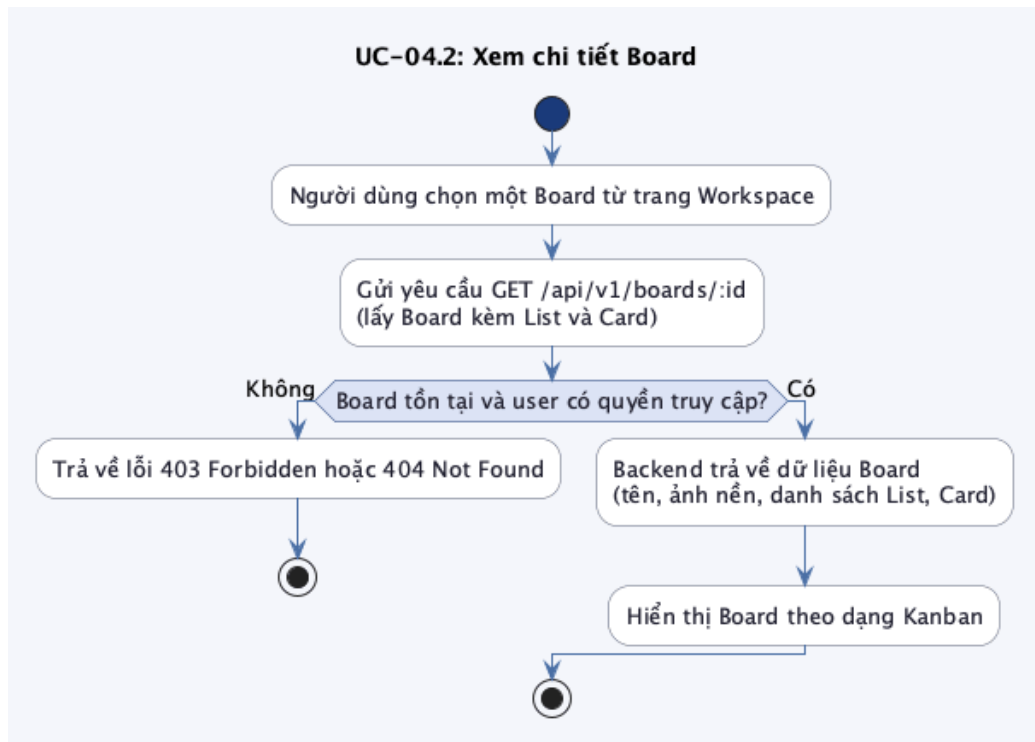
Người dùng nhập tên và chọn ảnh nền từ danh sách có sẵn; hệ thống khởi tạo Board trong Workspace và chuyển hướng đến giao diện Kanban.



Hình 27: Biểu đồ hoạt động UC-04.1: Tạo Board

UC-04.2: Xem chi tiết Board

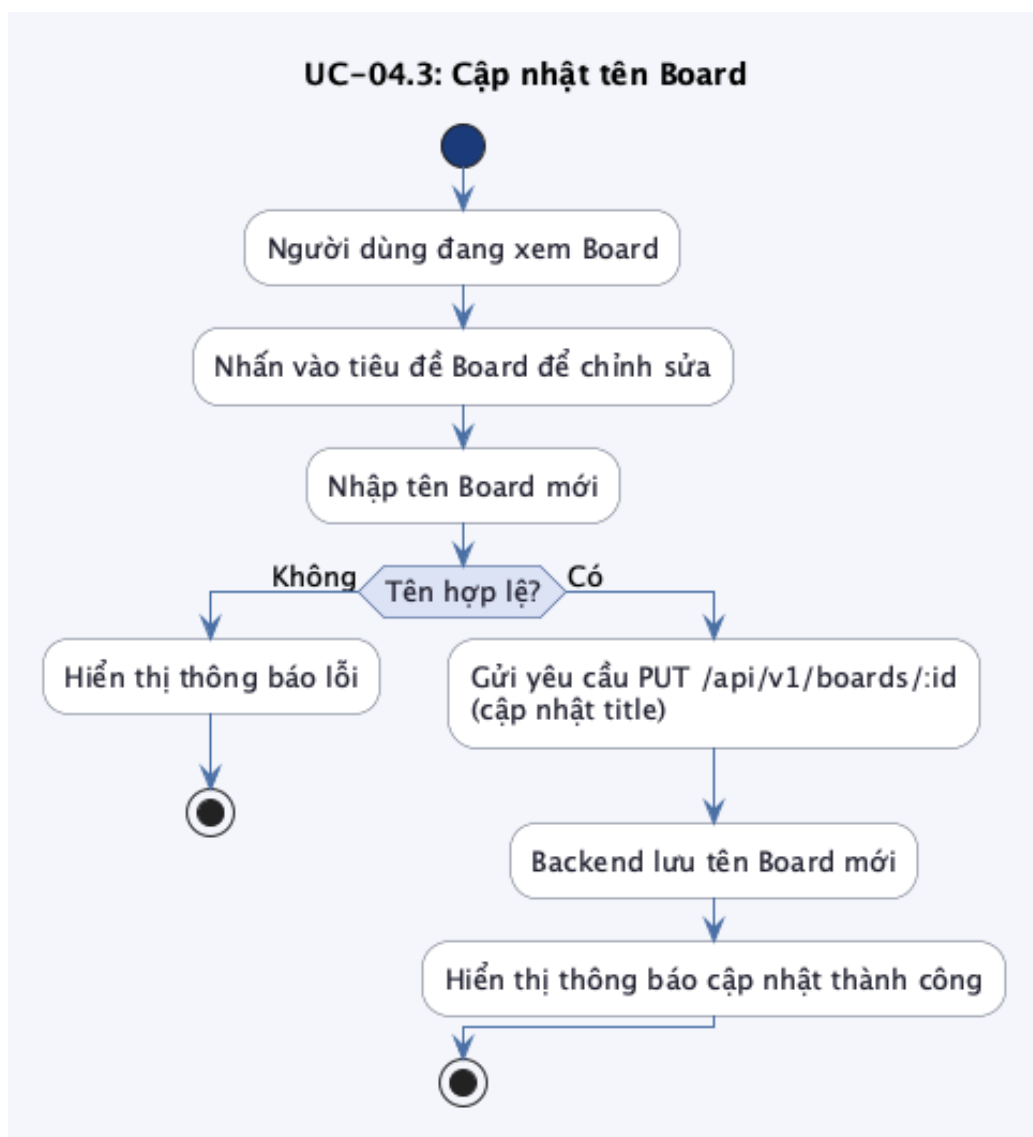
Hệ thống kiểm tra quyền truy cập, tải toàn bộ dữ liệu Board bao gồm các List và Card, rồi hiển thị theo dạng Kanban.



Hình 28: Biểu đồ hoạt động UC-04.2: Xem chi tiết Board

UC-04.3: Cập nhật tên Board

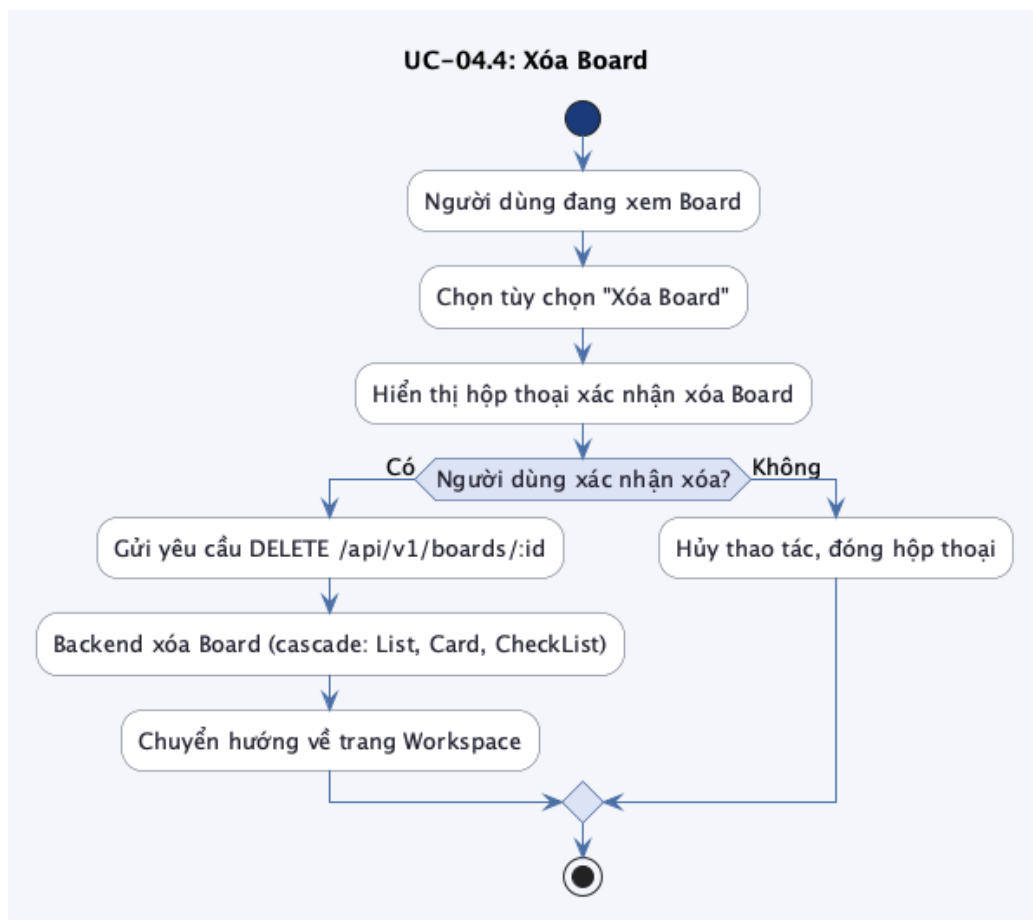
Người dùng chỉnh sửa tiêu đề Board trực tiếp trên giao diện; hệ thống xác thực và lưu tên mới, thông báo cập nhật thành công.



Hình 29: Biểu đồ hoạt động UC-04.3: Cập nhật tên Board

UC-04.4: Xóa Board

Sau khi xác nhận xóa, hệ thống xóa Board cùng toàn bộ dữ liệu liên quan (List, Card, Checklist) theo cơ chế cascade và chuyển hướng về trang Workspace.



Hình 30: Biểu đồ hoạt động UC-04.4: Xóa Board

UC-04.5: Thêm thành viên vào Board

Quản trị viên chọn thành viên từ danh sách Workspace và gán vai trò; hệ thống thêm thành viên vào Board với quyền hạn tương ứng.



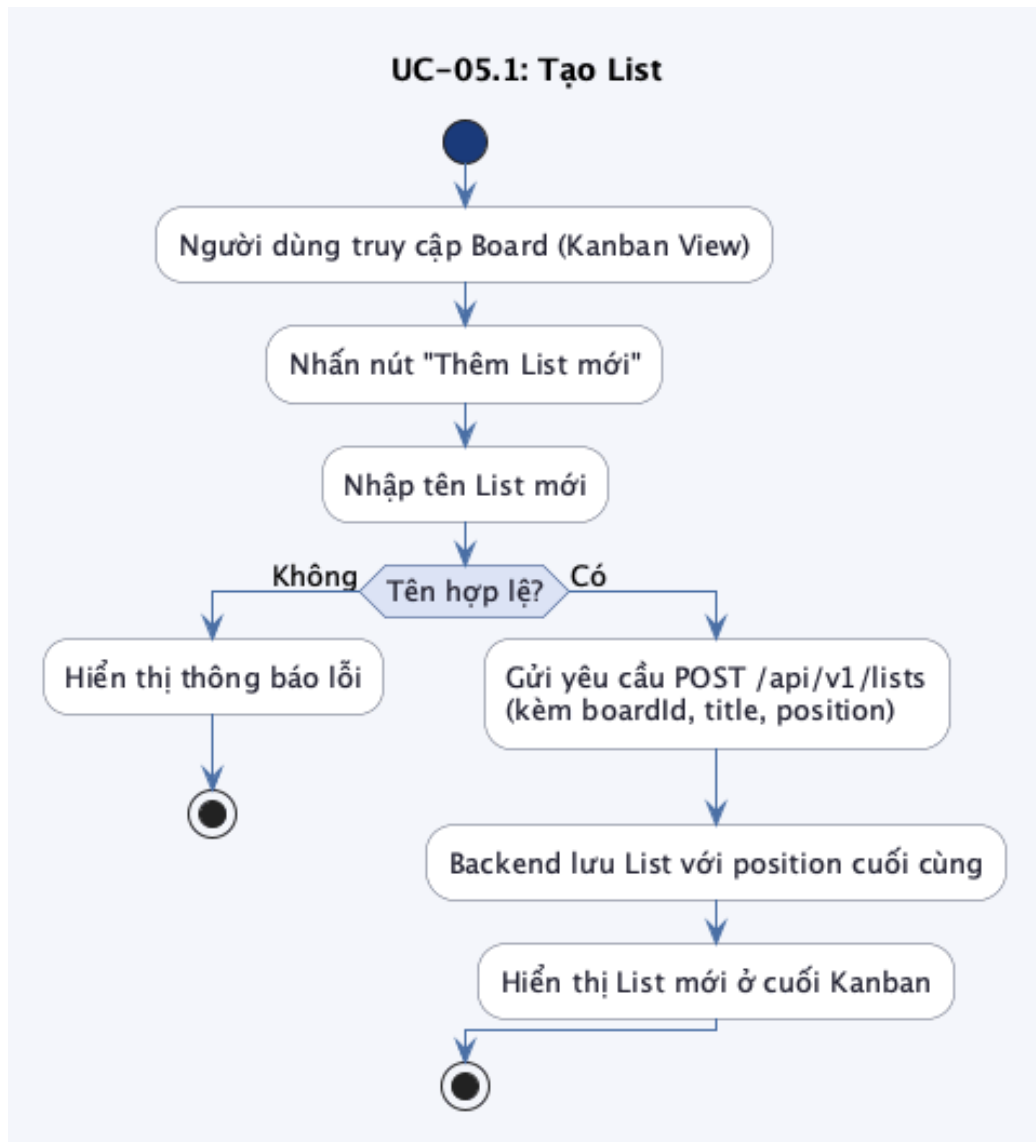
Hình 31: Biểu đồ hoạt động UC-04.5: Thêm thành viên vào Board

3.1.4.5 UC-05: Quản lý List

Nhóm chức năng quản lý List gồm bốn luồng: tạo, đổi tên, xóa và sắp xếp thứ tự. Tính năng sắp xếp áp dụng optimistic update để đảm bảo trải nghiệm tương tác tức thì.

UC-05.1: Tạo List

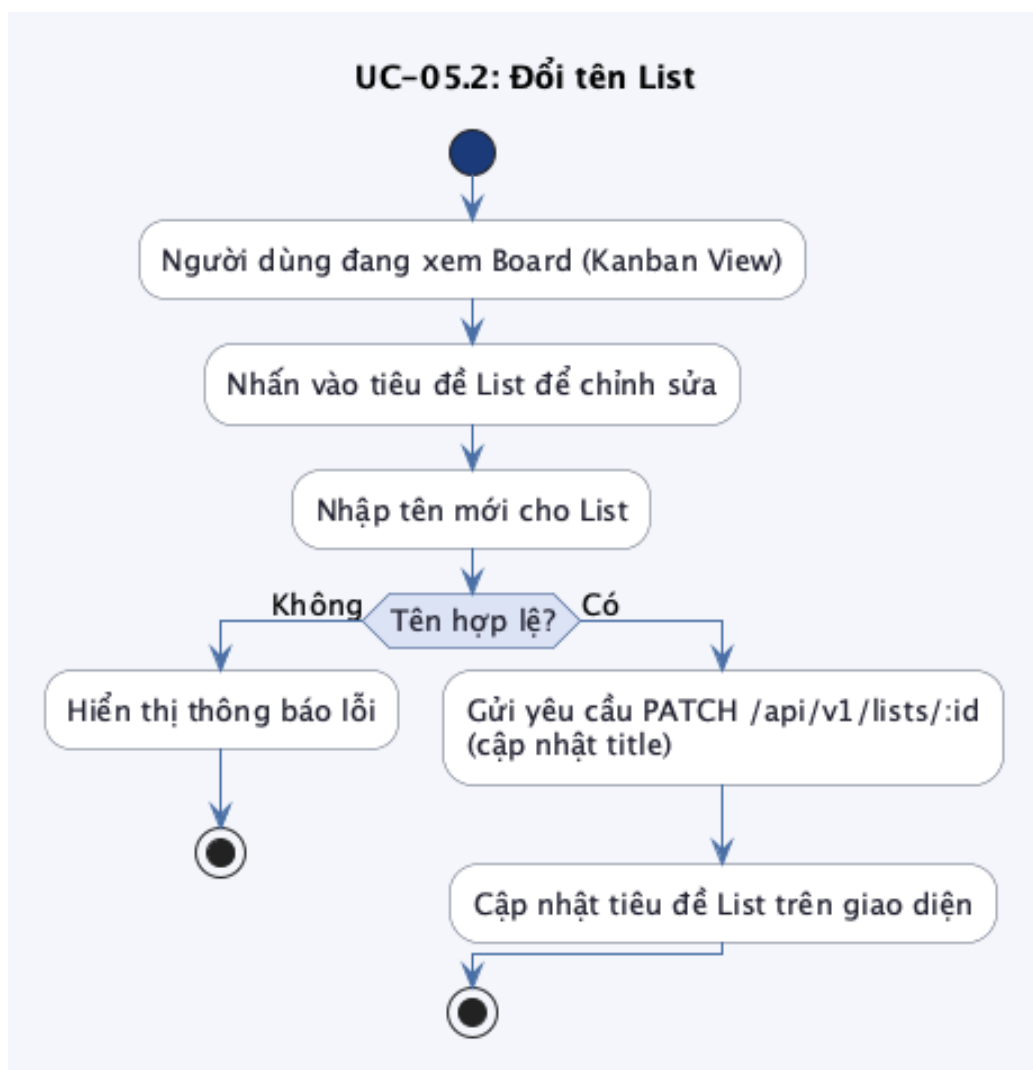
Người dùng nhập tên List mới; hệ thống lưu List với vị trí cuối cùng trong Board và hiển thị cột mới trên giao diện Kanban.



Hình 32: Biểu đồ hoạt động UC-05.1: Tạo List

UC-05.2: Đổi tên List

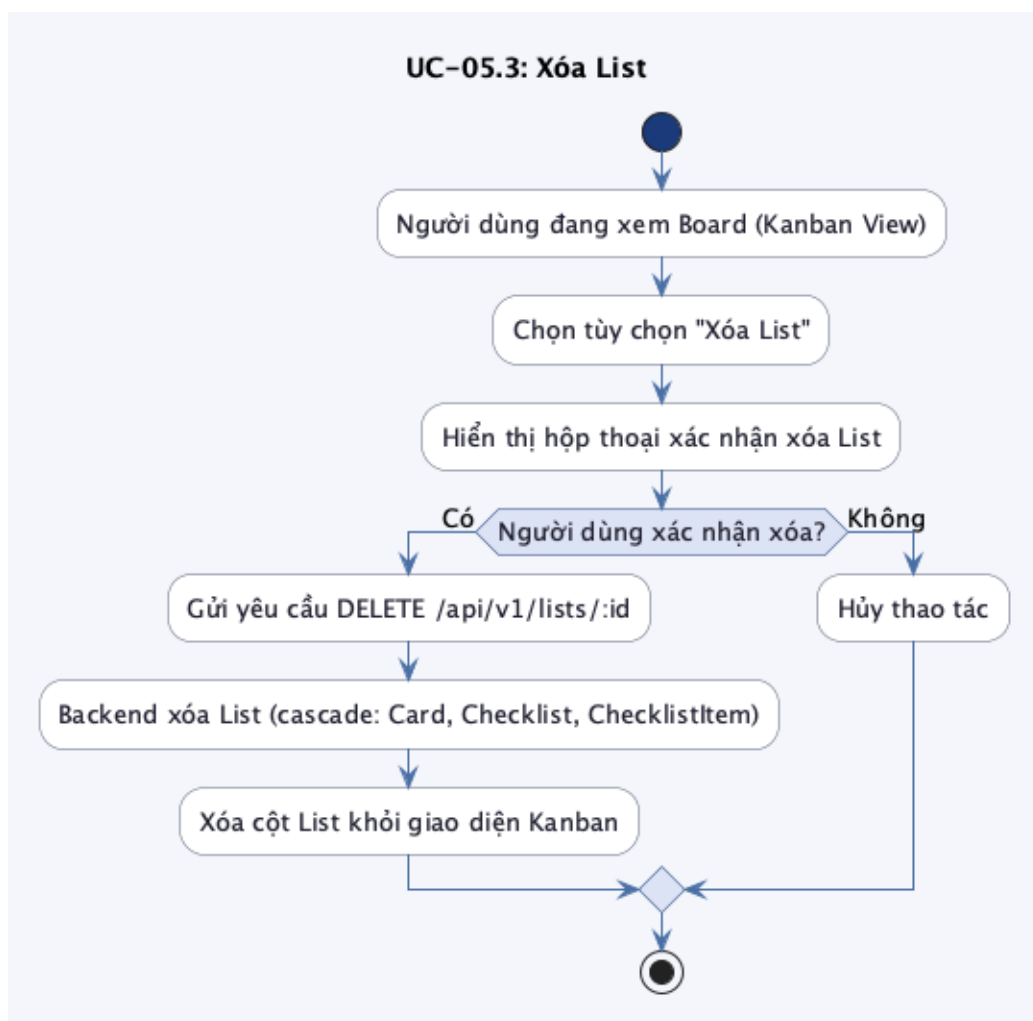
Người dùng chỉnh sửa tiêu đề List trực tiếp; hệ thống xác thực và cập nhật tên mới lên giao diện và cơ sở dữ liệu.



Hình 33: Biểu đồ hoạt động UC-05.2: Đổi tên List

UC-05.3: Xóa List

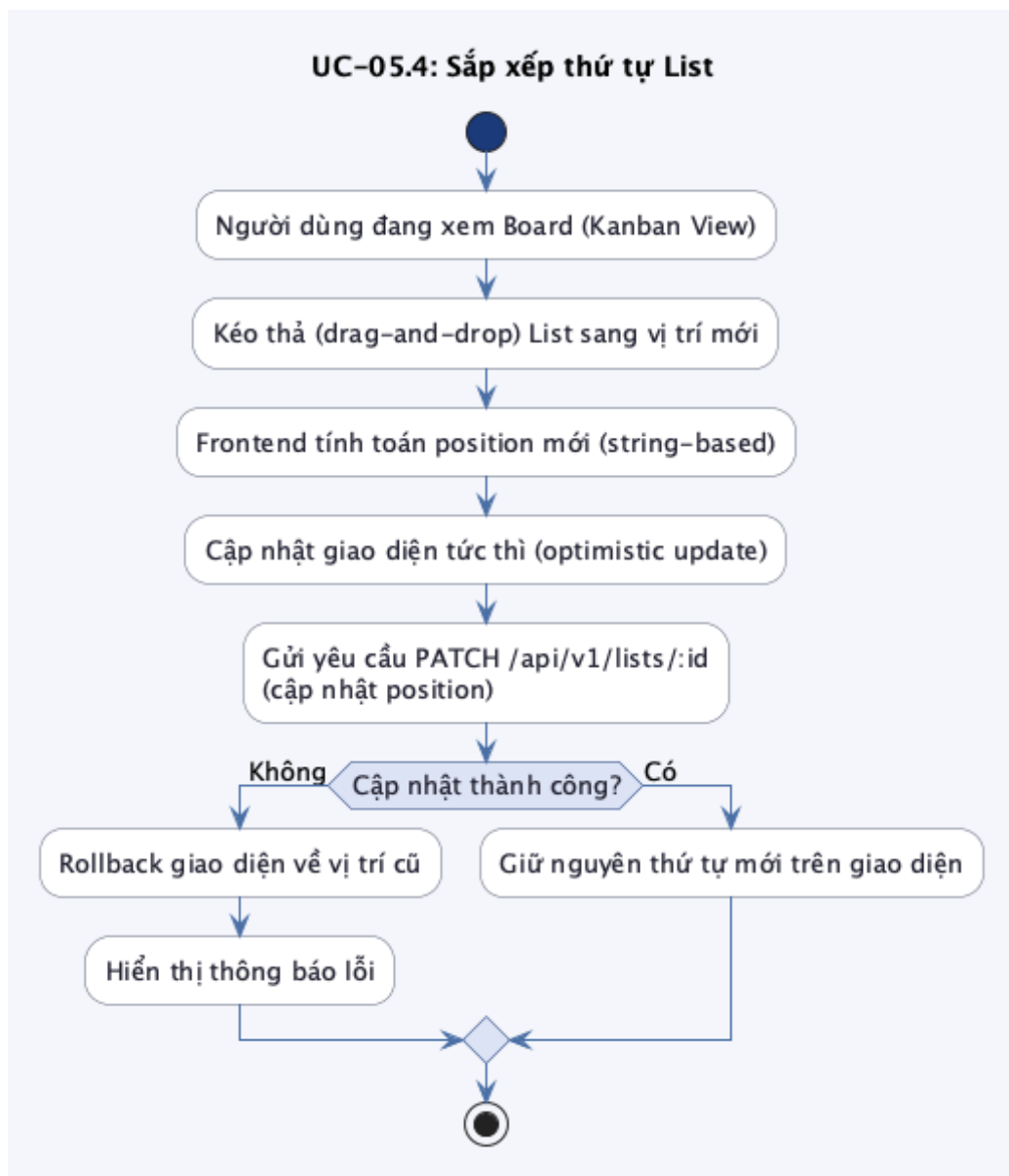
Sau khi xác nhận, hệ thống xóa List và toàn bộ dữ liệu liên quan (Card, Checklist, ChecklistItem) theo cơ chế cascade, đồng thời xóa cột tương ứng khỏi giao diện Kanban.



Hình 34: Biểu đồ hoạt động UC-05.3: Xóa List

UC-05.4: Sắp xếp thứ tự List

Người dùng kéo thả List sang vị trí mới; giao diện cập nhật tức thì (optimistic update) và đồng bộ với máy chủ; nếu thất bại, giao diện tự động khôi phục về thứ tự cũ.



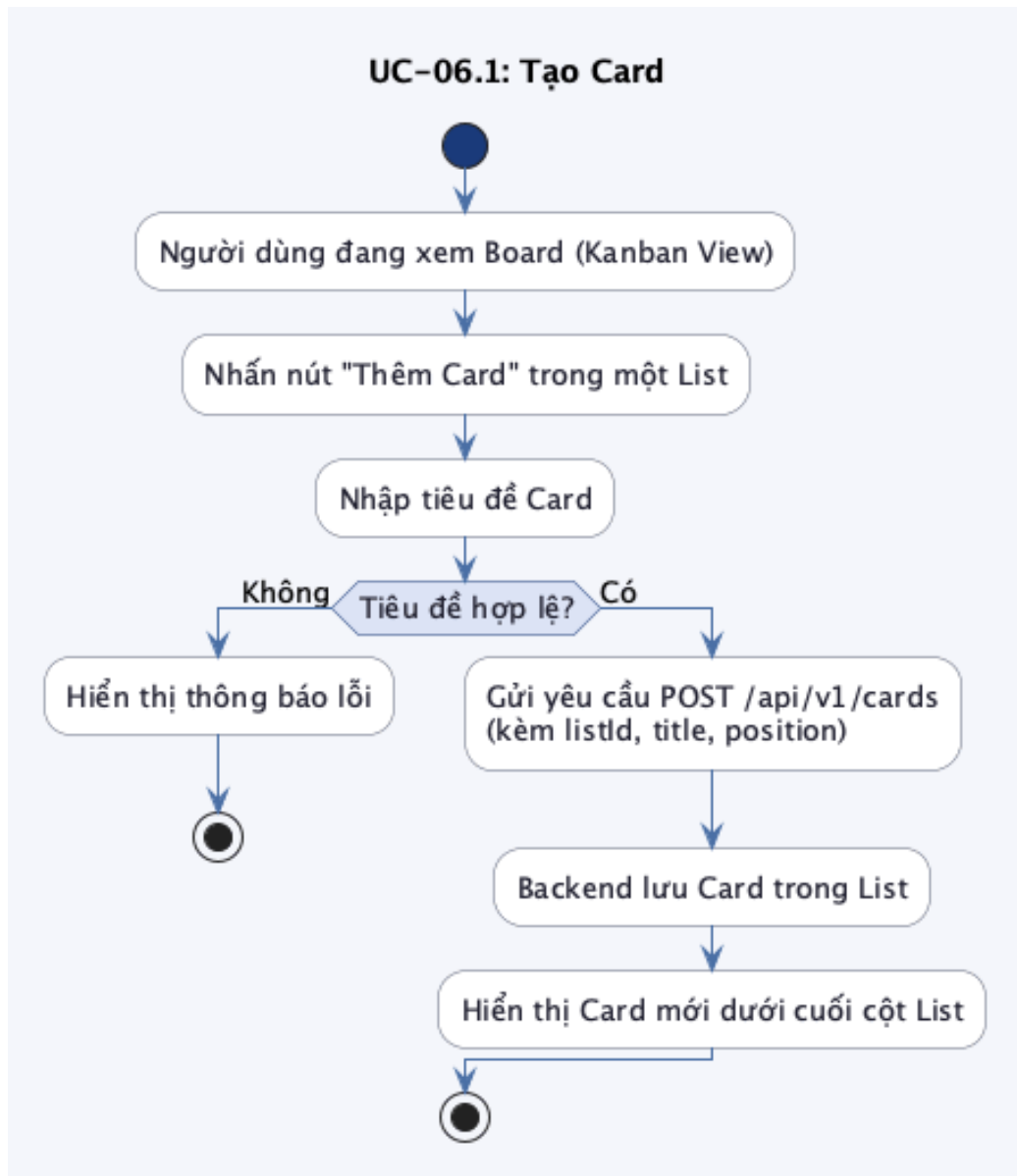
Hình 35: Biểu đồ hoạt động UC-05.4: Sắp xếp thứ tự List

3.1.4.6 UC-06: Quản lý Card

Nhóm chức năng quản lý Card gồm sáu luồng: tạo, xem chi tiết, cập nhật, đánh dấu hoàn thành, di chuyển giữa các List và xóa. Card là thực thể trung tâm của hệ thống, liên kết với Checklist và thành viên phụ trách.

UC-06.1: Tạo Card

Người dùng nhập tiêu đề Card trong một List; hệ thống lưu Card với vị trí cuối cùng trong List và hiển thị Card mới trên cột Kanban.

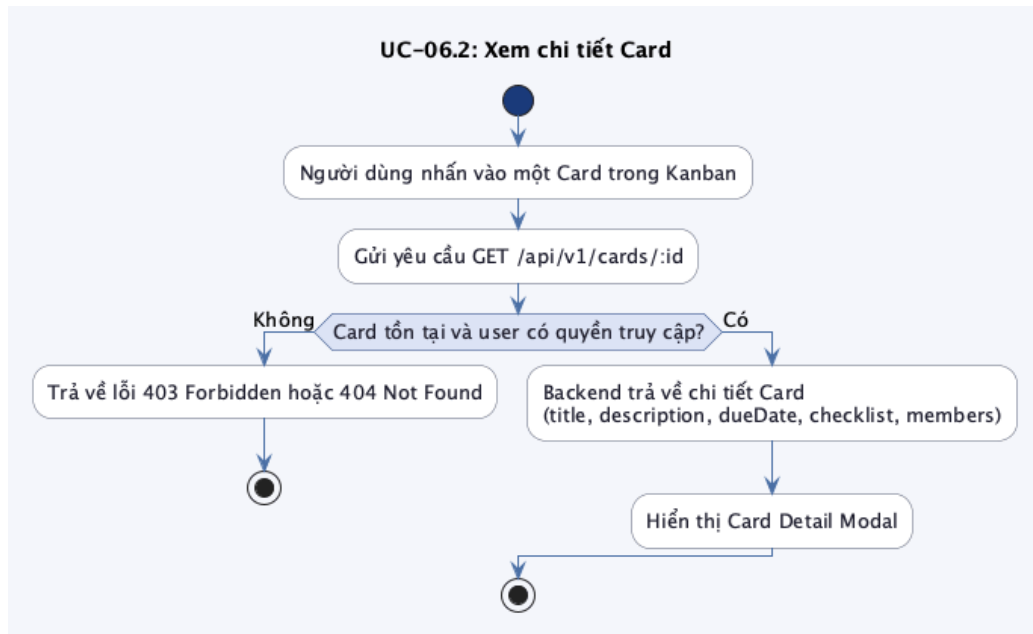


Hình 36: Biểu đồ hoạt động UC-06.1: Tạo Card

UC-06.2: Xem chi tiết Card

Hệ thống kiểm tra quyền truy cập và trả về đầy đủ thông tin Card gồm tiêu đề, mô tả, ngày hạn, Checklist và danh sách thành viên phụ trách, hiển

thị trong Card Detail Modal.



Hình 37: Biểu đồ hoạt động UC-06.2: Xem chi tiết Card

UC-06.3: Cập nhật Card

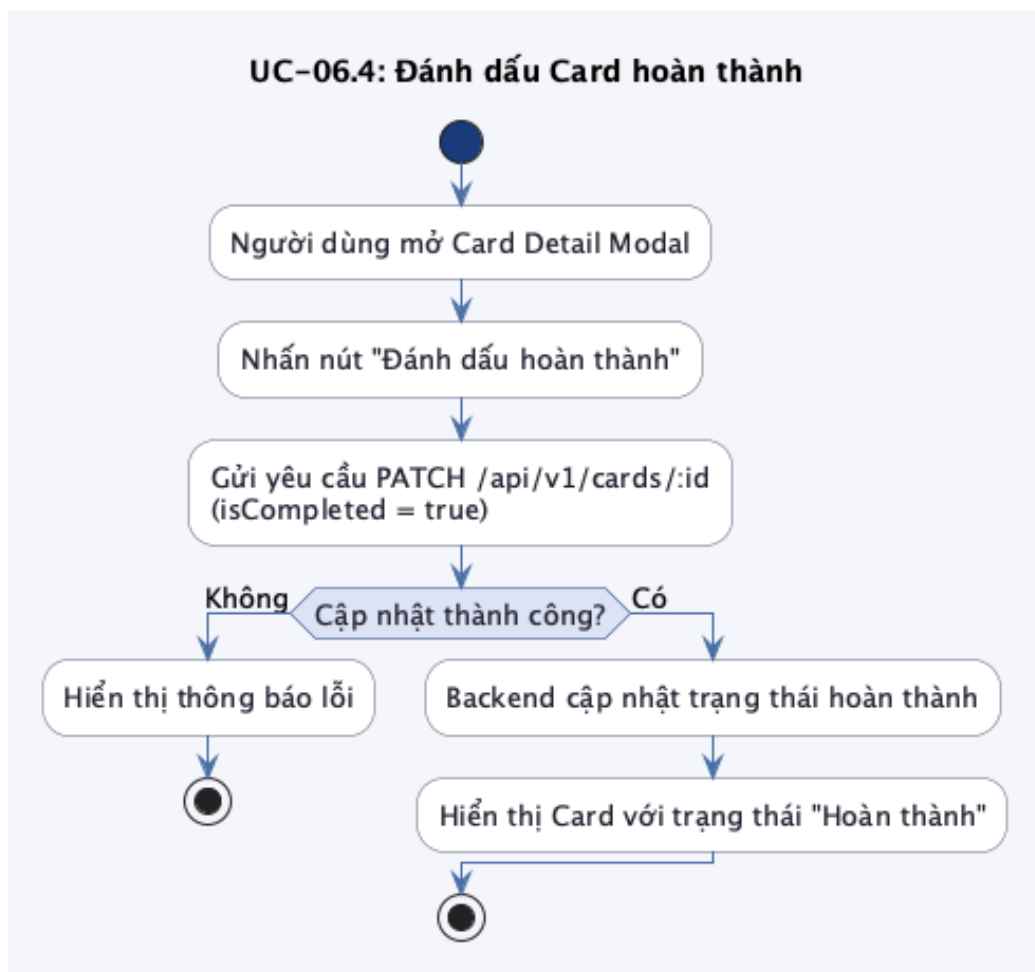
Người dùng chỉnh sửa các trường thông tin của Card (tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn); hệ thống xác thực dữ liệu và lưu thay đổi.



Hình 38: Biểu đồ hoạt động UC-06.3: Cập nhật Card

UC-06.4: Đánh dấu Card hoàn thành

Người dùng nhấn nút đánh dấu hoàn thành; hệ thống cập nhật trạng thái `isCompleted = true` và phản ánh thay đổi trên giao diện Card.



Hình 39: Biểu đồ hoạt động UC-06.4: Đánh dấu Card hoàn thành

UC-06.5: Di chuyển Card giữa các List

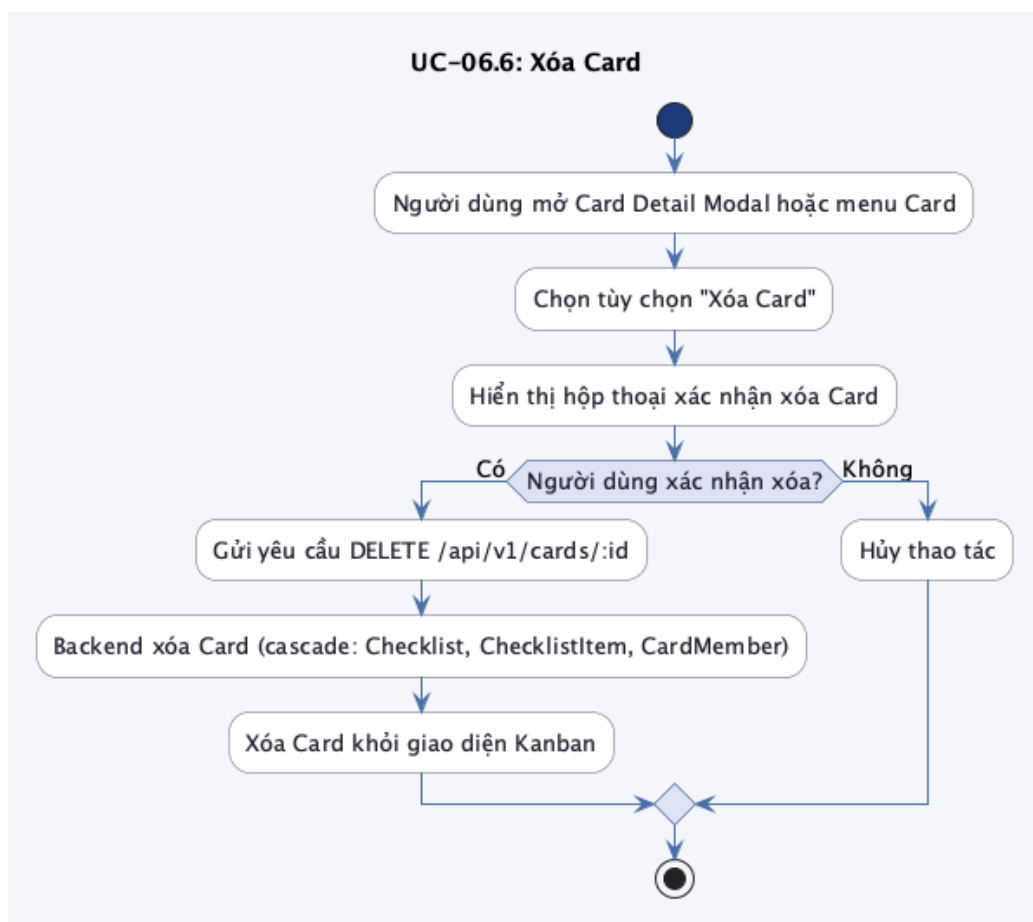
Người dùng kéo thả Card sang List đích; giao diện cập nhật tức thì (optimistic update) và đồng bộ vị trí mới với máy chủ; nếu thất bại, Card được khôi phục về vị trí ban đầu.



Hình 40: Biểu đồ hoạt động UC-06.5: Di chuyển Card giữa các List

UC-06.6: Xóa Card

Sau khi xác nhận, hệ thống xóa Card và toàn bộ dữ liệu liên quan (Checklist, ChecklistItem, CardMember) theo cơ chế cascade.



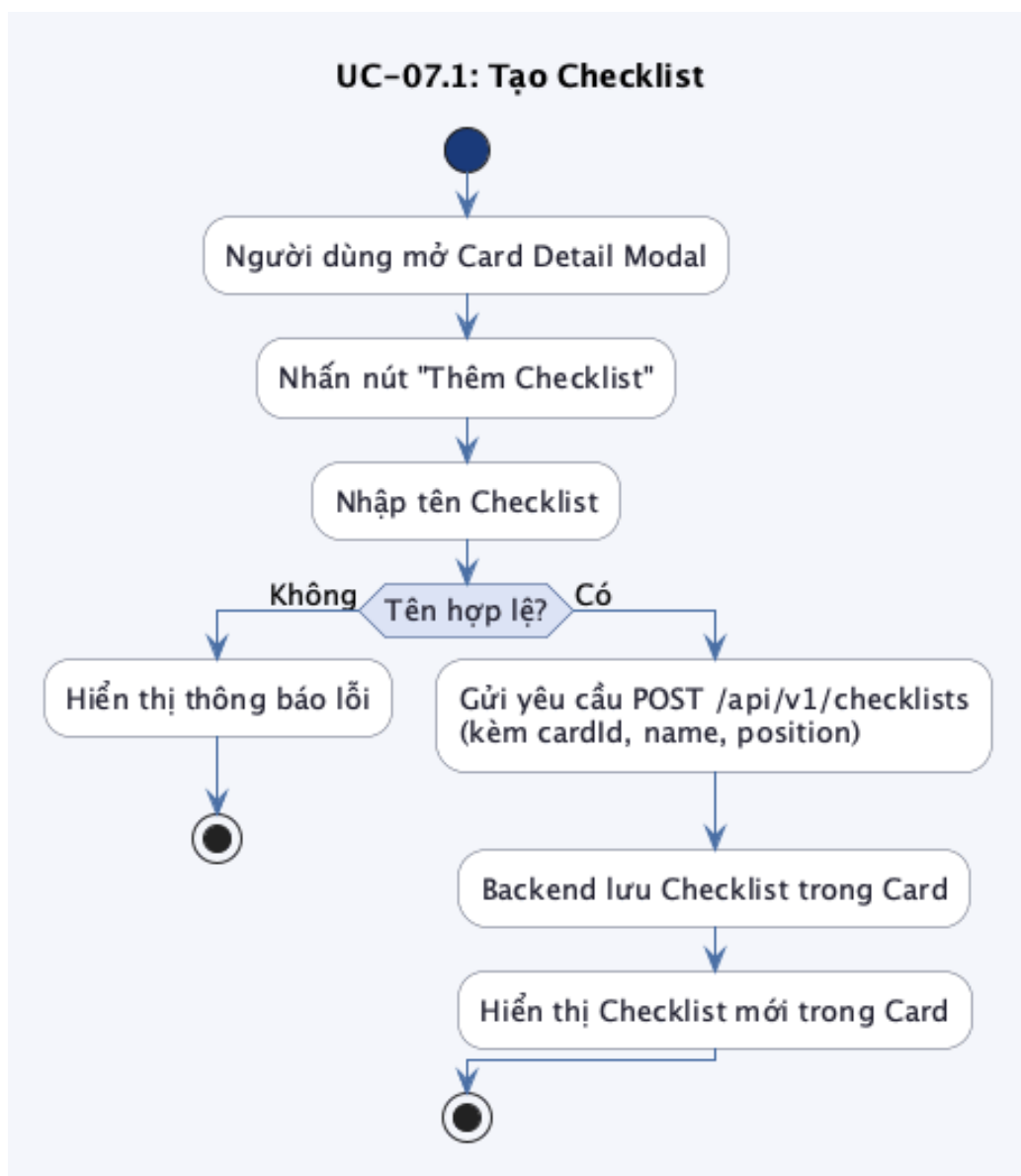
Hình 41: Biểu đồ hoạt động UC-06.6: Xóa Card

3.1.4.7 UC-07: Quản lý Checklist

Nhóm chức năng quản lý Checklist gồm bảy luồng: tạo, cập nhật và xóa Checklist; tạo, cập nhật, đánh dấu hoàn thành và xóa Checklist Item. Thanh tiến độ (progress bar) được cập nhật tức thì sau mỗi thay đổi trạng thái Item.

UC-07.1: Tạo Checklist

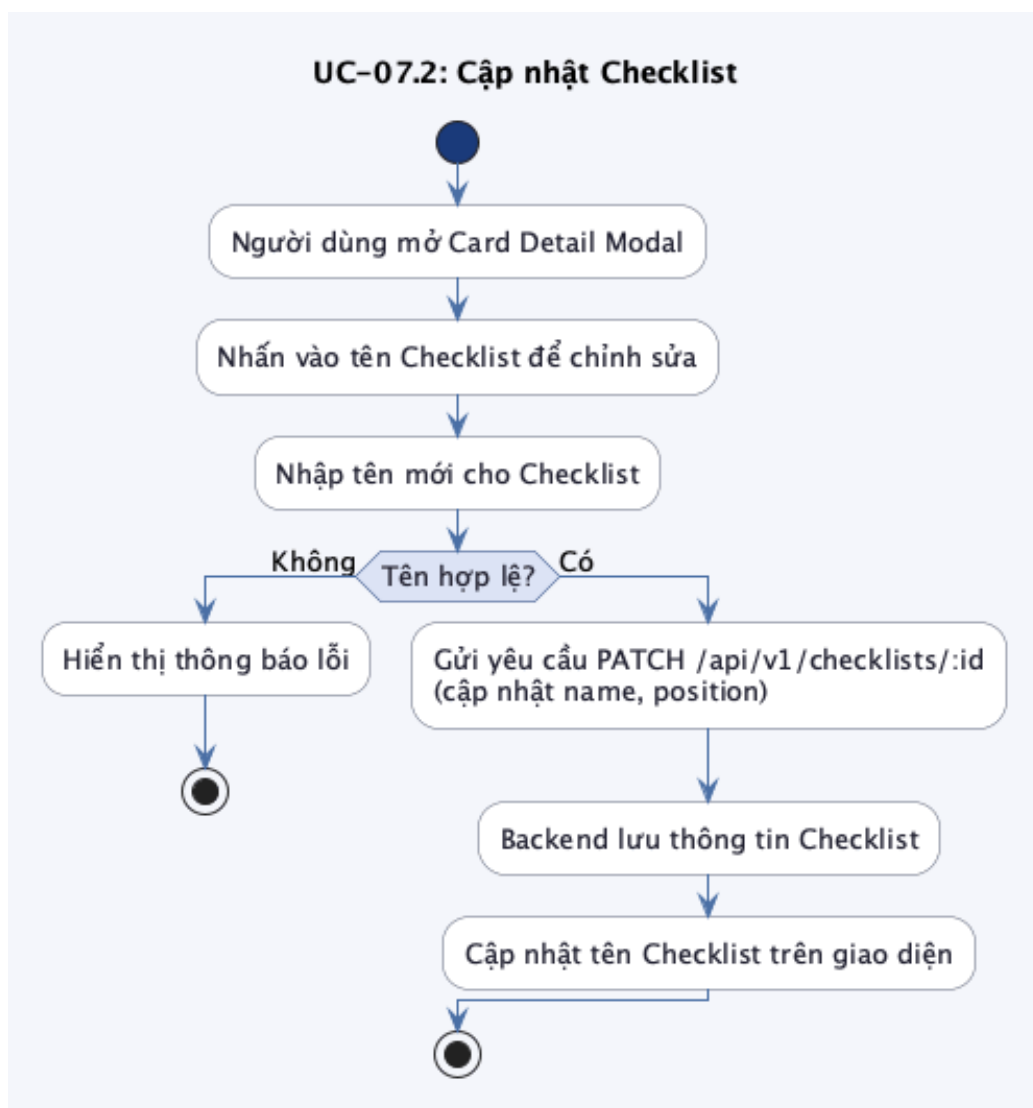
Người dùng nhập tên Checklist trong Card Detail Modal; hệ thống lưu Checklist và hiển thị ngay trong giao diện Card.



Hình 42: Biểu đồ hoạt động UC-07.1: Tạo Checklist

UC-07.2: Cập nhật Checklist

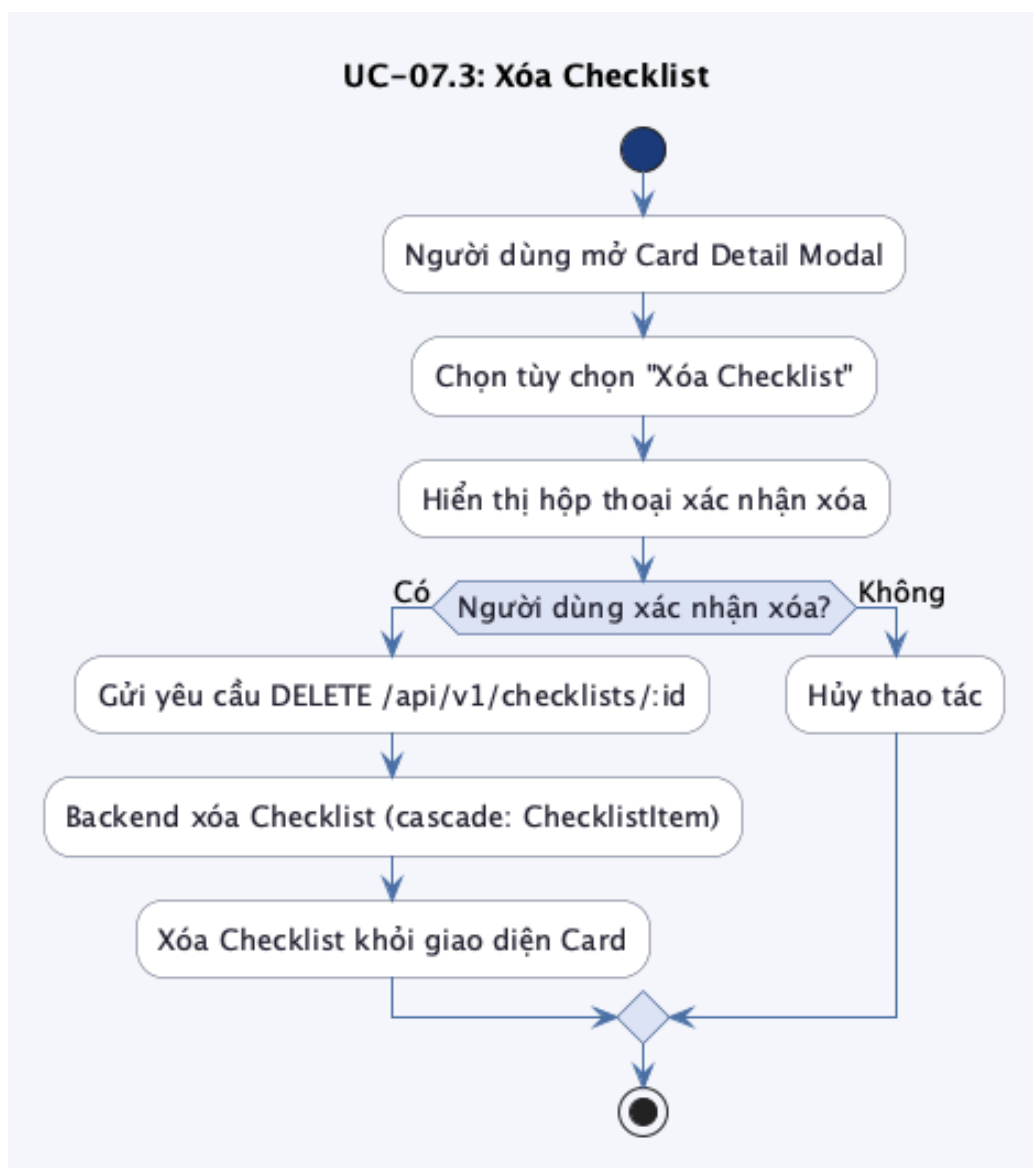
Người dùng chỉnh sửa tên Checklist; hệ thống lưu thông tin mới và cập nhật giao diện.



Hình 43: Biểu đồ hoạt động UC-07.2: Cập nhật Checklist

UC-07.3: Xóa Checklist

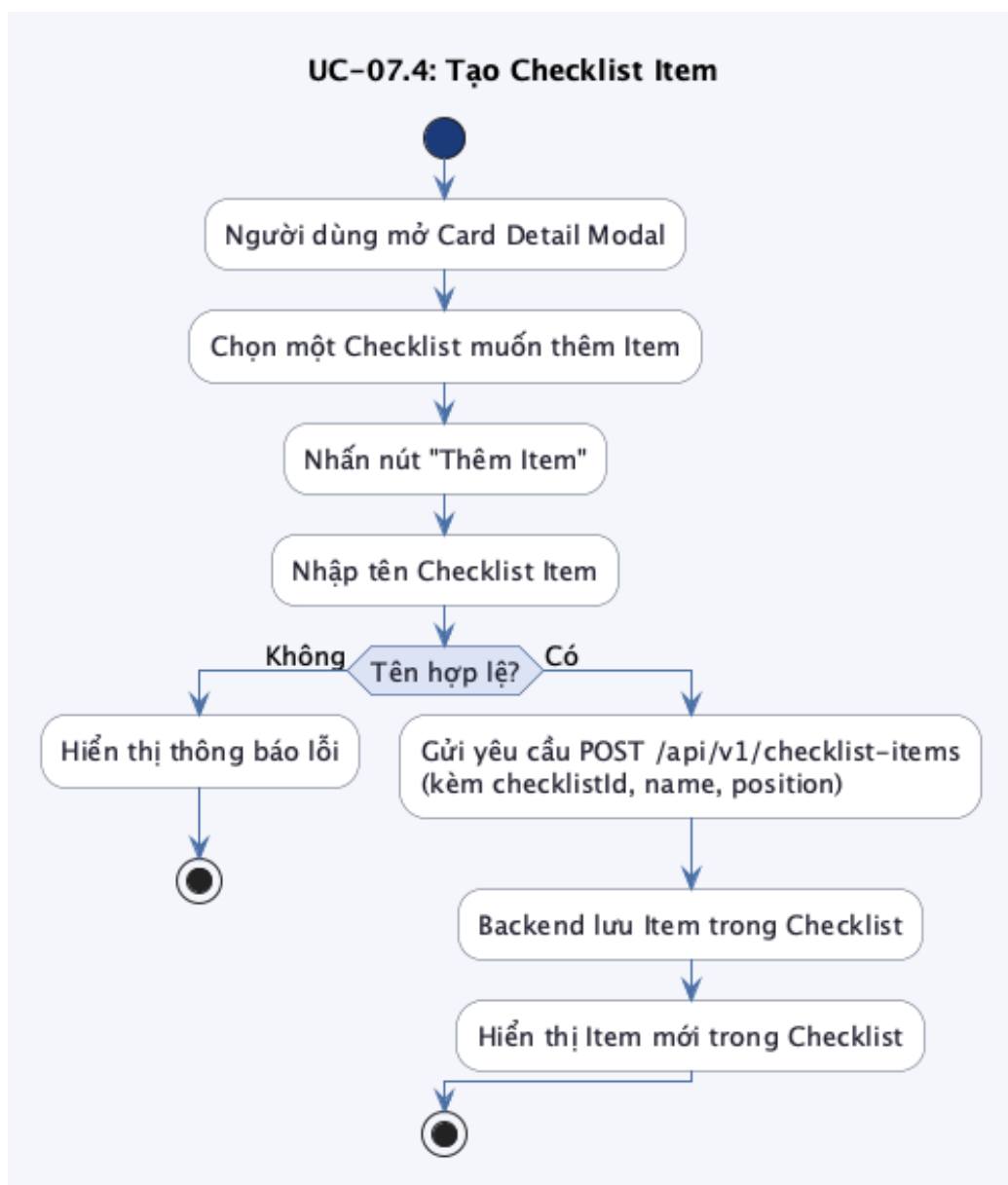
Sau khi xác nhận, hệ thống xóa Checklist và toàn bộ ChecklistItem bên trong theo cơ chế cascade.



Hình 44: Biểu đồ hoạt động UC-07.3: Xóa Checklist

UC-07.4: Tạo Checklist Item

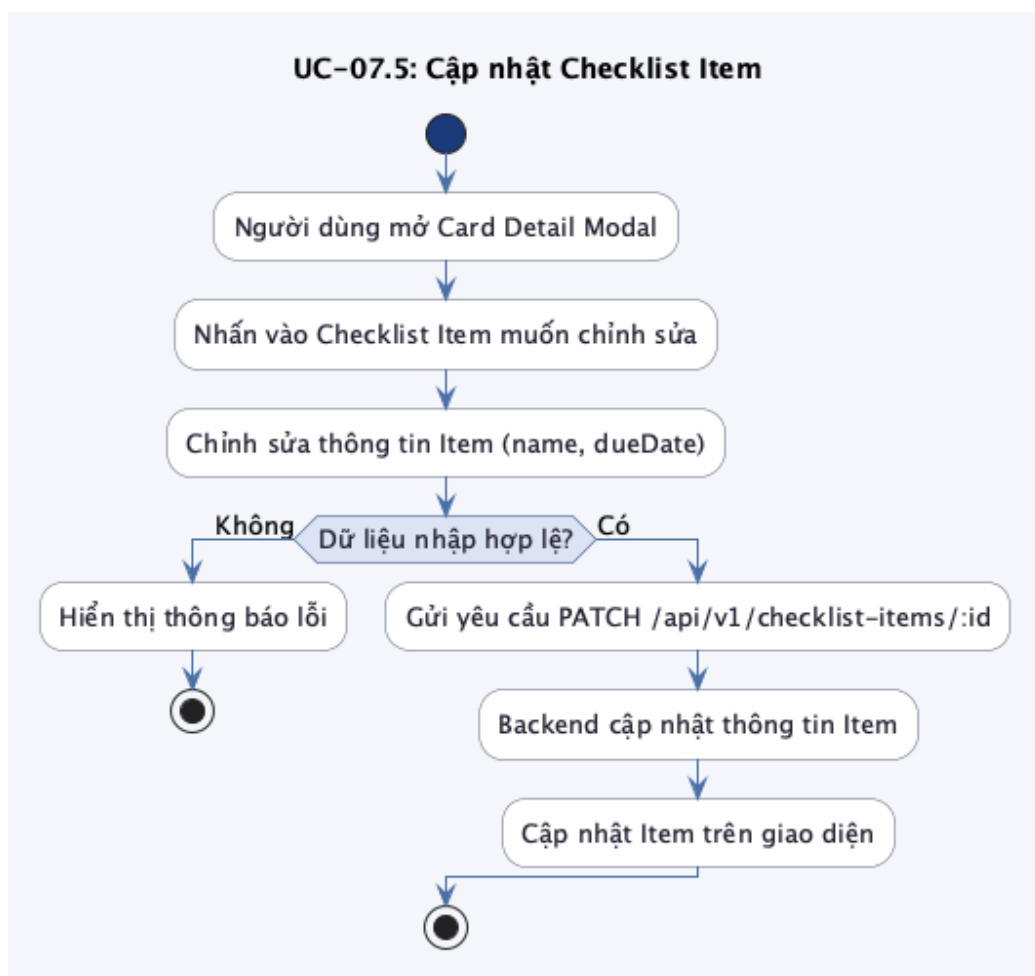
Người dùng nhập tên Item trong một Checklist; hệ thống lưu Item với vị trí cuối cùng và hiển thị ngay trong danh sách.



Hình 45: Biểu đồ hoạt động UC-07.4: Tạo Checklist Item

UC-07.5: Cập nhật Checklist Item

Người dùng chỉnh sửa tên hoặc ngày đến hạn của Item; hệ thống xác thực và lưu thay đổi lên cơ sở dữ liệu.



Hình 46: Biểu đồ hoạt động UC-07.5: Cập nhật Checklist Item

UC-07.6: Đánh dấu Checklist Item hoàn thành

Người dùng tick checkbox của Item; hệ thống cập nhật trạng thái `isCompleted` và làm mới thanh tiến độ của Checklist tương ứng.



Hình 47: Biểu đồ hoạt động UC-07.6: Đánh dấu Checklist Item hoàn thành

UC-07.7: Xóa Checklist Item

Sau khi xác nhận, hệ thống xóa Item và cập nhật thanh tiến độ của Checklist phản ánh số lượng Item còn lại.



Hình 48: Biểu đồ hoạt động UC-07.7: Xóa Checklist Item

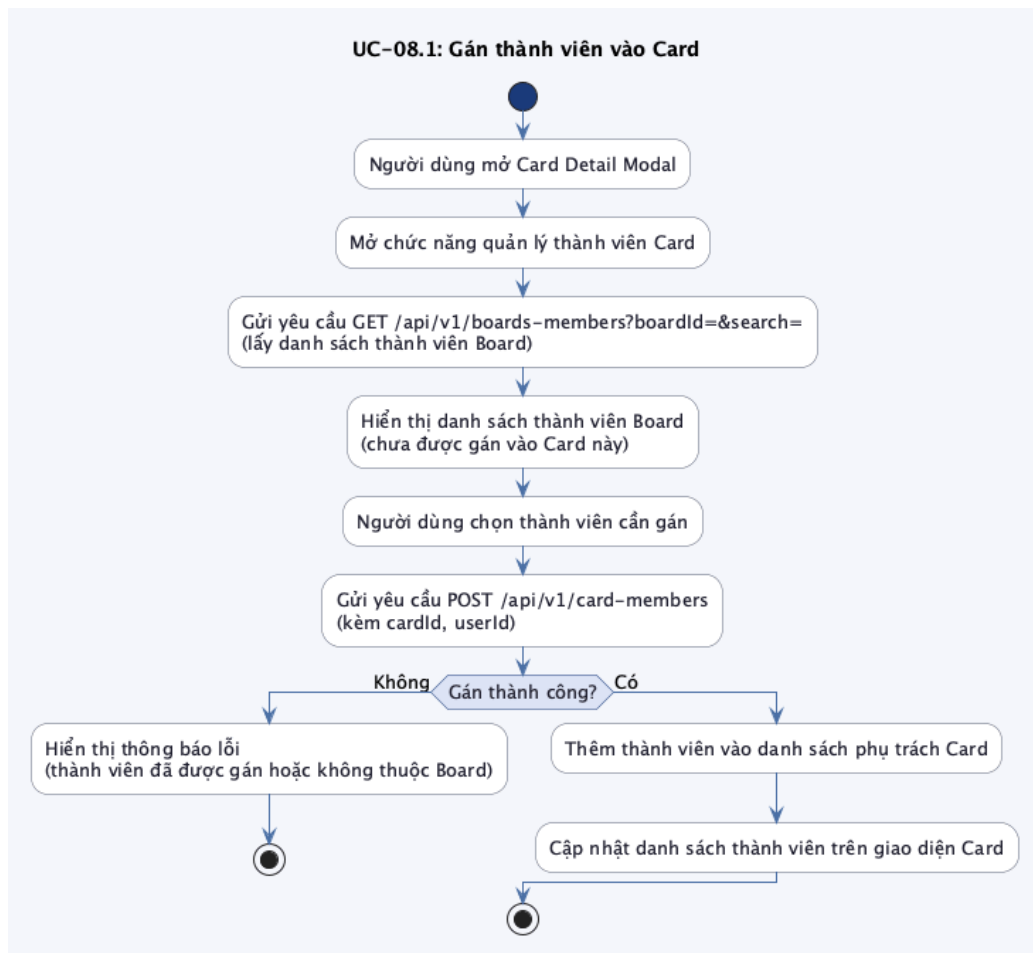
3.1.4.8 UC-08: Quản lý thành viên Card

Nhóm chức năng quản lý thành viên Card gồm hai luồng: gán và hủy gán thành viên phụ trách. Hệ thống chỉ cho phép gán thành viên đã có trong

Board, ngăn trùng lặp.

UC-08.1: Gán thành viên vào Card

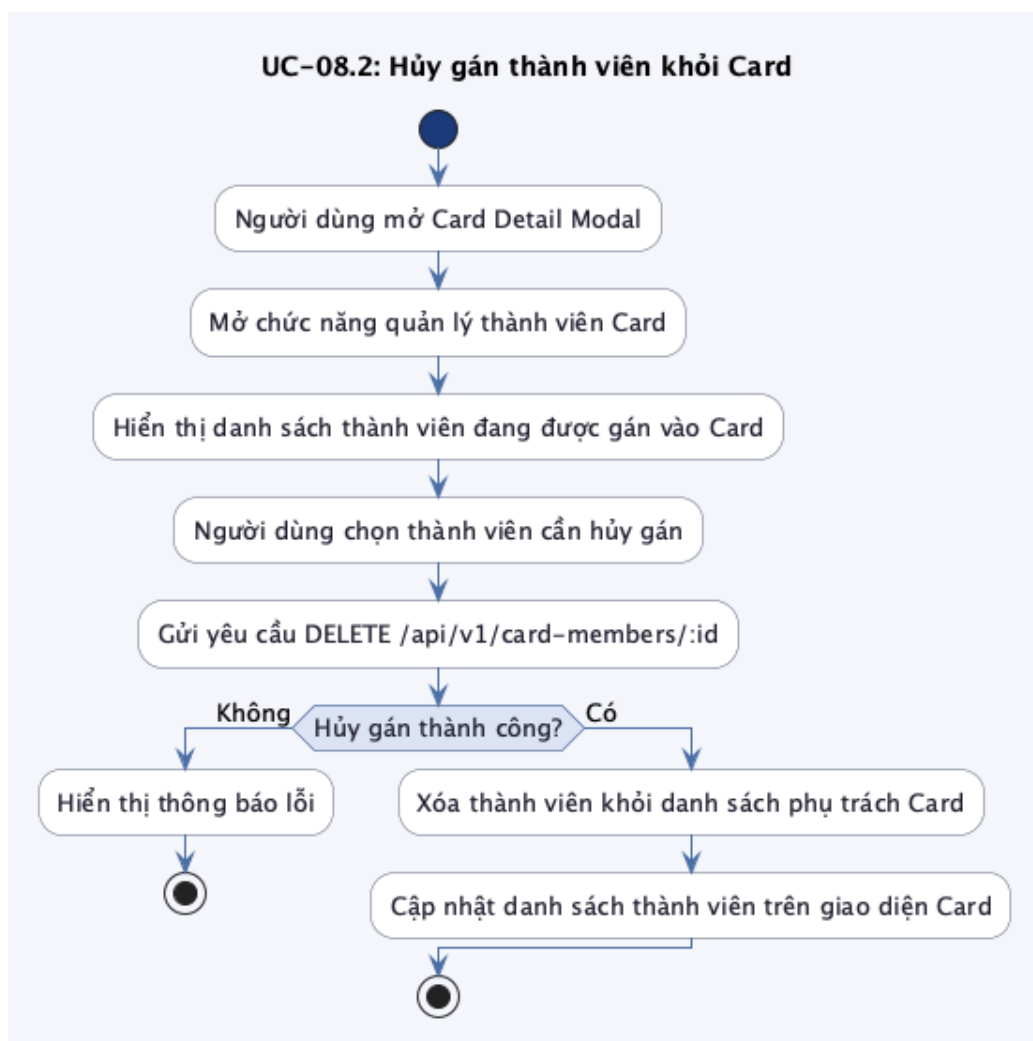
Hệ thống tải danh sách thành viên Board chưa được gán vào Card; người dùng chọn thành viên và hệ thống thêm vào danh sách phụ trách, cập nhật giao diện Card ngay lập tức.



Hình 49: Biểu đồ hoạt động UC-08.1: Gán thành viên vào Card

UC-08.2: Hủy gán thành viên khỏi Card

Người dùng chọn thành viên cần gỡ bỏ khỏi Card; hệ thống xóa quan hệ gán và cập nhật danh sách phụ trách trên giao diện.



Hình 50: Biểu đồ hoạt động UC-08.2: Hủy gán thành viên khỏi Card

3.1.5 Phân tích dữ liệu

3.1.5.1 Các thực thể chính

Hệ thống xoay quanh sáu thực thể nghiệp vụ chính được tổ chức theo cấu trúc phân cấp:

- **User (Người dùng):** Đại diện cho tài khoản người dùng trong hệ thống. Thông tin danh tính và xác thực được quản lý bởi Amazon Cognito; hệ thống nội bộ lưu trữ thông tin bổ sung như họ tên, ảnh đại diện và mã định danh Cognito.

- **Workspace (Không gian làm việc):** Đơn vị tổ chức cấp cao nhất, quản lý tập trung nhiều Board liên quan. Mỗi Workspace có một người sở hữu (owner) và nhiều thành viên.
- **Board (Bảng công việc):** Đại diện cho một dự án cụ thể trong Workspace. Mỗi Board chứa nhiều List và có danh sách thành viên riêng.
- **List (Danh sách):** Cột trạng thái trong Board, thể hiện giai đoạn thực hiện của công việc. Thứ tự các List được quản lý qua trường `position`.
- **Card (Thẻ công việc):** Đơn vị công việc cơ bản, chứa thông tin chi tiết về nhiệm vụ cần thực hiện. Card thuộc về một List và có thứ tự xác định bằng `position`.
- **Checklist:** Danh sách kiểm tra trong Card, hỗ trợ theo dõi các công việc con. Mỗi Checklist có nhiều ChecklistItem.
- **ChecklistItem:** Hạng mục cụ thể trong Checklist, có thể được đánh dấu hoàn thành hoặc chưa hoàn thành, và có thể thiết lập thời hạn riêng.

3.1.5.2 Quan hệ giữa các thực thể

- Một **User** có thể sở hữu nhiều **Workspace** và là thành viên của nhiều **Workspace** và **Board** khác.
- Một **Workspace** chứa nhiều **Board** và có nhiều **User** làm thành viên (quan hệ nhiều-nhiều qua bảng `workspace_member`).
- Một **Board** chứa nhiều **List** và có nhiều **User** làm thành viên với các vai trò khác nhau (quan hệ nhiều-nhiều qua bảng `board_member`).
- Một **List** chứa nhiều **Card**; các Card được sắp xếp theo thứ tự `position`.
- Một **Card** có thể chứa nhiều **Checklist** và được gán cho nhiều **User** (quan hệ nhiều-nhiều qua bảng `card_member`).
- Một **Checklist** chứa nhiều **ChecklistItem**.

3.2 Thiết kế hệ thống

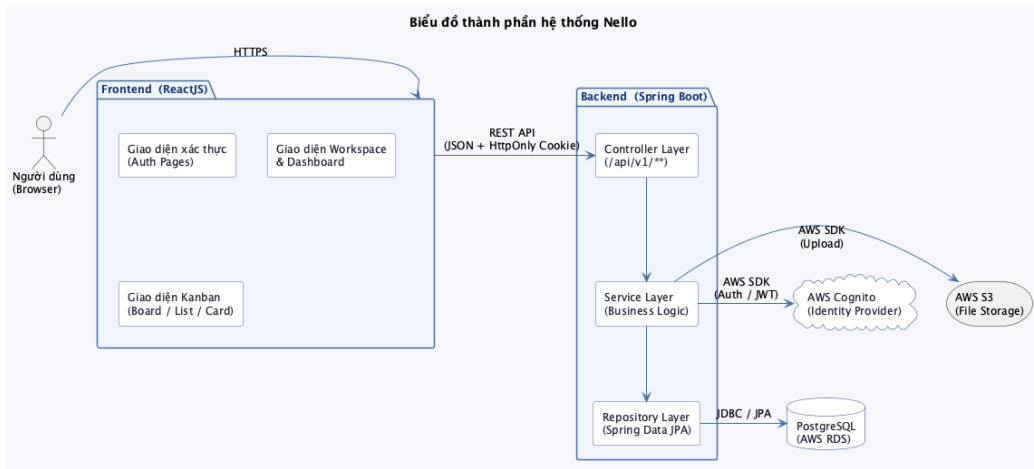
3.2.1 Thiết kế kiến trúc tổng thể

Hệ thống Nello áp dụng kiến trúc **Client-Server phân tách hoàn toàn** (decoupled Frontend/Backend). Frontend và Backend là hai ứng dụng độc lập, giao tiếp với nhau thông qua RESTful API. Toàn bộ logic nghiệp vụ được xử lý tập trung tại Backend; Frontend chịu trách nhiệm trình bày giao diện và tương tác người dùng.

3.2.1.1 Phân tầng hệ thống

Hệ thống được tổ chức thành ba tầng chính:

- **Frontend (ReactJS):** Ứng dụng chạy trên trình duyệt, bao gồm ba nhóm giao diện chính: xác thực (Auth Pages), quản lý Workspace và Dashboard, và giao diện Kanban (Board/List/Card). Frontend giao tiếp với Backend qua REST API sử dụng JSON, token được lưu trong HttpOnly Cookie.
- **Backend (Spring Boot):** Cung cấp toàn bộ RESTful API tại base path `/api/v1`. Kiến trúc nội bộ Backend tuân theo mô hình phân lớp gồm ba tầng:
 - **Controller Layer:** Tiếp nhận HTTP request, kiểm tra JWT từ cookie, ủy quyền xử lý cho Service Layer và trả về HTTP response chuẩn hóa.
 - **Service Layer:** Chứa toàn bộ logic nghiệp vụ, điều phối giữa Repository và các dịch vụ ngoài (Cognito, S3).
 - **Repository Layer:** Truy xuất và lưu trữ dữ liệu thông qua Spring Data JPA, giao tiếp trực tiếp với PostgreSQL qua JDBC.
- **Dịch vụ ngoài (AWS):** Gồm Amazon Cognito (xác thực người dùng và cấp JWT), PostgreSQL trên AWS RDS (cơ sở dữ liệu chính), và Amazon S3 (lưu trữ tệp đa phương tiện).



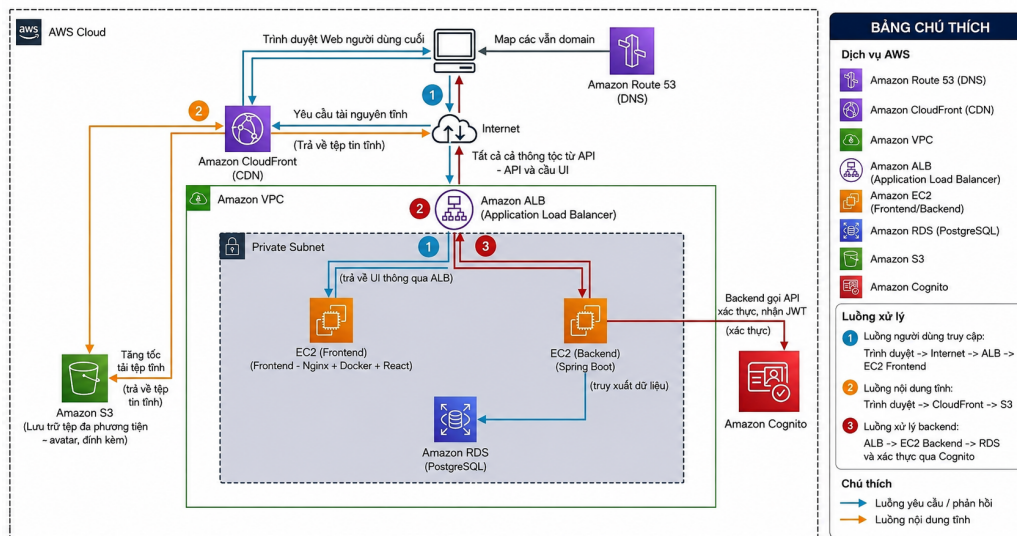
Hình 51: Biểu đồ thành phần hệ thống Nello

3.2.1.2 Kiến trúc triển khai

Hệ thống được triển khai trên nền tảng AWS với toàn bộ máy chủ và cơ sở dữ liệu đặt trong một Virtual Private Cloud (VPC) riêng biệt, cách ly hoàn toàn với Internet. Các thành phần triển khai chính gồm:

- **Amazon ALB (Application Load Balancer):** Điểm vào (entry point) duy nhất nhận lưu lượng từ Internet, phân phối request đến EC2 Frontend hoặc EC2 Backend trong mạng private.
- **EC2 (Frontend):** Ứng dụng ReactJS được đóng gói bằng Docker và phục vụ qua Nginx. Chỉ nhận lưu lượng từ ALB, không tiếp xúc trực tiếp với Internet.
- **EC2 (Backend):** Ứng dụng Spring Boot xử lý API. Chỉ nhận request nội bộ từ ALB và giao tiếp với RDS và Cognito.
- **Amazon RDS (PostgreSQL):** Cơ sở dữ liệu quan hệ trong mạng private sâu nhất, chỉ Backend mới được phép truy cập.
- **Amazon CloudFront + S3:** CloudFront đóng vai trò CDN phân phối tài nguyên tĩnh từ S3 đến người dùng với độ trễ thấp.

Luồng hoạt động chính gồm ba hướng: (1) Tải giao diện: Client → ALB → EC2 Frontend; (2) Tài nguyên tĩnh: Client → CloudFront → S3; (3) Xử lý nghiệp vụ: Client → ALB → EC2 Backend → RDS hoặc Cognito.



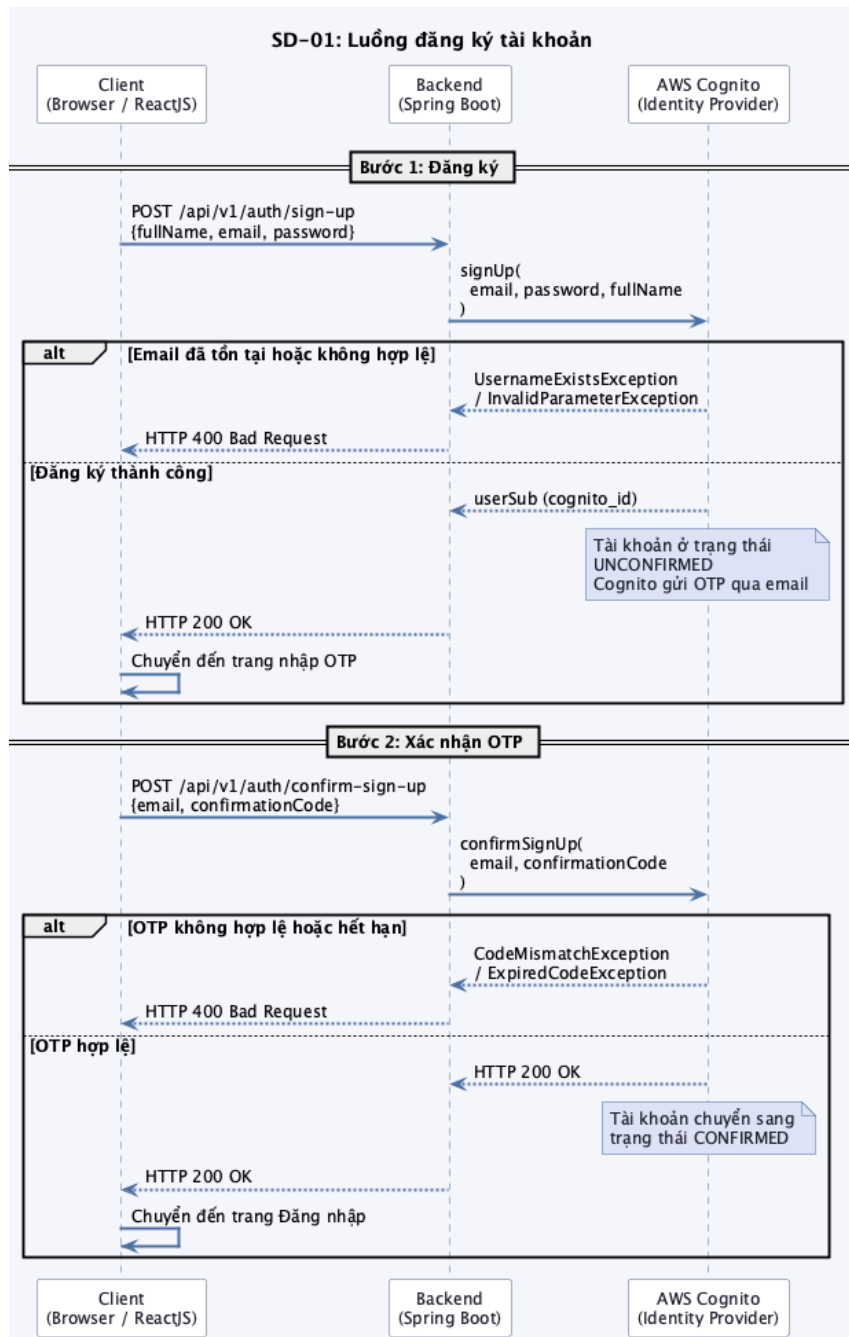
Hình 52: Kiến trúc triển khai hệ thống Nello trên AWS

3.2.2 Thiết kế tương tác giữa các thành phần

Biểu đồ tuần tự (Sequence Diagram) mô tả cách các thành phần trong hệ thống giao tiếp với nhau theo thứ tự thời gian. Phần này trình bày mười luồng tương tác tiêu biểu, bao quát toàn bộ vòng đời sử dụng hệ thống Nello: từ xác thực tài khoản, khởi tạo không gian làm việc, vận hành bằng Kanban, đến các thao tác kéo thả nâng cao với kỹ thuật optimistic update.

3.2.2.1 SD-01: Luồng đăng ký tài khoản

Đăng ký là luồng hai bước: bước đầu Backend gọi Cognito để tạo tài khoản tạm thời và Cognito tự động gửi mã OTP qua email; bước hai người dùng nhập OTP và Backend gọi Cognito xác nhận để kích hoạt tài khoản. Toàn bộ trạng thái tài khoản (UNCONFIRMED / CONFIRMED) được quản lý bởi Cognito, Backend không lưu trữ mật khẩu.

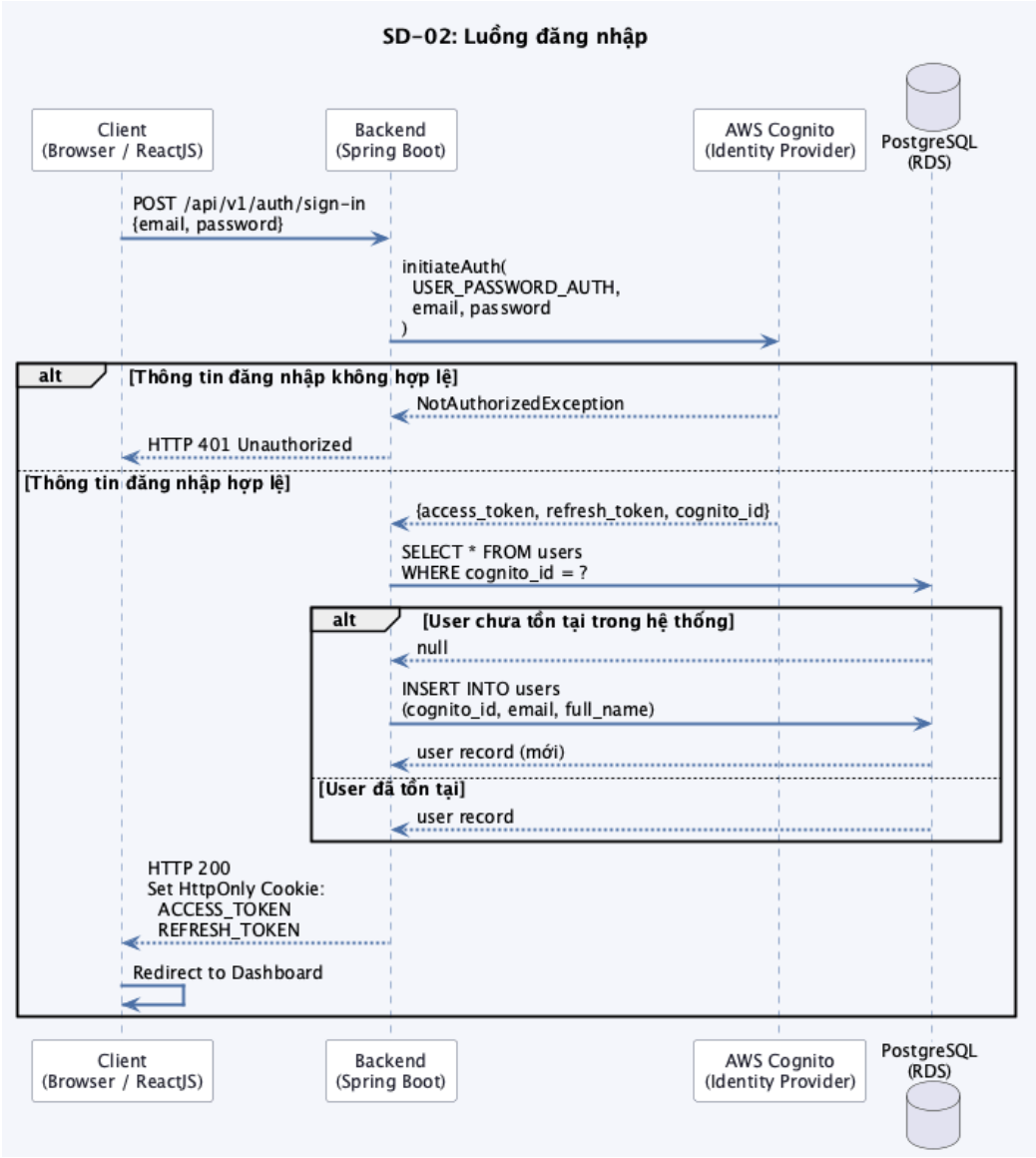


Hình 53: Biểu đồ tuần tự SD-01: Luồng đăng ký tài khoản

3.2.2.2 SD-02: Luồng đăng nhập

Backend đóng vai trò trung gian — thay vì Client gọi trực tiếp Cognito,

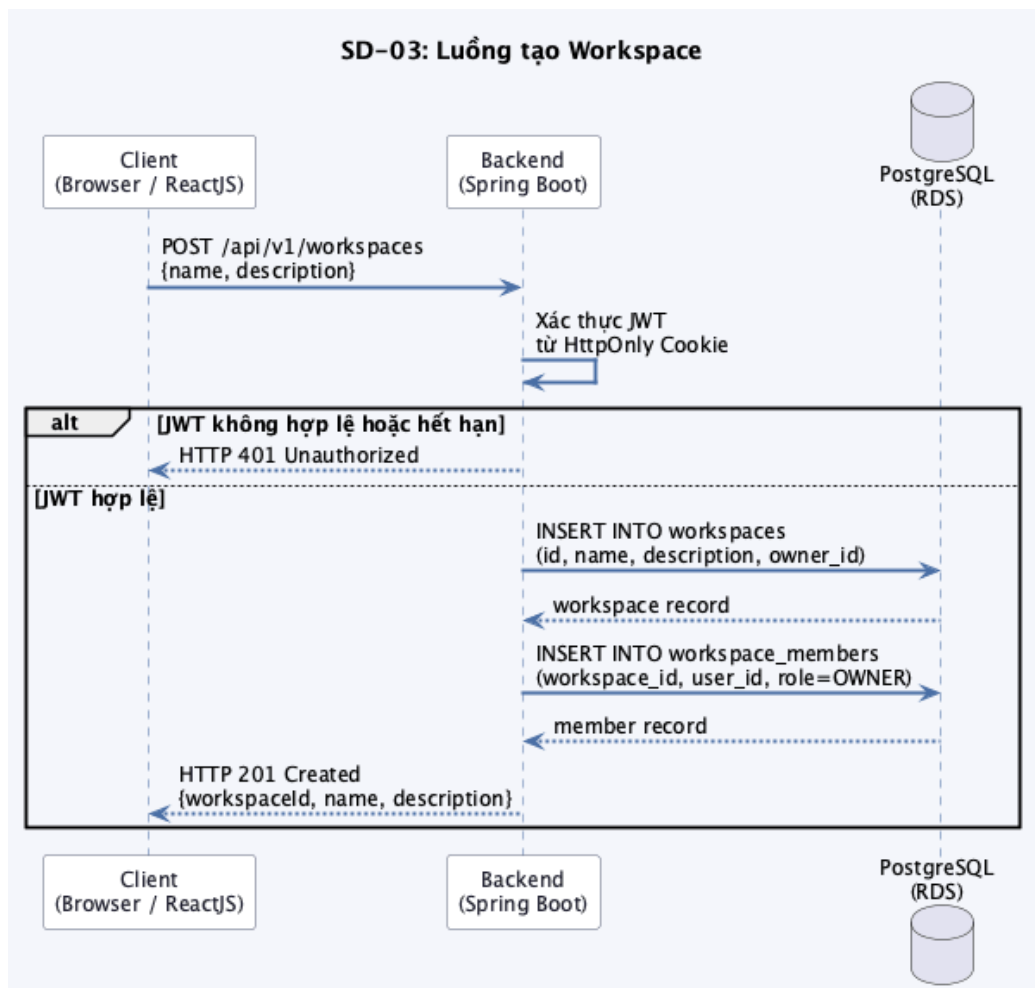
toàn bộ quá trình xác thực được Backend điều phối qua AWS SDK. Sau khi Cognito xác thực thành công, Backend tìm kiếm user trong PostgreSQL theo `cognito_id`; nếu chưa tồn tại (lần đăng nhập đầu tiên), Backend tự động tạo mới bản ghi user. JWT nhận được được lưu vào HttpOnly Cookie để bảo vệ khỏi tấn công XSS.



Hình 54: Biểu đồ tuần tự SD-02: Luồng đăng nhập

3.2.2.3 SD-03: Luồng tạo Workspace

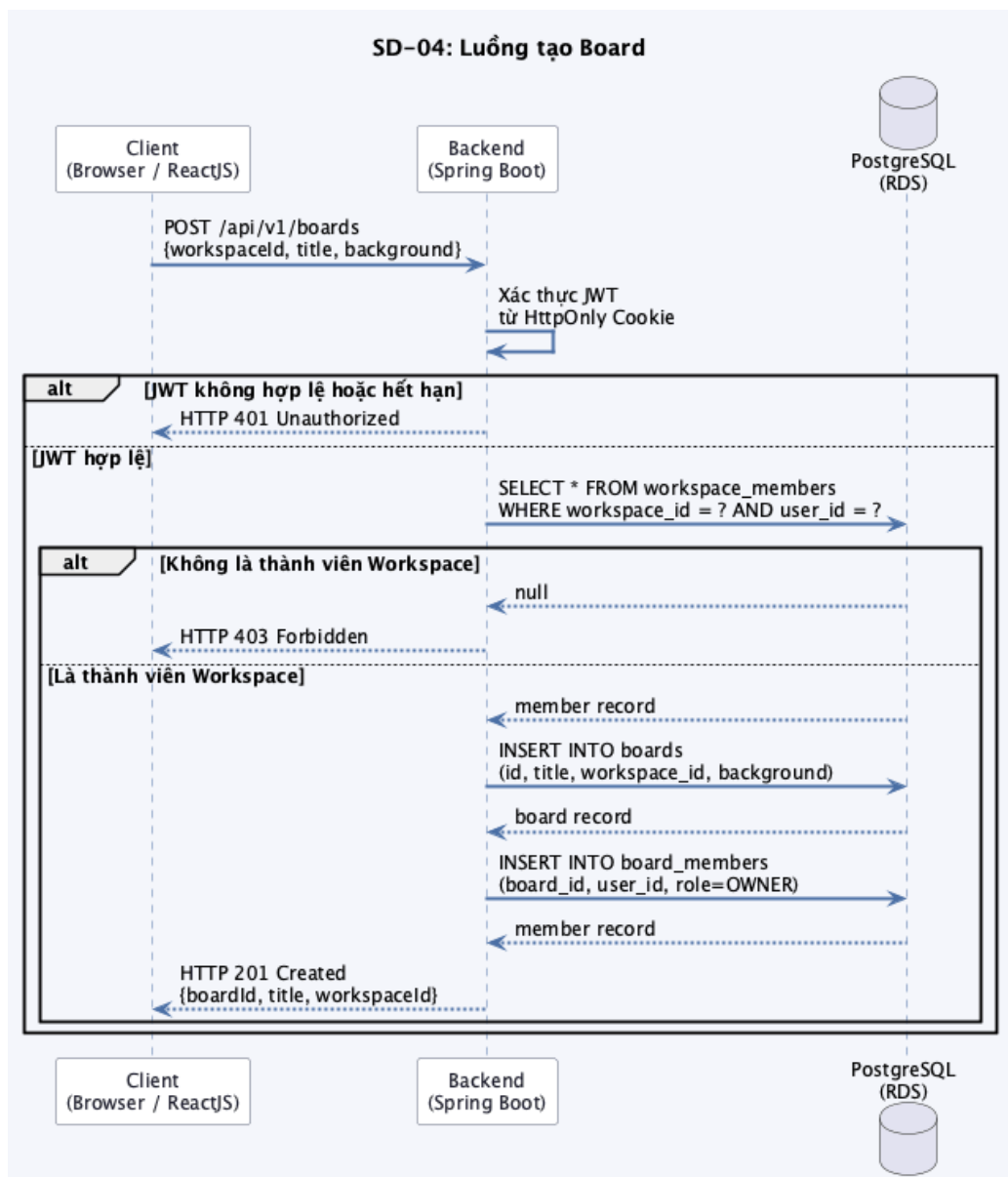
Workspace là đơn vị tổ chức cấp cao nhất trong hệ thống. Sau khi Backend xác thực JWT, hệ thống thực hiện hai thao tác ghi tuần tự: tạo bản ghi Workspace và tự động ghi nhận người tạo là thành viên với vai trò OWNER trong bảng `workspace_members`.



Hình 55: Biểu đồ tuần tự SD-03: Luồng tạo Workspace

3.2.2.4 SD-04: Luồng tạo Board

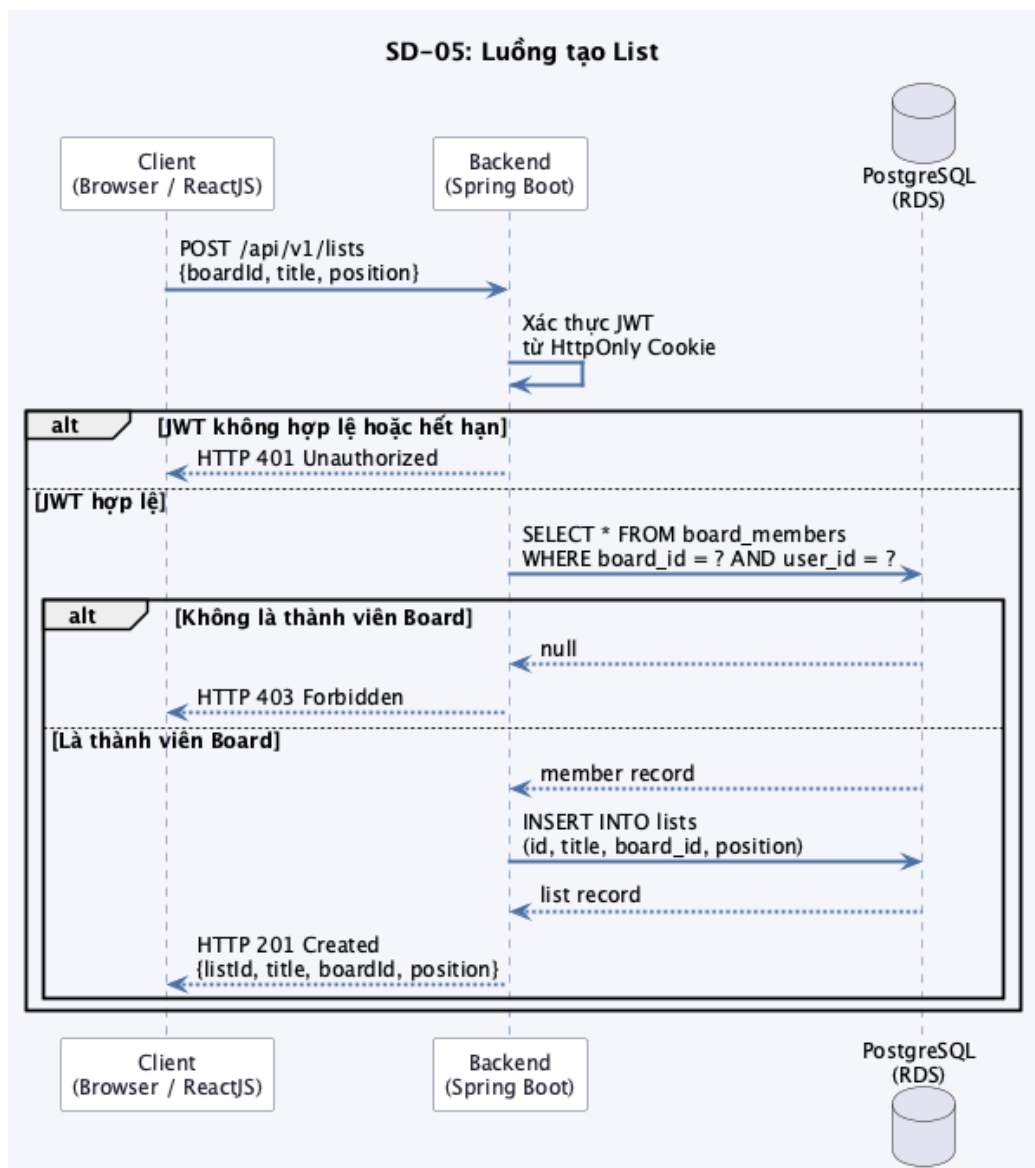
Board được tạo trong phạm vi một Workspace. Backend kiểm tra người dùng có phải thành viên của Workspace trước khi cho phép tạo Board. Sau khi Board được tạo, người tạo được ghi nhận tự động là thành viên Board với vai trò OWNER.



Hình 56: Biểu đồ tuần tự SD-04: Luồng tạo Board

3.2.2.5 SD-05: Luồng tạo List

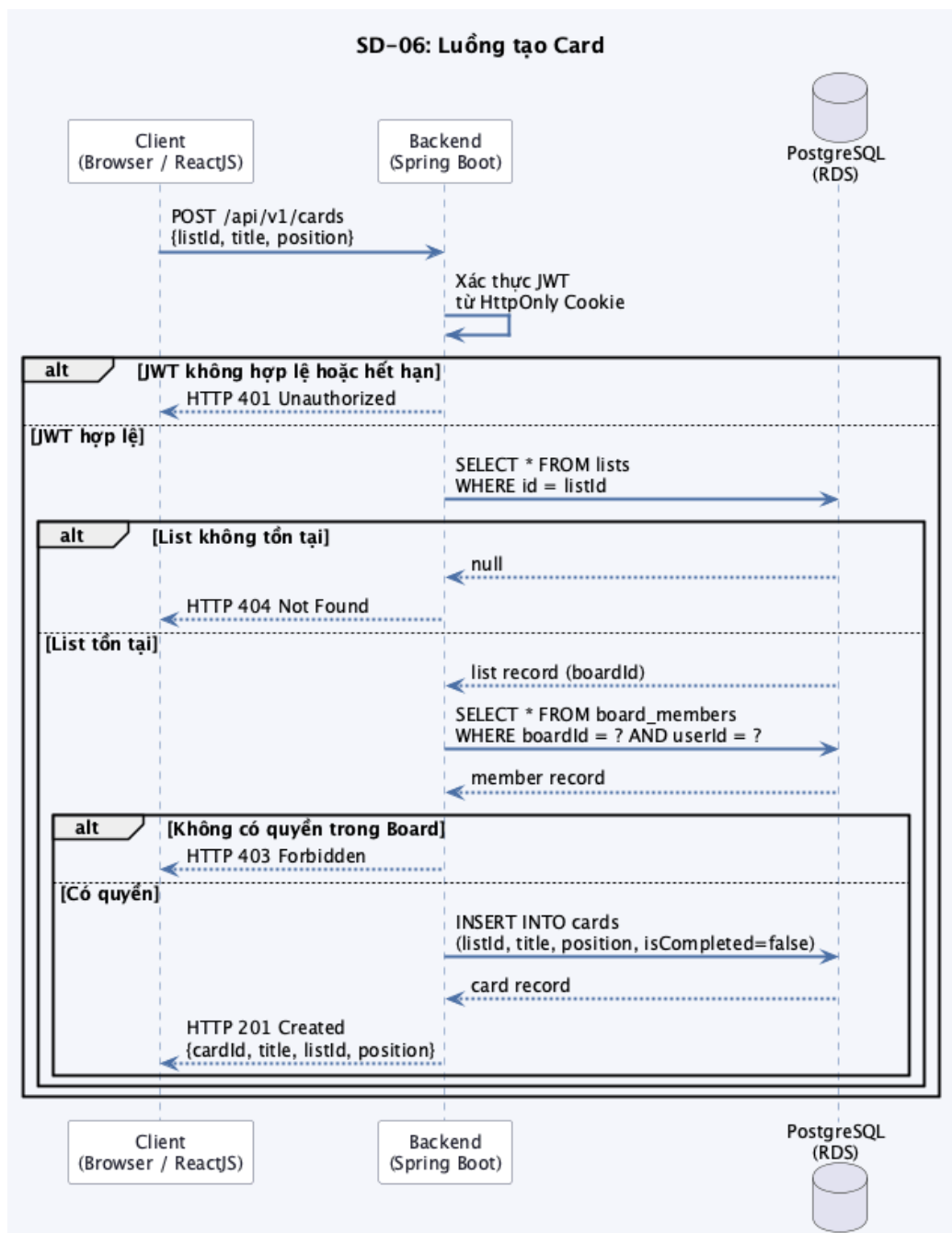
List là các cột trong giao diện Kanban. Backend kiểm tra quyền thành viên Board trước khi thực hiện INSERT. Trường `position` sử dụng giá trị chuỗi phân số (fractional indexing) để hỗ trợ sắp xếp linh hoạt mà không cần cập nhật lại toàn bộ danh sách.



Hình 57: Biểu đồ tuần tự SD-05: Luồng tạo List

3.2.2.6 SD-06: Luồng tạo Card

Card là đơn vị công việc cơ bản trong hệ thống. Backend thực hiện hai bước kiểm tra tuần tự trước khi ghi: xác minh List đích tồn tại và xác minh người dùng có quyền trong Board tương ứng. Cơ chế phân quyền này đảm bảo mọi thao tác tạo dữ liệu đều được kiểm soát chặt chẽ.

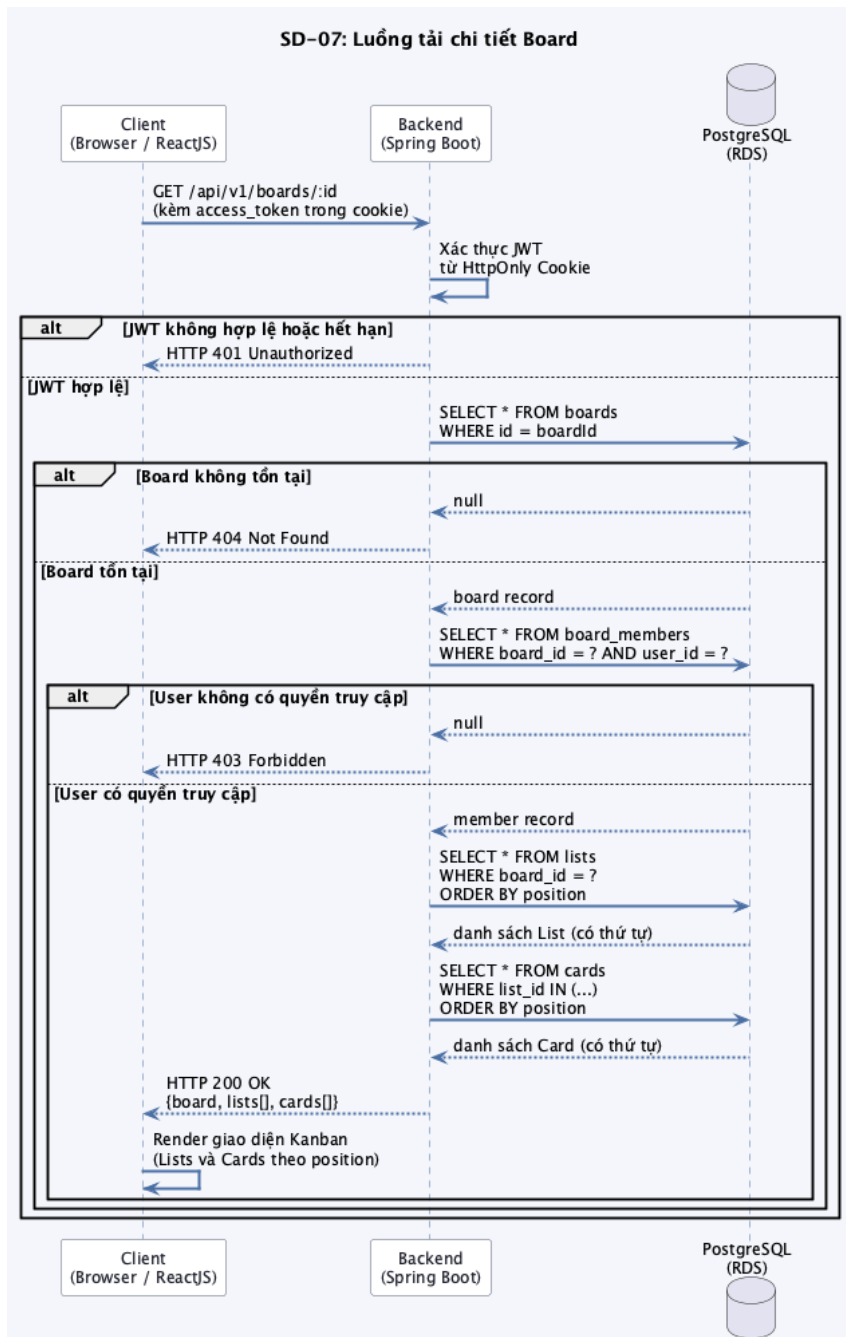


Hình 58: Biểu đồ tuần tự SD-06: Luồng tạo Card

3.2.2.7 SD-07: Luồng tải chi tiết Board

Đây là luồng đọc dữ liệu lồng nhau phức tạp nhất trong hệ thống. Backend

thực hiện tuần tự: kiểm tra Board tồn tại, kiểm tra quyền truy cập của người dùng, sau đó truy vấn danh sách List và Card theo thứ tự **position**. Toàn bộ dữ liệu được trả về trong một response để Frontend có thể render ngay giao diện Kanban.

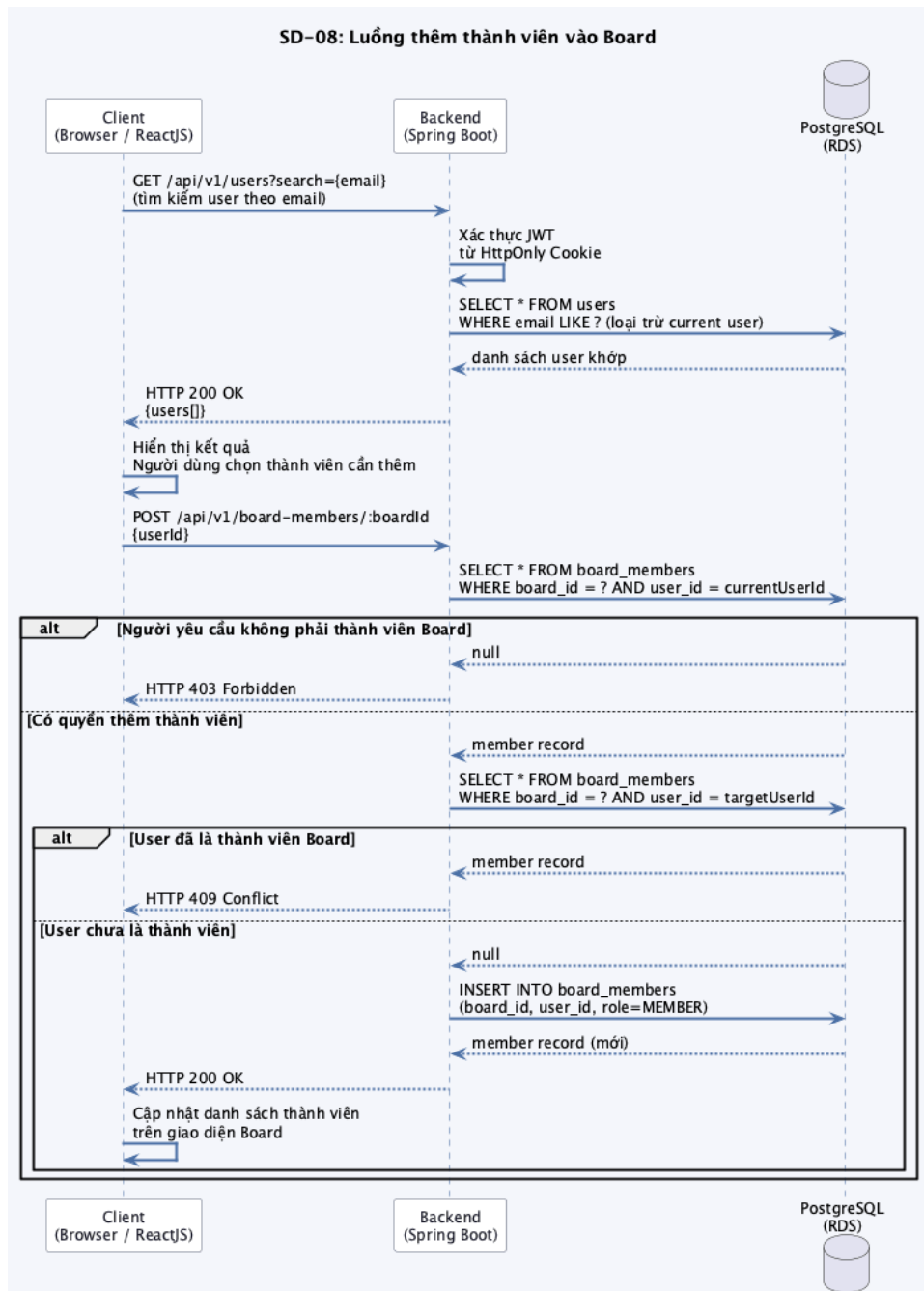


Hình 59: Biểu đồ tuần tự SD-07: Luồng tải chi tiết Board

3.2.2.8 SD-08: Luồng thêm thành viên vào Board

Luồng gồm hai giai đoạn: tìm kiếm user theo email và thêm vào Board.

Backend xác minh người yêu cầu có quyền trong Board, kiểm tra user đích tồn tại và chưa là thành viên trước khi thực hiện INSERT vào bảng `board_members`; nếu đã là thành viên, hệ thống trả về lỗi `409 Conflict` để tránh trùng lặp.

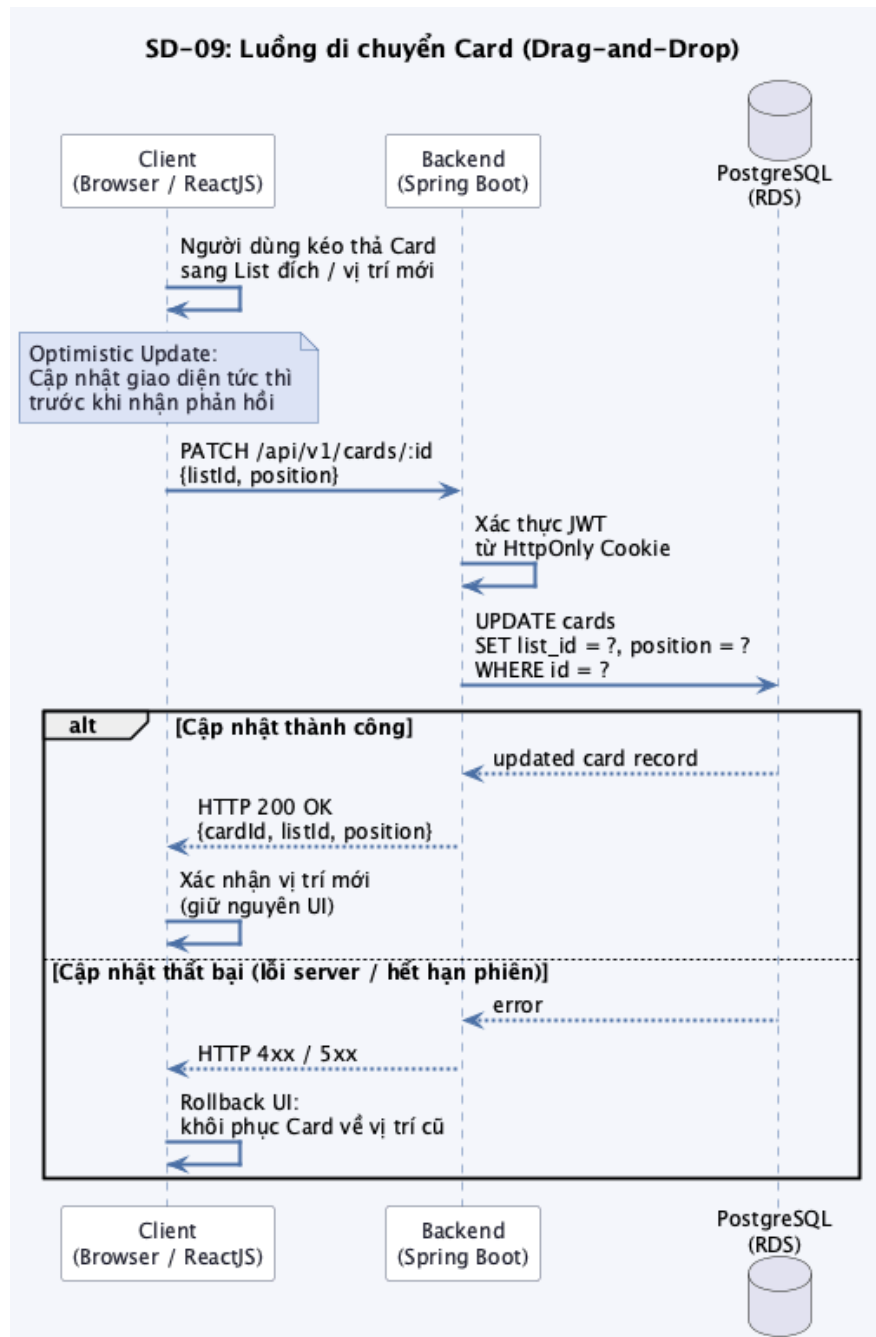


Hình 60: Biểu đồ tuần tự SD-08: Luồng thêm thành viên vào Board

3.2.2.9 SD-09: Luồng di chuyển Card (Drag-and-Drop)

Luồng này minh họa kỹ thuật **optimistic update** — giao diện được cập

nhập tức thì ngay khi người dùng kéo thả Card, trước khi nhận phản hồi từ Backend, nhằm tối ưu trải nghiệm người dùng. Nếu yêu cầu cập nhật thất bại (lỗi mạng, hết hạn phiên...), Frontend tự động rollback giao diện về trạng thái trước đó.

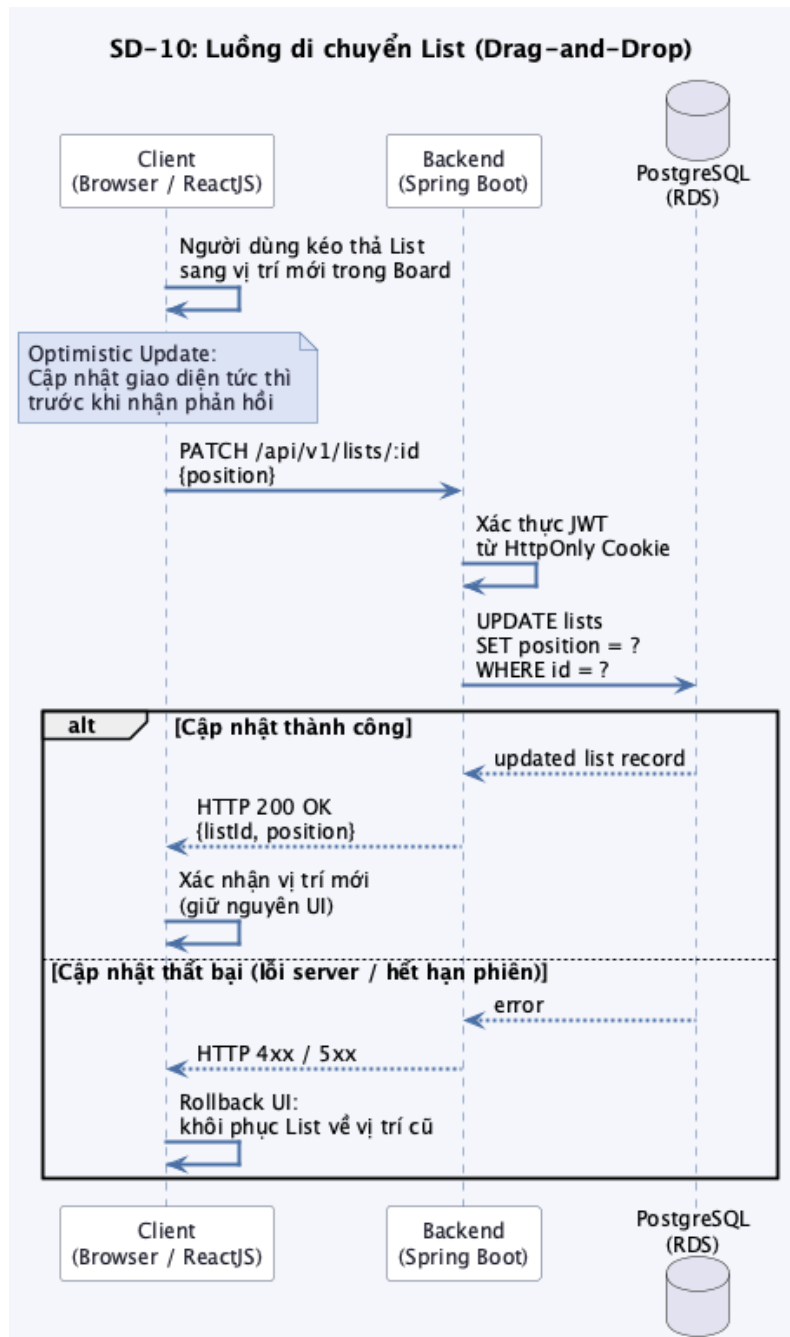


Hình 61: Biểu đồ tuần tự SD-09: Luồng di chuyển Card (Drag-and-Drop)

3.2.2.10 SD-10: Luồng di chuyển List (Drag-and-Drop)

Tương tự SD-09, luồng di chuyển List cũng áp dụng kỹ thuật **optimistic**

update. Khi người dùng kéo thả một List sang vị trí khác trong Board, Frontend cập nhật giao diện ngay lập tức trong khi Backend cập nhật trường `position` vào cơ sở dữ liệu. Nếu thao tác thất bại, UI được rollback về trạng thái ban đầu.

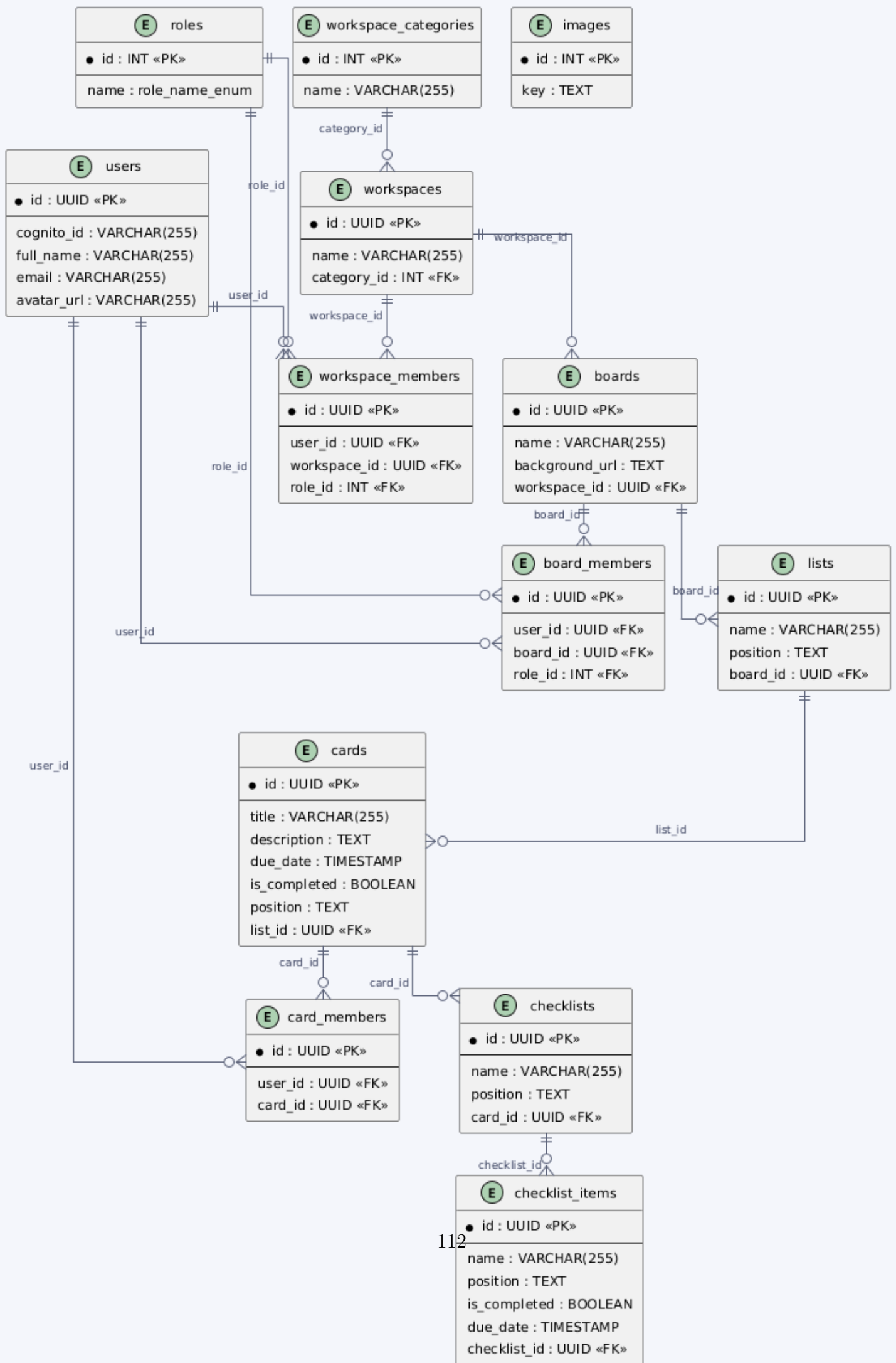


Hình 62: Biểu đồ tuần tự SD-10: Luồng di chuyển List (Drag-and-Drop)

3.2.3 Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống Nello sử dụng PostgreSQL, bao gồm 13 bảng được tổ chức thành ba nhóm: bảng tra cứu (lookup tables), bảng tham chiếu (reference tables) và bảng nghiệp vụ (business tables). Khóa chính của các bảng nghiệp vụ sử dụng kiểu UUID để đảm bảo tính duy nhất toàn cục; các bảng tra cứu sử dụng INT. Cơ chế xóa theo tầng (cascade delete) được áp dụng nhất quán theo chuỗi: Workspace → Board → List → Card → Checklist → ChecklistItem.

3.2.3.1 Sơ đồ quan hệ



Sơ đồ quan hệ thể hiện toàn bộ cấu trúc dữ liệu của hệ thống với 13 bảng được tổ chức thành ba nhóm chính: bảng tra cứu dùng chung, chuỗi phân cấp nghiệp vụ và bảng trung gian quản lý thành viên.

Nhóm 1: Bảng tra cứu dùng chung

- **roles – workspace_members / board_members (1–N):** Bảng **roles** lưu ba vai trò **ADMIN**, **MEMBER**, **VIEWER**. Cả hai bảng **workspace_members** và **board_members** đều tham chiếu đến **roles** qua khóa ngoại **role_id**, đảm bảo tính nhất quán của hệ thống phân quyền trong toàn hệ thống.
- **workspace_categories – workspaces (1–N):** Mỗi **Workspace** bắt buộc phải thuộc một loại được định nghĩa sẵn trong **workspace_categories** thông qua khóa ngoại **category_id**. Một loại **Workspace** có thể được gán cho nhiều **Workspace** khác nhau.
- **images – boards:** Bảng **images** cung cấp danh sách 8 ảnh nền cố định. Board lưu đường dẫn ảnh được chọn vào trường **background_url**.

Nhóm 2: Chuỗi phân cấp nghiệp vụ

Hệ thống tổ chức dữ liệu theo chuỗi phân cấp từ trên xuống dưới: **users** → **workspaces** → **boards** → **lists** → **cards** → **checklists** → **checklist_items**. Mỗi cấp đều áp dụng ràng buộc **ON DELETE CASCADE**, nghĩa là khi một bản ghi ở cấp trên bị xóa, toàn bộ dữ liệu ở các cấp dưới sẽ bị xóa theo.

- **users – workspaces (1–N):** Mỗi **Workspace** được tạo bởi một người dùng xác định qua **created_by**. Trường **updated_by** ghi lại người thực hiện cập nhật gần nhất, cả hai đều là khóa ngoại tham chiếu **users.id**.
- **workspaces – boards (1–N):** Mỗi **Board** thuộc về đúng một **Workspace** qua khóa ngoại **workspace_id**. Khi **Workspace** bị xóa, toàn bộ **Board** bên trong bị xóa theo (**CASCADE**).
- **boards – lists (1–N):** Mỗi **List** thuộc về đúng một **Board** qua khóa ngoại **board_id**. Thứ tự hiển thị của các **List** được quản lý qua trường **position** theo thuật toán fractional indexing.
- **lists – cards (1–N):** Mỗi **Card** thuộc về đúng một **List** qua khóa ngoại **list_id**. **Card** có thể được di chuyển giữa các **List** trong cùng **Board** bằng cách cập nhật **list_id**.

- **cards – checklists (1–N):** Mỗi Checklist gắn với một Card duy nhất qua khóa ngoại `card_id`. Một Card có thể chứa nhiều Checklist, thứ tự hiển thị được quản lý qua `position`.
- **checklists – checklist_items (1–N):** Mỗi ChecklistItem thuộc về đúng một Checklist qua khóa ngoại `checklist_id`. Tỷ lệ hoàn thành của Checklist được tính dựa trên số lượng item có `is_completed = true`.

Nhóm 3: Quan hệ nhiều–nhiều quản lý thành viên

- **users – workspaces qua workspace_members (N–N):** Một người dùng có thể là thành viên của nhiều Workspace và một Workspace có thể có nhiều thành viên. Bảng trung gian `workspace_members` lưu cặp (`user_id`, `workspace_id`) cùng với `role_id` xác định quyền hạn. Ràng buộc UNIQUE trên cặp này đảm bảo mỗi người dùng chỉ có một vai trò duy nhất trong mỗi Workspace.
- **users – boards qua board_members (N–N):** Bảng `board_members` quản lý danh sách thành viên trong từng Board kèm vai trò tương ứng, độc lập với danh sách thành viên Workspace. Một người dùng bất kỳ có thể được thêm vào Board mà không cần phải là thành viên của Workspace chứa Board đó. Ràng buộc UNIQUE trên cặp (`user_id`, `board_id`) ngăn chặn trùng lặp.
- **users – cards qua card_members (N–N):** Bảng `card_members` ghi nhận các thành viên được gán làm người phụ trách cho Card. Không có trường `role_id` vì quan hệ này chỉ mang ý nghĩa gán trách nhiệm, không phân quyền. Ràng buộc UNIQUE trên cặp (`user_id`, `card_id`) đảm bảo mỗi thành viên chỉ được gán một lần.

3.2.3.2 Đặc tả bảng

3.2.3.3 Bảng roles

a. Mô tả

Bảng `roles` lưu trữ các vai trò của thành viên trong hệ thống. Đây là bảng tra cứu dùng chung cho cả Workspace và Board.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	Khóa chính	Mã vai trò.
name	role_name_enum	NOT NULL, UNIQUE	Tên vai trò.

Bảng 9: Bảng roles

c. Ghi chú

Trường name thuộc kiểu liệt kê role_name_enum nhận một trong ba giá trị: ADMIN, MEMBER, VIEWER. Dữ liệu được nạp sẵn khi khởi tạo hệ thống.

3.2.3.4 Bảng images

a. Mô tả

Bảng images cung cấp danh sách 8 ảnh nền cố định cho Board. Người dùng chọn một trong các ảnh này khi tạo hoặc cập nhật Board.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	Khóa chính	Mã ảnh nền.
key	TEXT	NOT NULL	Đường dẫn tương đối đến tệp ảnh nền.

Bảng 10: Bảng images

3.2.3.5 Bảng workspace_categories

a. Mô tả

Bảng workspace_categories lưu danh sách các loại Workspace để phân loại không gian làm việc khi tạo mới.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	INT	Khóa chính	Mã loại Workspace.
name	VARCHAR(255)	NOT NULL	Tên loại Workspace.

Bảng 11: Bảng workspace_categories

c. Ghi chú

Hệ thống cung cấp sẵn 8 loại: Sales CRM, Education, Human Resources, Operation, Marketing, IT, Small Business, Other.

3.2.3.6 Bảng users

a. Mô tả

Bảng **users** lưu thông tin người dùng trong hệ thống. Thông tin xác thực được quản lý bởi Amazon Cognito; trường **cognito_id** là liên kết giữa hệ thống nội bộ và Cognito User Pool.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh nội bộ.
cognito_id	VARCHAR(255)	NOT NULL, UNIQUE	Mã định danh từ Amazon Cognito.
full_name	VARCHAR(255)	NOT NULL	Họ tên đầy đủ.
email	VARCHAR(255)	NOT NULL, UNIQUE	Địa chỉ email.
avatar_url	VARCHAR(255)	NULL	Đường dẫn ảnh đại diện.
bio	TEXT	NULL	Giới thiệu bản thân.
phone	VARCHAR(255)	NULL	Số điện thoại.
address	VARCHAR(255)	NULL	Địa chỉ.
deleted_at	TIMESTAMP	NULL	Thời điểm xóa mềm tài khoản.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 12: Bảng users

3.2.3.7 Bảng workspaces

a. Mô tả

Bảng **workspaces** lưu thông tin các không gian làm việc. Đây là đơn vị tổ chức cấp cao nhất trong hệ thống, chứa nhiều Board bên trong.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên Workspace.
description	VARCHAR(255)	NULL	Mô tả Workspace.
category_id	INT	NOT NULL, FK → workspace_categories	Loại Workspace.
created_by	UUID	NOT NULL, FK → users	Người tạo Workspace.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 13: Bảng workspaces

3.2.3.8 Bảng workspace_members

a. Mô tả

Bảng `workspace_members` là bảng trung gian thể hiện quan hệ nhiều-nhiều giữa người dùng và Workspace. Mỗi bản ghi xác định một thành viên trong một Workspace cùng vai trò tương ứng.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
user_id	UUID	NOT NULL, FK → users	Thành viên được thêm vào Workspace.
workspace_id	UUID	NOT NULL, FK → workspaces	Workspace chứa thành viên.
role_id	INT	NOT NULL, FK → roles	Vai trò trong Workspace.
created_by	UUID	NOT NULL, FK → users	Người thêm thành viên.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm thêm thành viên.

Bảng 14: Bảng `workspace_members`

c. Ghi chú

Ràng buộc `UNIQUE` trên cặp (`user_id`, `workspace_id`) đảm bảo mỗi người dùng chỉ có một vai trò duy nhất trong một Workspace. Khi Workspace hoặc User bị xóa, bản ghi liên quan sẽ bị xóa theo (`ON DELETE CASCADE`).

3.2.3.9 Bảng `boards`

a. Mô tả

Bảng `boards` lưu thông tin các bảng công việc bên trong Workspace. Mỗi Board có thể có ảnh nền riêng được chọn từ bảng `images`.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên Board.
background_url	TEXT	NULL	Đường dẫn ảnh nền Board.
workspace_id	UUID	NOT NULL, FK → workspaces, CASCADE	Workspace chứa Board.
created_by	UUID	NOT NULL, FK → users	Người tạo Board.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 15: Bảng boards

3.2.3.10 Bảng board_members

a. Mô tả

Bảng `board_members` là bảng trung gian thể hiện quan hệ nhiều-nhiều giữa người dùng và Board. Mỗi thành viên trong Board có một vai trò xác định quyền thao tác trong Board đó.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
user_id	UUID	NOT NULL, FK → users	Thành viên được thêm vào Board.
board_id	UUID	NOT NULL, FK → boards, CASCADE	Board chứa thành viên.
role_id	INT	NOT NULL, FK → roles	Vai trò trong Board.
created_by	UUID	NOT NULL, FK → users	Người thêm thành viên.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm thêm thành viên.

Bảng 16: Bảng `board_members`

c. Ghi chú

Ràng buộc UNIQUE trên cặp (`user_id`, `board_id`) đảm bảo mỗi người dùng chỉ có một vai trò duy nhất trong một Board. Danh sách thành viên Board hoàn toàn độc lập với danh sách thành viên Workspace — người dùng có thể được thêm trực tiếp vào Board mà không cần thuộc Workspace chứa Board đó.

3.2.3.11 Bảng `lists`

a. Mô tả

Bảng `lists` lưu các cột trạng thái (List) bên trong Board theo mô hình Kanban. Thứ tự hiển thị của List được quản lý qua trường `position`.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên List.
position	TEXT	NOT NULL	Giá trị vị trí dùng để sắp xếp và kéo thả.
board_id	UUID	NOT NULL, FK → boards, CASCADE	Board chứa List.
created_by	UUID	NOT NULL, FK → users	Người tạo List.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 17: Bảng `lists`

c. Ghi chú

Trường `position` sử dụng kiểu `TEXT` thay vì `INT` để tránh phải đánh lại chỉ số toàn bộ khi chèn phần tử vào giữa. Thuật toán fractional indexing được áp dụng để tính giá trị `position` khi kéo thả.

3.2.3.12 Bảng `cards`

a. Mô tả

Bảng `cards` là bảng trung tâm của hệ thống, lưu toàn bộ thông tin về các thẻ công việc. Mỗi Card thuộc về một List và có thể được gán thành viên phụ trách, thiết lập ngày bắt đầu và ngày đến hạn.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
title	VARCHAR(255)	NOT NULL	Tiêu đề Card.
description	TEXT	NULL	Mô tả chi tiết.
start_date	TIMESTAMP	NULL	Ngày bắt đầu.
due_date	TIMESTAMP	NULL	Ngày đến hạn hoàn thành.
is_completed	BOOLEAN	NOT NULL, DEFAULT FALSE	Trạng thái hoàn thành.
position	TEXT	NOT NULL	Giá trị vị trí dùng để sắp xếp và kéo thả.
list_id	UUID	NOT NULL, FK → lists, CASCADE	List chứa Card.
created_by	UUID	NOT NULL, FK → users	Người tạo Card.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 18: Bảng cards

3.2.3.13 Bảng card_members

a. Mô tả

Bảng `card_members` là bảng trung gian thể hiện quan hệ nhiều-nhiều giữa người dùng và Card. Mỗi bản ghi xác định một thành viên được gán làm người phụ trách cho Card.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
user_id	UUID	NOT NULL, FK → users	Thành viên được gán vào Card.
card_id	UUID	NOT NULL, FK → cards, CASCADE	Card được gán thành viên.
created_by	UUID	NOT NULL, FK → users	Người gán thành viên.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm gán thành viên.

Bảng 19: Bảng `card_members`

c. Ghi chú

Ràng buộc UNIQUE trên cặp (`user_id`, `card_id`) đảm bảo mỗi thành viên chỉ được gán một lần cho mỗi Card.

3.2.3.14 Bảng `checklists`

a. Mô tả

Bảng `checklists` lưu các danh sách kiểm tra bên trong Card. Mỗi Card có thể có nhiều Checklist, mỗi Checklist lại chứa nhiều ChecklistItem.

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên Checklist.
position	TEXT	NOT NULL	Vị trí hiển thị trong Card.
card_id	UUID	NOT NULL, FK → cards, CASCADE	Card chứa Checklist.
created_by	UUID	NOT NULL, FK → users	Người tạo Checklist.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 20: Bảng `checklists`

3.2.3.15 Bảng `checklist_items`

a. Mô tả

Bảng `checklist_items` lưu các hạng mục công việc con bên trong Checklist. Mỗi hạng mục có trạng thái hoàn thành riêng và có thể thiết lập ngày đến hạn độc lập. Frontend sử dụng tỷ lệ `is_completed` của các item để hiển thị thanh tiến độ (progress bar).

b. Cấu trúc bảng

Tên trường	Kiểu dữ liệu	Ràng buộc	Ý nghĩa
id	UUID	Khóa chính	Mã định danh.
name	VARCHAR(255)	NOT NULL	Tên hạng mục.
position	TEXT	NOT NULL	Vị trí hiển thị trong Checklist.
is_completed	BOOLEAN	NOT NULL, DEFAULT FALSE	Trạng thái hoàn thành của hạng mục.
due_date	TIMESTAMP	NULL	Thời hạn riêng của hạng mục.
checklist_id	UUID	NOT NULL, FK → checklists, CASCADE	Checklist chứa hạng mục.
created_by	UUID	NOT NULL, FK → users	Người tạo hạng mục.
updated_by	UUID	NULL, FK → users	Người cập nhật gần nhất.
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Thời điểm cập nhật gần nhất.

Bảng 21: Bảng checklist_items

c. Ghi chú

Ràng buộc CHECK trên trường is_completed đảm bảo chỉ nhận hai giá trị hợp lệ: true hoặc false.

3.2.4 Thiết kế giao diện

TODO: Bổ sung nội dung

4 CHƯƠNG 4: TRIỂN KHAI VÀ KẾT QUẢ

4.1 Các chức năng đã hoàn thành

Sau 8 tuần thực hiện (từ 30/03 đến 24/05/2026), hệ thống Nello đã hoàn thành **27/31 chức năng** yêu cầu, đạt tỷ lệ **87,1%**.

STT	Mã FR	Tên chức năng
Nhóm 1: Xác thực và quản lý tài khoản		
1	FR-01	Đăng ký tài khoản
2	FR-02	Xác nhận email
3	FR-03	Đăng nhập
4	FR-04	Đăng xuất
5	FR-05	Làm mới phiên làm việc
6	FR-32	Đổi mật khẩu
7	FR-33	Cập nhật hồ sơ cá nhân
Nhóm 2: Quản lý Workspace		
8	FR-06	Tạo Workspace
9	FR-07	Xem danh sách Workspace
10	FR-09	Quản lý thành viên Workspace
Nhóm 3: Quản lý Board		
11	FR-10	Tạo Board
12	FR-11	Xem Board
13	FR-13	Quản lý thành viên Board
Nhóm 4: Quản lý List		
14	FR-14	Tạo List
15	FR-15	Cập nhật List
16	FR-16	Xóa List
Nhóm 5: Quản lý Card		
17	FR-17	Tạo Card
18	FR-18	Xem chi tiết Card
19	FR-19	Cập nhật Card
20	FR-20	Xóa Card
21	FR-22	Gán thành viên vào Card
Nhóm 6: Kéo thả (Drag and Drop)		
22	FR-23	Kéo thả Card
23	FR-24	Kéo thả List trong Board
Nhóm 7: Quản lý Checklist		

STT	Mã FR	Tên chức năng
24	FR-26	Tạo Checklist
25	FR-27	Cập nhật Checklist
26	FR-28	Xóa Checklist
27	FR-29	Quản lý ChecklistItem

Bảng 22: Danh sách chức năng đã hoàn thành (27/31 — 87,1%)

4.2 Kết quả đạt được

4.2.1 Thống kê tổng quát

Hạng mục	Kết quả
Thời gian thực hiện	8 tuần (30/03/2026 – 24/05/2026)
Số API endpoint hoàn thành	43 endpoint trên 14 nhóm module (toàn bộ Backend)
Số màn hình Frontend	6 màn hình chính
Số test case đã kiểm thử	Hơn 30 test case
Trạng thái Backend	Hoàn chỉnh
Trạng thái Frontend	Hoàn chỉnh các chức năng cốt lõi
Môi trường triển khai	Local development

Bảng 23: Tổng hợp kết quả thực hiện

4.2.2 Kết quả kiểm thử theo module

Quá trình kiểm thử được thực hiện theo từng tuần phát triển. Phần lớn các test case tập trung vào luồng thành công (happy case) và một số trường hợp lỗi cơ bản.

Module Authentication (W3) — 14 test case

Mã TC	Kịch bản	Kết quả
TC01–TC03	Đăng ký: hợp lệ, trùng email, thiếu trường bắt buộc	Pass
TC04–TC05	Xác nhận OTP: mã đúng, mã sai	Pass
TC06–TC09	Đăng nhập: hợp lệ, sai mật khẩu, chưa xác nhận email, kiểm tra cookie	Pass
TC10–TC12	Làm mới token: hợp lệ, thiếu token, token hết hạn	Pass
TC13–TC14	Gửi lại mã: user hợp lệ, user không tồn tại	Pass

Bảng 24: Kết quả kiểm thử module Authentication

Module Workspace & Board (W4) — 16 test case

Mã TC	Kịch bản	Kết quả
TC01–TC05	Workspace: tạo mới, thiếu tên, token không hợp lệ, lấy danh sách, danh sách rỗng	Pass
TC06–TC08	Board: tạo mới, Workspace không tồn tại, không có quyền	Pass
TC09–TC12	Board: cập nhật, không tồn tại, xóa, xóa không có quyền	Pass
TC13–TC16	List: tạo mới, Board không tồn tại, cập nhật, lấy danh sách	Pass

Bảng 25: Kết quả kiểm thử module Workspace & Board

Module Card & Checklist (W5) — 9 test case

Mã TC	Kịch bản	Kết quả
TC17–TC22	Card: tạo mới, lấy chi tiết, cập nhật, di chuyển (drag-drop), xóa, lấy danh sách theo Board	Pass
TC23–TC25	Checklist: tạo mới, tạo ChecklistItem, cập nhật trạng thái hoàn thành	Pass

Bảng 26: Kết quả kiểm thử module Card & Checklist

Kiểm thử tích hợp Frontend (W6) — 9 test case

Mã TC	Kịch bản	Kết quả
TC01–TC03	Luồng xác thực: đăng ký → xác nhận OTP → đăng nhập	Pass
TC04–TC06	Xác thực lỗi: user đã tồn tại, sai mật khẩu, mã OTP sai	Pass
TC07–TC09	Dashboard: tải giao diện, tạo Workspace, tạo Board	Pass

Bảng 27: Kết quả kiểm thử tích hợp Frontend (W6)

Kiểm thử giao diện Board (W7) — 8 test case

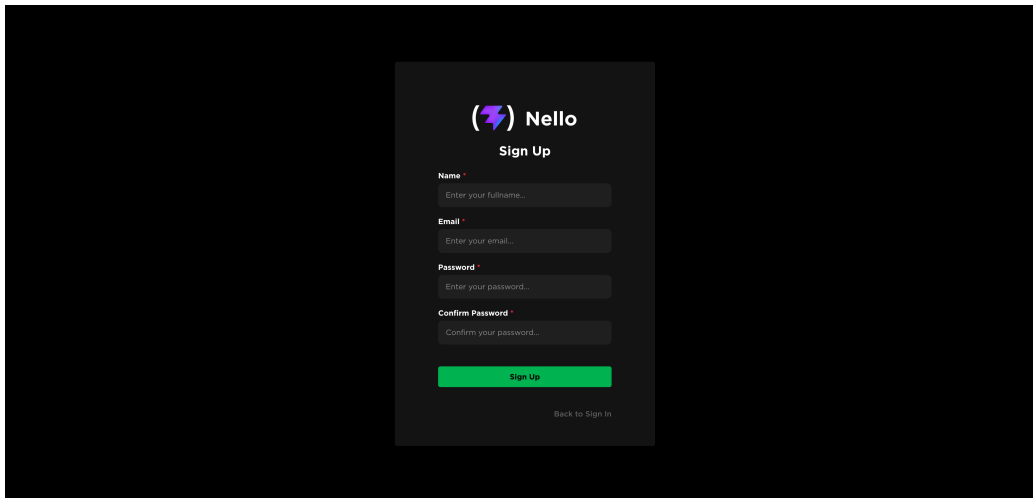
Mã TC	Kịch bản	Kết quả
TC01–TC02	Workspace Detail: tải giao diện, mời thành viên	Pass
TC04–TC08	Board Kanban: tải Board, tạo List, tạo Card, xóa Card, xóa List	Pass

Bảng 28: Kết quả kiểm thử giao diện Board (W7)

4.2.3 Các màn hình giao diện đã hoàn thành

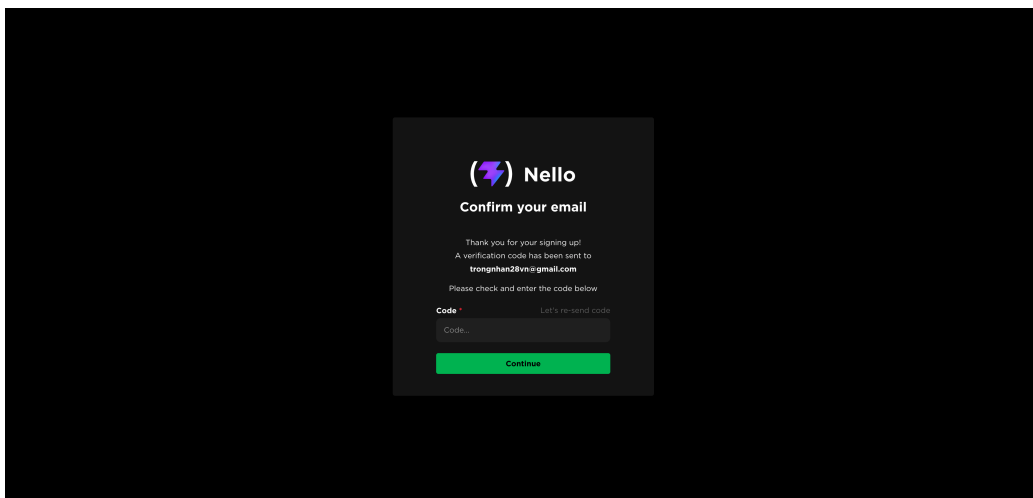
Hệ thống Frontend bao gồm 6 màn hình chính đã được phát triển và kiểm thử:

1. **Màn hình Đăng ký (/sign-up):** Form nhập họ tên, email, mật khẩu với validation phía Client; thông báo lỗi khi tài khoản đã tồn tại; tự động chuyển hướng sang màn hình xác nhận OTP sau khi đăng ký.



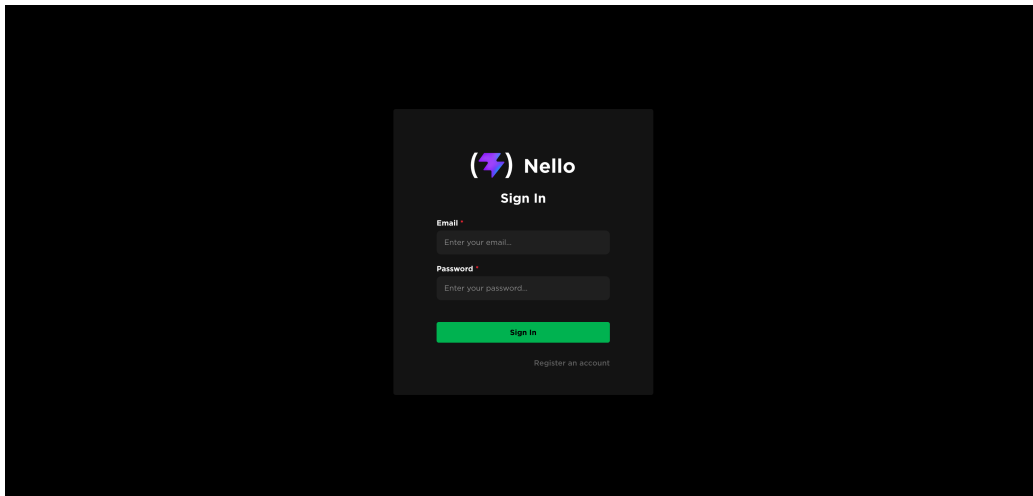
Hình 64: Màn hình Đăng ký tài khoản

2. **Màn hình Xác nhận OTP (/confirm-sign-up):** Ô nhập mã OTP 6 chữ số; nút gửi lại mã có bộ đếm thời gian; thông báo lỗi khi mã không hợp lệ; chuyển hướng về đăng nhập sau khi xác nhận.



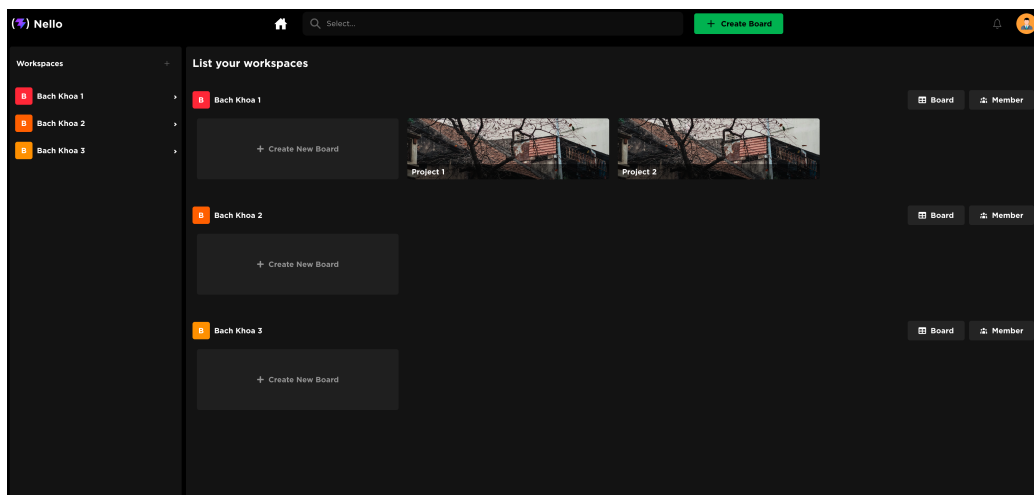
Hình 65: Màn hình Xác nhận OTP

3. **Màn hình Đăng nhập (/sign-in):** Form email và mật khẩu; thông báo lỗi khi thông tin không đúng; lưu token vào HttpOnly cookie; chuyển hướng đến Dashboard.



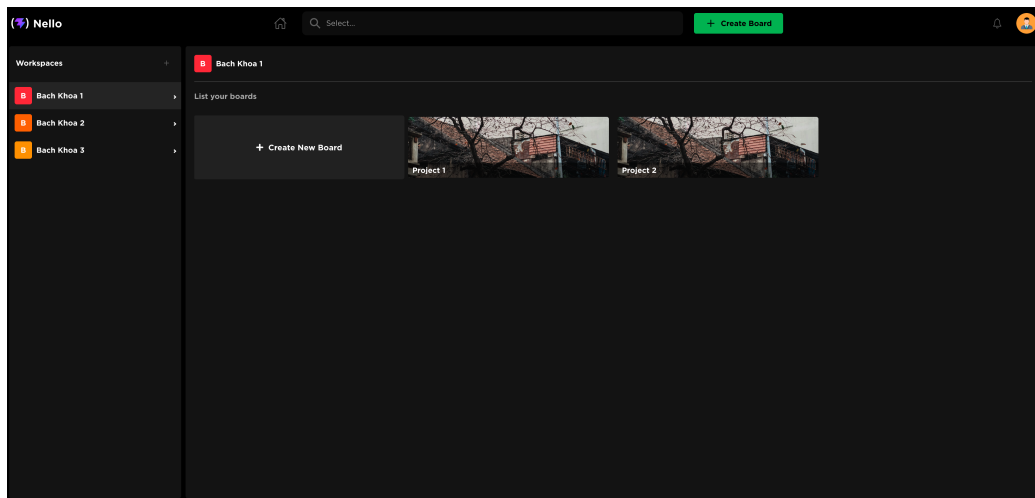
Hình 66: Màn hình Đăng nhập

4. **Dashboard:** Hiển thị danh sách Workspace trong sidebar; hiển thị Board theo từng Workspace dưới dạng thumbnail với ảnh nền; xử lý trạng thái trống khi chưa có dữ liệu; tự động làm mới token khi phiên hết hạn.



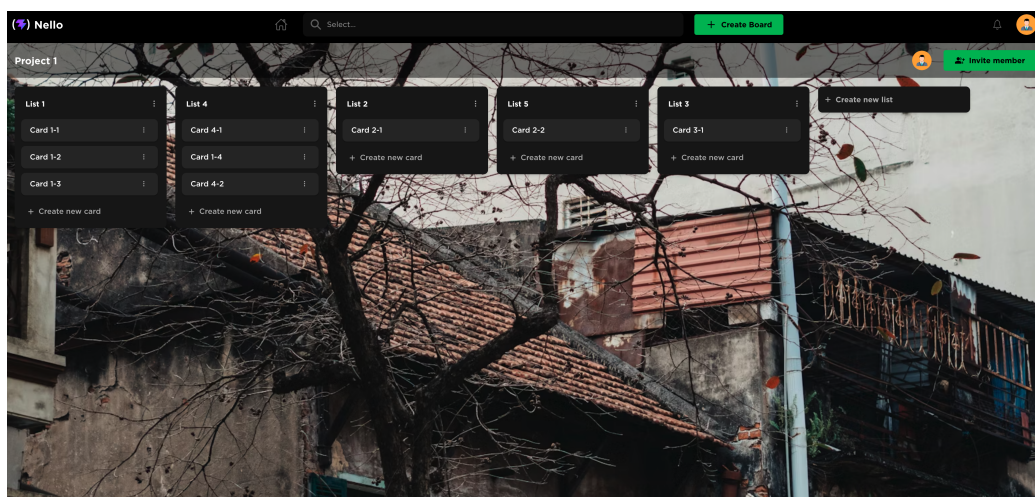
Hình 67: Màn hình Dashboard

5. **Màn hình Chi tiết Workspace (/workspaces/:id):** Thông tin Workspace, danh sách Board, danh sách thành viên với vai trò; modal mời thành viên bằng email.



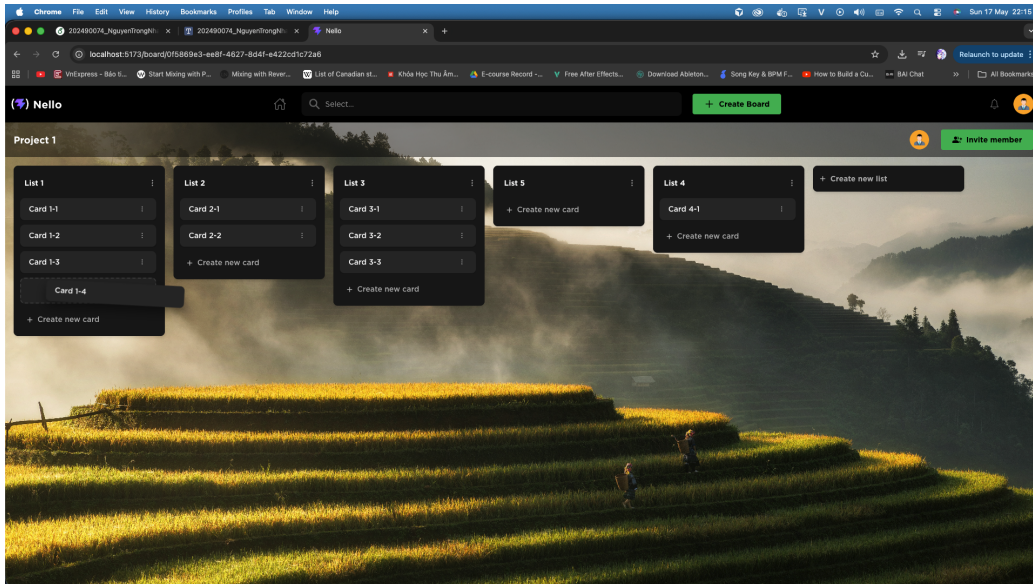
Hình 68: Màn hình Chi tiết Workspace

6. **Màn hình Board Kanban (/boards/:id):** Giao diện Kanban với các List theo dạng cột ngang; hiển thị Card trong từng List; form inline tạo List và Card; modal chi tiết Card; tính năng kéo thả List và Card; quản lý thành viên Board; badge cảnh báo Due Date; avatar thành viên trên Card.

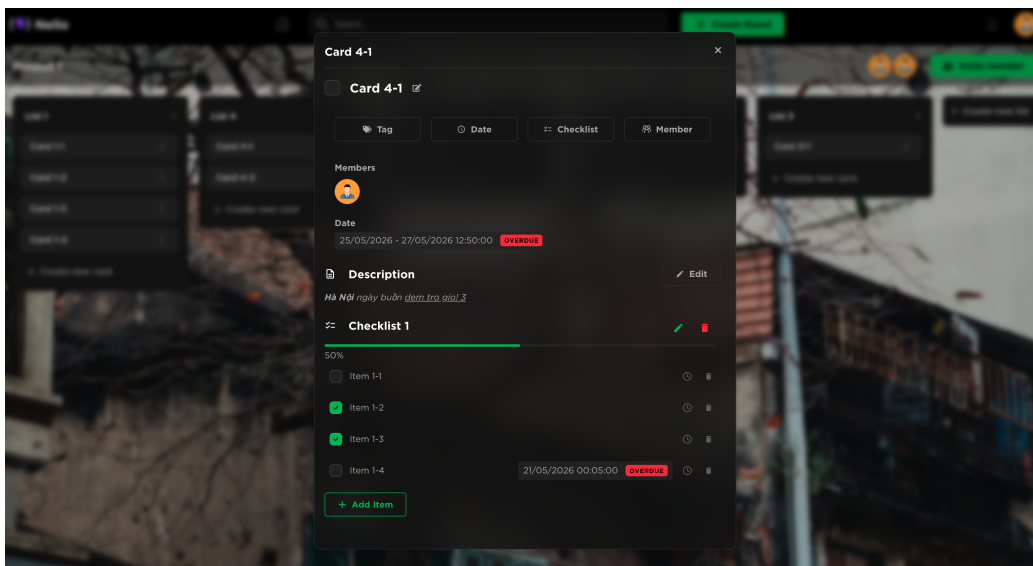


Hình 69: Màn hình Board Kanban

4.3 Hình ảnh minh họa



Hình 70: Tính năng kéo thả Card giữa các List



Hình 71: Modal Chi tiết Card với Checklist, Due Date và thành viên

4.4 Đánh giá kết quả

4.4.1 Ưu điểm

Qua quá trình phát triển và kiểm thử, hệ thống quản lý công việc Nello đạt được một số ưu điểm nổi bật:

- **Kiến trúc phân tách rõ ràng:** Frontend (ReactJS) và Backend (Spring Boot) hoàn toàn độc lập, tạo điều kiện bảo trì và mở rộng từng thành phần một cách linh hoạt.
- **Bảo mật theo tiêu chuẩn:** Toàn bộ xác thực được ủy thác cho Amazon Cognito. Token lưu trong HttpOnly cookie giúp bảo vệ khỏi tấn công XSS; tài nguyên Backend và cơ sở dữ liệu được cô lập trong mạng nội bộ VPC.
- **Trải nghiệm người dùng trực quan:** Giao diện Kanban quen thuộc với hỗ trợ kéo thả và cơ chế optimistic update cho phép người dùng thấy phản hồi tức thì mà không cần chờ phản hồi từ Backend.
- **Codebase chất lượng:** Áp dụng DTO và validation, xử lý ngoại lệ tập trung (Exception Handler), định dạng response chuẩn hóa trên toàn bộ API.

4.4.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống còn tồn tại một số hạn chế cần được khắc phục trong giai đoạn tiếp theo:

- **Một số chức năng chưa hoàn thiện:** Các API Backend cho phép cập nhật và xóa Workspace (FR-08, FR-30) và cập nhật Board (FR-12) đã được triển khai đầy đủ; tuy nhiên, giao diện Frontend tương ứng chưa được xây dựng. Người dùng hiện không thể thực hiện các thao tác này qua giao diện web mà chỉ có thể gọi API trực tiếp.
- **Kiểm thử chưa bao phủ đầy đủ các trường hợp biên và luồng lỗi:** Phần lớn các test case được thực hiện tập trung vào luồng thành công (happy case) và một số lỗi cơ bản. Các kịch bản lỗi nâng cao như dữ liệu vượt giới hạn, truy cập đồng thời, mất kết nối hoặc dữ liệu không nhất quán chưa được kiểm thử, dẫn đến nguy cơ tồn tại lỗi tiềm ẩn trong các điều kiện sử dụng thực tế.

- **Lỗi kéo thả ở vị trí đầu/cuối:** Thuật toán tính `position` dựa trên `beforeId/afterId` chưa xử lý đầy đủ các trường hợp biên khi kéo Card hoặc List về vị trí đầu tiên hoặc cuối cùng, dẫn đến sai lệch thứ tự hiển thị sau một số thao tác liên tiếp.
- **Phân quyền VIEWER chưa hoàn thiện:** Vai trò VIEWER được định nghĩa trong hệ thống nhưng chưa được áp dụng đầy đủ ở cả tầng Backend (kiểm tra quyền tại API endpoint) và tầng Frontend (ẩn/vô hiệu hóa các thao tác chỉnh sửa).
- **Chưa triển khai lên môi trường Staging:** Hệ thống hiện chỉ hoạt động trên môi trường phát triển cục bộ. Nguyên nhân chính là do xung đột cấu hình `HttpOnly` cookie khi Frontend và Backend được triển khai trên các domain khác nhau, đòi hỏi phải điều chỉnh chính sách `SameSite` và `CORS`.
- **Chưa có tính năng đồng bộ thời gian thực:** Hệ thống hoạt động theo mô hình request-response thuần túy. Khi một thành viên thực hiện thay đổi, các thành viên khác đang xem cùng Board cần tải lại trang thủ công để cập nhật dữ liệu, ảnh hưởng đến trải nghiệm cộng tác.

5 CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

5.1.1 Kết quả đạt được

Sau 8 tuần thực hiện (từ 30/03/2026 đến 24/05/2026), hệ thống quản lý công việc Nello đã được hoàn thiện với các số liệu triển khai cụ thể như sau:

Hạng mục	Kết quả
Thời gian thực hiện	8 tuần (30/03/2026 – 24/05/2026)
Số chức năng hoàn thành	27/31 chức năng (87,1%)
Số API endpoint hoàn thành	43 endpoint trên 14 nhóm module
Số màn hình Frontend	6 màn hình chính
Số test case đã kiểm thử	Hơn 30 test case (tỷ lệ pass 90%)
Trạng thái Backend	Hoàn chỉnh
Trạng thái Frontend	Hoàn chỉnh các chức năng cốt lõi
Môi trường triển khai	Local development

Bảng 29: Tổng hợp kết quả học phần Project I

Đối chiếu với các mục tiêu đề ra ban đầu, các kết quả cụ thể đạt được như sau:

- **Xây dựng nền tảng quản lý khoa học:** Hoàn thành. Hệ thống hỗ trợ đầy đủ việc tổ chức công việc theo cấu trúc phân cấp Workspace → Board → List → Card, đáp ứng nhu cầu quản lý dự án đa cấp.
- **Quản lý thẻ công việc chi tiết:** Hoàn thành. Card hỗ trợ đầy đủ các thuộc tính gồm tiêu đề, mô tả, ngày bắt đầu, ngày đến hạn, gán thành viên và danh sách kiểm tra (Checklist) với thanh tiến độ trực quan.
- **Tích hợp cơ chế tương tác trực quan:** Hoàn thành. Tính năng kéo thả hoạt động ổn định cho cả List và Card trong Board, với phản hồi giao diện tức thì thông qua cơ chế optimistic update.
- **Đồng bộ dữ liệu và cộng tác:** Hoàn thành một phần. Các cập nhật về trạng thái công việc, phân công nhân sự và thời hạn được đồng bộ hóa với Backend sau mỗi thao tác; tuy nhiên, đồng bộ thời gian thực giữa các thành viên chưa được triển khai.

Hệ thống đã chứng minh tính khả thi của việc xây dựng một ứng dụng quản lý công việc theo mô hình Kanban với kiến trúc phân tách Frontend/Backend, tích hợp dịch vụ đám mây AWS và đảm bảo các tiêu chuẩn bảo mật cơ bản.

5.2 Các vấn đề còn tồn tại

5.2.1 Chức năng chưa hoàn thiện

Một số chức năng đã được thiết kế và triển khai ở tầng Backend nhưng giao diện Frontend tương ứng chưa được xây dựng:

- Cập nhật và xóa Workspace (**FR-08, FR-30**)
- Cập nhật và xóa Board (**FR-12, FR-31**)

Ngoài ra, một số tính năng đã được xác định là cần thiết nhưng nằm ngoài phạm vi thực hiện ban đầu:

Tính năng	Mô tả
Quản lý nhãn dán (Tag)	Tạo, gắn, tìm kiếm và lọc Card theo Tag màu sắc.
Hệ thống thông báo	Thông báo khi Card được giao, khi gần đến hạn hoặc khi được mời vào Workspace/Board.
Nhật ký hoạt động	Ghi nhận lịch sử tạo, cập nhật, xóa dữ liệu; hiển thị timeline hoạt động trong Card.
Board View	Xem Board theo nhiều mode view khác nhau.
Upload tệp đính kèm	Đính kèm tệp tin vào Card thông qua Amazon S3.
Đồng bộ thời gian thực	Cập nhật tự động giao diện khi thành viên khác thay đổi dữ liệu trong cùng workspace.

Bảng 30: Các tính năng chưa được triển khai

5.2.2 Hạn chế kỹ thuật

- **Lỗi kéo thả ở vị trí đầu/cuối danh sách:** Thuật toán tính `position` dựa trên `beforeId/afterId` chưa xử lý đầy đủ các trường hợp biên khi kéo Card hoặc List về vị trí đầu tiên (không tồn tại `beforeId`) hoặc vị trí cuối cùng (không tồn tại `afterId`), dẫn đến sai lệch thứ tự hiển thị sau một số thao tác kéo thả liên tiếp.

- **Phân quyền VIEWER chưa hoàn thiện:** Vai trò VIEWER được định nghĩa trong cơ sở dữ liệu nhưng chưa được kiểm soát đầy đủ ở tầng Backend (thiếu middleware kiểm tra quyền trên các write operation) và tầng Frontend (chưa ẩn hoặc vô hiệu hóa các nút thao tác chỉnh sửa tương ứng).
- **Chưa triển khai lên môi trường Staging:** Hệ thống hiện chỉ vận hành trên môi trường phát triển cục bộ. Vấn đề kỹ thuật cốt lõi là xung đột cấu hình HttpOnly cookie khi Frontend và Backend được đặt trên các domain khác nhau, đòi hỏi phải điều chỉnh chính sách SameSite và cấu hình CORS phù hợp với môi trường production.
- **Chưa có đồng bộ thời gian thực:** Hệ thống hoạt động theo mô hình request-response thuần túy. Khi một thành viên thực hiện thay đổi, các thành viên khác đang xem cùng Board cần tải lại trang thủ công để nhận cập nhật mới nhất.
- **Kiểm thử chưa bao phủ đầy đủ:** Phần lớn test case tập trung vào luồng thành công và một số lỗi cơ bản. Các kịch bản nâng cao như dữ liệu vượt giới hạn, truy cập đồng thời, mất kết nối mạng và dữ liệu không nhất quán chưa được kiểm thử, tiềm ẩn nguy cơ lỗi trong môi trường thực tế.

5.3 Phương hướng phát triển

5.3.1 Ưu tiên cao — Hoàn thiện hệ thống hiện tại

- **Sửa lỗi thuật toán kéo thả:** Rà soát và hoàn thiện thuật toán tính beforeId/afterId, đảm bảo xử lý đúng tất cả các trường hợp biên bao gồm kéo vào đầu danh sách, cuối danh sách và danh sách trống; đồng thời đảm bảo tính nhất quán giữa trạng thái Frontend và dữ liệu Backend.
- **Hoàn thiện phân quyền VIEWER:** Bổ sung kiểm tra vai trò tại các API endpoint write operation ở tầng Backend và ẩn hoặc vô hiệu hóa các thành phần giao diện thao tác tương ứng khi người dùng có vai trò VIEWER.
- **Giải quyết vấn đề triển khai Staging:** Lựa chọn và triển khai chiến lược quản lý token phù hợp với môi trường cross-domain, bao gồm cấu hình SameSite=None; Secure cho cookie hoặc chuyển sang cơ chế lưu token trong Authorization header, sau đó tiến hành triển khai hệ thống lên môi trường Staging trên hạ tầng AWS đã thiết kế.

- **Hoàn thiện giao diện cập nhật và xóa Workspace/Board:** Xây dựng các màn hình và modal giao diện tương ứng cho các API FR-08, FR-12, FR-30, FR-31 đã có sẵn ở Backend.

5.3.2 Ưu tiên trung bình — Mở rộng tính năng

- **Đồng bộ thời gian thực (Realtime Websocket):** Tích hợp Spring WebSocket với giao thức STOMP vào Backend và SockJS/STOMP client vào Frontend để đẩy sự kiện cập nhật dữ liệu (tạo, cập nhật, xóa Card; thay đổi Checklist; kéo thả) đến toàn bộ thành viên đang hoạt động trên cùng Board, nâng cao đáng kể trải nghiệm cộng tác nhóm.
- **Quản lý nhãn dán (Tag):** Triển khai tính năng tạo, chỉnh sửa và xóa Tag với màu sắc tùy chọn; gắn nhiều Tag cho một Card; tìm kiếm và lọc Card theo Tag trong Board.
- **Hệ thống thông báo (Notification):** Xây dựng cơ chế thông báo trong ứng dụng khi Card được giao cho người dùng, khi công việc sắp đến hạn hoặc đã quá hạn, và khi được mời vào Workspace hoặc Board.

5.3.3 Ưu tiên thấp — Phát triển dài hạn

- **Nhật ký hoạt động (Activity Log):** Ghi nhận lịch sử toàn bộ thao tác tạo, cập nhật và xóa dữ liệu; hiển thị timeline hoạt động theo thời gian thực trong modal Card Detail và màn hình Board.
- **Các chế độ xem thay thế:** Phát triển Calendar View hiển thị Card theo due date trên lịch tháng và Timeline View trình bày tiến trình công việc theo dòng thời gian, giúp người dùng có thêm góc nhìn tổng quan về tiến độ dự án.
- **Upload tệp đính kèm:** Tích hợp khả năng đính kèm tệp tin vào Card thông qua Amazon S3, hỗ trợ xem trước và tải xuống trực tiếp từ giao diện Card Detail.

Tài liệu

- [1] Amazon Web Services. *Amazon Cognito Developer Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/cognito/latest/developerguide>. Ngày truy cập: 30/05/2026.
- [2] Amazon Web Services. *Amazon EC2 User Guide for Linux Instances*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide>. Ngày truy cập: 30/05/2026.
- [3] Amazon Web Services. *Amazon RDS User Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide>. Ngày truy cập: 30/05/2026.
- [4] Amazon Web Services. *Amazon S3 User Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AmazonS3/latest/userguide>. Ngày truy cập: 30/05/2026.
- [5] Amazon Web Services. *Amazon CloudFront Developer Guide*. 2024. Truy cập tại: <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide>. Ngày truy cập: 30/05/2026.
- [6] Amazon Web Services. *User Guide for Application Load Balancers*. 2024. Truy cập tại: <https://docs.aws.amazon.com/elasticloadbalancing/latest/application>. Ngày truy cập: 30/05/2026.
- [7] Atlassian. *Trello — Manage Your Team's Projects From Anywhere*. 2024. Truy cập tại: <https://trello.com>. Ngày truy cập: 30/05/2026.
- [8] Atlassian. *Jira Software — Plan, Track, and Release Great Software*. 2024. Truy cập tại: <https://www.atlassian.com/software/jira>. Ngày truy cập: 30/05/2026.

A PHỤ LỤC A: DANH SÁCH API END-POINT ĐẦY ĐỦ

Bảng dưới đây liệt kê toàn bộ API endpoint đã được triển khai, gồm 14 nhóm chức năng với tổng số hơn 40 endpoint. Tất cả endpoint đều có tiền tố `/api/v1`.

Module	Method	Endpoint	Mô tả
Authentication	POST	<code>/auth/sign-up</code>	Đăng ký tài khoản mới
Authentication	POST	<code>/auth/confirm-sign-up</code>	Xác nhận OTP kích hoạt tài khoản
Authentication	POST	<code>/auth/sign-in</code>	Đăng nhập, nhận JWT qua HttpOnly Cookie
Authentication	POST	<code>/auth/refresh-token</code>	Làm mới access token
Authentication	POST	<code>/auth/resent-code</code>	Gửi lại mã OTP
User	GET	<code>/users</code>	Tìm kiếm user theo email hoặc tên
User	GET	<code>/users/me</code>	Lấy thông tin user hiện tại từ token
Workspace	POST	<code>/workspaces</code>	Tạo workspace mới
Workspace	GET	<code>/workspaces</code>	Lấy danh sách workspace của user
Workspace	GET	<code>/workspaces/:id</code>	Lấy chi tiết workspace
Workspace	GET	<code>/workspaces/:id/boards</code>	Lấy danh sách board thuộc workspace
Workspace	GET	<code>/workspaces/:id/members</code>	Lấy danh sách thành viên workspace
Workspace Member	POST	<code>/workspaces-members/:id</code>	Mời thành viên vào workspace
Board	POST	<code>/boards</code>	Tạo board trong workspace
Board	GET	<code>/boards/:id</code>	Lấy chi tiết board kèm lists và cards
Board	PUT	<code>/boards/:id</code>	Cập nhật tên board
Board	DELETE	<code>/boards/:id</code>	Xóa board
Board Member	POST	<code>/boards-members</code>	Thêm thành viên vào board

(tiếp theo trang sau)

Module	Method	Endpoint	Mô tả
Board Member	GET	/board-members	Tìm kiếm thành viên trong board
List	POST	/lists	Tạo list trong board
List	PATCH	/lists/:id	Cập nhật tên hoặc vị trí list
List	DELETE	/lists/:id	Xóa list (cascade xóa cards)
List	GET	/lists?boardId=	Lấy danh sách list theo thứ tự vị trí
Card	POST	/cards	Tạo card trong list
Card	GET	/cards/:id	Lấy chi tiết card
Card	PATCH	/cards/:id	Cập nhật thông tin hoặc di chuyển card
Card	DELETE	/cards/:id	Xóa card
Card	GET	/cards?boardId=	Lấy danh sách card theo board
Checklist	POST	/checklists	Tạo checklist trong card
Checklist	PATCH	/checklists/:id	Cập nhật tên hoặc vị trí checklist
Checklist	DELETE	/checklists/:id	Xóa checklist (cascade xóa items)
Checklist	GET	/checklists?cardId=	Lấy danh sách checklist kèm items
Checklist Item	POST	/checklist-items	Tạo item trong checklist
Checklist Item	PATCH	/checklist-items/:id	Cập nhật hoặc đánh dấu hoàn thành
Checklist Item	DELETE	/checklist-items/:id	Xóa item
Card Member	POST	/card-members	Gán thành viên vào card
Card Member	DELETE	/card-members/:id	Hủy gán thành viên khỏi card
Image	GET	/images	Lấy danh sách 8 ảnh nền board
WorkspaceCategory	GET	/workspace-categories	Lấy 8 danh mục workspace
Role	GET	/roles	Lấy danh sách vai trò (ADMIN, MEMBER, VIEWER)

Bảng 31: Danh sách API endpoint đầy đủ của hệ thống

B PHỤ LỤC B: CHI TIẾT BẢNG CƠ SỞ DỮ LIỆU

Hệ thống sử dụng PostgreSQL với 13 bảng dữ liệu, quản lý migration bằng Flyway (`V1__init.sql`). Khóa chính của các bảng nghiệp vụ sử dụng kiểu UUID; các bảng tham chiếu dùng INT.

Bảng roles

Cột	Kiểu dữ liệu	Ràng buộc
id	INT	PRIMARY KEY
name	role_name_enum	UNIQUE NOT NULL — ADMIN, MEMBER, VIEWER

Bảng images

Cột	Kiểu dữ liệu	Ràng buộc
id	INT	PRIMARY KEY
key	TEXT	NOT NULL — đường dẫn ảnh nền board

Bảng workspace_categories

Cột	Kiểu dữ liệu	Ràng buộc
id	INT	PRIMARY KEY
name	VARCHAR(255)	NOT NULL

Bảng users

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
cognito_id	VARCHAR(255)	NOT NULL, UNIQUE
full_name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	NOT NULL, UNIQUE
avatar_url	VARCHAR(255)	nullable
bio	TEXT	nullable
phone	VARCHAR(255)	nullable
address	VARCHAR(255)	nullable
deleted_at	TIMESTAMP	Soft delete
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng workspaces

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
description	VARCHAR(255)	nullable
category_id	INT	FK → workspace_categories(id)
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng workspace_members

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
user_id	UUID	FK → users(id) ON DELETE CASCADE
workspace_id	UUID	FK → workspaces(id) ON DELETE CASCADE
role_id	INT	FK → roles(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
UNIQUE (user_id, workspace_id)		

Bảng boards

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
background_url	TEXT	nullable
workspace_id	UUID	FK → workspaces(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng board_members

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
user_id	UUID	FK → users(id) ON DELETE CASCADE
board_id	UUID	FK → boards(id) ON DELETE CASCADE
role_id	INT	FK → roles(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
UNIQUE (user_id, board_id)		

Bảng lists

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
board_id	UUID	FK → boards(id) ON DELETE CASCADE
position	TEXT	NOT NULL — fractional indexing
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng cards

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
title	VARCHAR(255)	NOT NULL
description	TEXT	nullable
start_date	TIMESTAMP	nullable
due_date	TIMESTAMP	nullable
is_completed	BOOLEAN	DEFAULT FALSE
position	TEXT	NOT NULL — fractional indexing
list_id	UUID	FK → lists(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng card_members

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
user_id	UUID	FK → users(id) ON DELETE CASCADE
card_id	UUID	FK → cards(id) ON DELETE CASCADE
created_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
UNIQUE (user_id, card_id)		

Bảng checklists

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
card_id	UUID	FK → cards(id) ON DELETE CASCADE
position	TEXT	NOT NULL
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Bảng checklist_items

Cột	Kiểu dữ liệu	Ràng buộc
id	UUID	PRIMARY KEY
name	VARCHAR(255)	NOT NULL
checklist_id	UUID	FK → checklists(id) ON DELETE CASCADE
position	TEXT	NOT NULL
is_completed	BOOLEAN	NOT NULL DEFAULT FALSE
due_date	TIMESTAMP	nullable
created_by	UUID	FK → users(id) ON DELETE SET NULL
updated_by	UUID	FK → users(id) ON DELETE SET NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

C PHỤ LỤC C: DANH SÁCH KỊCH BẢN KIỂM THỬ

Hệ thống đã được kiểm thử qua 5 đợt, bao phủ tổng cộng hơn 50 kịch bản kiểm thử. Toàn bộ kịch bản đều đạt kết quả Pass tại thời điểm kết thúc học phần.

Module Authentication (W3) — 14 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Đăng ký tài khoản với thông tin hợp lệ	Pass
TC02	Đăng ký với email đã tồn tại trong hệ thống	Pass
TC03	Đăng ký với thiếu trường bắt buộc	Pass
TC04	Xác nhận OTP với mã đúng	Pass
TC05	Xác nhận OTP với mã sai	Pass
TC06	Đăng nhập với thông tin hợp lệ	Pass
TC07	Đăng nhập với mật khẩu sai	Pass
TC08	Đăng nhập với tài khoản chưa xác nhận email	Pass
TC09	Kiểm tra token được lưu vào HttpOnly Cookie	Pass
TC10	Làm mới access token với refresh token hợp lệ	Pass
TC11	Làm mới token khi thiếu refresh token	Pass
TC12	Làm mới token khi token đã hết hạn	Pass
TC13	Gửi lại mã OTP cho user hợp lệ	Pass
TC14	Gửi lại mã OTP cho user không tồn tại	Pass

Module Workspace & Board (W4) — 16 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Tạo workspace với thông tin hợp lệ	Pass
TC02	Tạo workspace với tên trống	Pass
TC03	Tạo workspace với token xác thực sai	Pass
TC04	Lấy danh sách workspace của user hiện tại	Pass
TC05	Lấy danh sách workspace khi chưa có workspace nào	Pass
TC06	Tạo board trong workspace hợp lệ	Pass
TC07	Tạo board trong workspace không tồn tại	Pass
TC08	Tạo board khi user không có quyền trong workspace	Pass
TC09	Cập nhật tên board	Pass
TC10	Cập nhật board không tồn tại	Pass
TC11	Xóa board	Pass
TC12	Xóa board khi không có quyền	Pass
TC13	Tạo list trong board hợp lệ	Pass
TC14	Tạo list trong board không tồn tại	Pass
TC15	Cập nhật tên list	Pass
TC16	Lấy danh sách list theo board	Pass

Module Card & Checklist (W5) — 9 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC17	Tạo card trong list hợp lệ	Pass
TC18	Lấy chi tiết card	Pass
TC19	Cập nhật thông tin card (title, description, due-Date)	Pass
TC20	Di chuyển card giữa các List (drag-and-drop)	Pass
TC21	Xóa card	Pass
TC22	Lấy danh sách card theo board	Pass
TC23	Tạo checklist trong card	Pass
TC24	Tạo checklist item trong checklist	Pass
TC25	Cập nhật trạng thái hoàn thành của checklist item	Pass

Frontend Integration (W6) — 9 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Luồng xác thực đầy đủ: Đăng ký → Xác nhận OTP → Đăng nhập	Pass
TC02	Thông báo lỗi khi đăng ký với user đã tồn tại	Pass
TC03	Thông báo lỗi khi đăng nhập với mật khẩu sai	Pass
TC04	Thông báo lỗi khi xác nhận OTP sai	Pass
TC05	Tải màn hình Dashboard và hiển thị danh sách workspace	Pass
TC06	Tạo workspace mới từ Dashboard	Pass
TC07	Tạo board mới trong workspace	Pass
TC08	Xử lý empty state khi chưa có workspace	Pass
TC09	Auto refresh token khi access token hết hạn	Pass

Board UI (W7) — 8 kịch bản

Mã TC	Kịch bản kiểm thử	Kết quả
TC01	Tải màn hình Workspace Detail và hiển thị danh sách board và thành viên	Pass
TC02	Mời thành viên vào workspace qua modal	Pass
TC03	Tải màn hình Board Kanban với đầy đủ List và Card	Pass
TC04	Tạo List mới trong Board	Pass
TC05	Tạo Card mới trong List	Pass
TC06	Xóa Card với modal xác nhận	Pass
TC07	Xóa List với modal xác nhận	Pass
TC08	Mở Card Detail Modal và hiển thị Checklist, Due Date, Members	Pass